Available online at www.sciencedirect.com
ScienceDirect
Theoretical Computer Science 375 (2007) 41–75
Theoretical Computer Science
www.elsevier.com/locate/tcs

On the relations between monadic semantics

Andrzej Filinski

Department of Computer Science, University of Copenhagen, Universitetsparken 1, DK-2100 Copenhagen, Denmark

Abstract

We present a simple computational metalanguage with general recursive types and multiple notions of effects, through which a variety of concrete denotational semantics can be conveniently factored, by suitably interpreting the effects as monads. We then propose a methodology for relating two such interpretations of the metalanguage, with the aim of showing that the semantics they induce agree for complete programs. As a prototypical instance of such a relation, we use the framework to show agreement between a direct and a continuation semantics of the simple, untyped functional language from Reynolds's original paper on the subject.
© 2007 Elsevier B.V. All rights reserved.

Keywords: Monads; Computational metalanguage; Logical relations; Recursive types; Continuations

1. Introduction

It is an honor to contribute to this Festschrift for John Reynolds on the occasion of his 70th birthday. It has been said that much of Western philosophy can be characterized as a series of footnotes to Plato; arguably, Reynolds's work holds a position of similar prominence in the field of programming language semantics. In this spirit, we revisit his 1974 paper, *On the Relation between Direct and Continuation Semantics* [30]. Our aim is to show that, with just a few minor adjustments and generalizations, the constructions presented there are still as relevant as ever for comparing different denotational semantics of the same language.

1.1. Background

The defining characteristic of a denotational semantics is a clear, syntax-neutral characterization of program *meanings*: every syntactic phrase (e.g., term or type) has an inherent meaning, independent of its context, and phrases can be considered equivalent precisely when they have the same meaning. One would expect functional languages to have a particularly simple such semantics, but it turns out that even a seemingly very simple such language – the pure, untyped lambda-calculus – presents a significant technical obstacle, because functions can effectively be passed themselves as parameters. Thus, in order to avoid the evident cardinality paradox, meanings of terms must belong to a mathematical structure more complicated than sets and (partial) functions between them. At the same time, however,

E-mail address: andrzej@diku.dk.

0304-3975/$ - see front matter © 2007 Elsevier B.V. All rights reserved.
doi:10.1016/j.tcs.2006.12.027

one wants to retain the intuition that a lambda-abstraction does indeed denote a mathematical function; that is, the semantic model should be a reasonable refinement of the simple, extensional model of functional programming.

That this is feasible at all relies on a body of work known as domain theory [32]. It is rather remarkable that the same general framework used to account for simple recursion at the term level, essentially also suffices for constructing general reflexive domains, i.e., those that contain a copy of their own function space. More precisely, to model term recursion by fixed points, one is naturally led to the notion of domains and continuous functions between them. The apparent cardinality problem in the untyped case is then resolved by the observation that there are not nearly as many continuous functions as set-theoretic ones, which makes it possible to construct a domain $D \cong \mathbf{N} + (D \to D)$, where $\to$ represents the space of continuous functions. Given the existence of such domains, it is quite straightforward to write down simple, and intuitively understandable, denotational semantics of most languages, typed or untyped. It is only when the time comes to formally reason about programs with (explicitly or implicitly) reflexive types that the true underlying complexity of the concept reveals itself.

An important instance of formal reasoning about semantics is proving that two denotational semantics actually define the same language, with respect to *observable* program behavior. That is, the denotations of complete, executable programs should agree, even though the meanings of subphrases may be quite different in the two semantics. The distinction is particularly well exemplified by the relationship between direct and continuation semantics of a language, which was first explored by Reynolds in the setting of interpreters [29], but is easily abstracted into a question of showing agreement between denotations.

Both semantics look fairly innocuous, and one might expect the task of showing that they define the same language to be a simple exercise. Unfortunately, this is not the case: the standard approach of proving properties of terms by structural induction is blocked by the apparent lack of a suitable induction hypothesis for relating direct and continuation meanings of functional terms. For typed languages, such relationships are usually expressed in terms of *logical relations*, defined by an auxiliary induction on the type structure, but this is evidently not an option for an untyped language. However, in an influential paper [30], Reynolds showed how to express the natural logical-relations argument in a setting where the relation is not type-indexed, but needs to have its existence proved explicitly, in much the same way as the underlying reflexive domains were constructed in the first place. The paper was very much a tour de force at the time, and is still frequently referred to as a cautionary example of how hard it is to reason about denotational semantics.

Over the subsequent 30+ years, the theory and practice of denotational semantics has evolved somewhat. The domain theory was streamlined: instead of the original complete lattices (notably with "tops"), one now usually considers the simpler structure of ω-cpos (i.e., only containing lubs of all ω-chains); this condition suffices for most applications. Two specific developments have had a particularly significant impact, however: the theories of *invariant relations* given by Pitts [26], and of *monads* given by Moggi [19]. The former established that – under rather mild requirements – recursive "definitions" of relations are as well behaved as recursive definitions of terms and types; that is, as long as relations are defined in terms of themselves using a fairly generous collection of operations, and the underlying domain-equation solutions are *minimal invariants*, the appropriate relational fixed points exist. The latter showed that a surprisingly wide variety of notions of "effectful" computations could be unified in a general framework. In particular, both direct and continuation semantics are natural instances of the monadic approach.

The goal of this paper is therefore twofold: to *update* Reynolds's seminal work in light of these developments, and to *generalize* it to allow reasoning about other (explicitly or implicitly) monadic-style semantics. Our main technical contribution is essentially an extension of previous work on simulating one monad by another [7,8] – which uses Reynolds's relational architecture for dealing with term recursion, but confined to a simply typed setting – to general recursive types. That is, we formally state and show that nothing essentially new happens: the constructions that work for simply typed languages generalize robustly to recursively typed ones.

These results allow us to reason not only about languages or programs with overtly recursive types, but also about ones that are nominally simply typed, but where recursion is hidden inside a monad, such as list or stream monads for modeling backtracking, or recursively typed state monads for modeling storable, store-accessing procedures. For reasons of space and scope, we concentrate on the framework itself, and consider just the single application of showing agreement between Reynolds's original (although slightly reformulated) two semantics for a very simple language. However, the intent is that the framework could also be directly employed for showing agreement of much more realistic languages, with complicated effect structures implemented in significantly different ways. Some remaining challenges in this direction are noted in the final section.

1.2. A sample functional language

As a motivating instance of the general problem of showing agreement between two semantics, we consider a very simple, untyped functional language — essentially that of Reynolds's original paper, but with a more concrete observation model. Its syntax is as follows:

$$t ::= \underline{n} \mid \mathrm{succ} \mid x \mid \lambda^{\mathrm{n}} x.t \mid \lambda^{\mathrm{v}} x.t \mid t_1\, t_2 \quad [n \in \mathbf{N}, x \in \mathbf{V}].$$

The absence of a zero-test or other arithmetic operations on numbers is deliberate: the numeric constants are intended for observation of final results only; any non-trivial computations should be expressed in terms of the usual constructions from the untyped lambda-calculus (Church encodings, the Y-combinator, etc.). The only other slightly unusual aspect of the language is that it supports both call-by-name and call-by-value function parameters.

To cement the intuition about the meanings of programs in the language, we briefly present its intended operational semantics. First, we designate a subset of closed terms as *values*:

$$v ::= \underline{n} \mid \mathrm{succ} \mid \lambda^{\mathrm{n}} x.t \mid \lambda^{\mathrm{v}} x.t.$$

Then, for closed t, we define the *evaluation judgment* $t \Downarrow v$ through the following rules:

$$\frac{}{v \Downarrow v} \qquad \frac{t_1 \Downarrow \mathrm{succ} \qquad t_2 \Downarrow \underline{n}}{t_1\, t_2 \Downarrow \underline{n+1}} \qquad \frac{t_1 \Downarrow \lambda^{\mathrm{n}} x.t_1' \qquad t_1'[t_2/x] \Downarrow v}{t_1\, t_2 \Downarrow v}$$

$$\frac{t_1 \Downarrow \lambda^{\mathrm{v}} x.t_1' \qquad t_2 \Downarrow v_2 \qquad t_1'[v_2/x] \Downarrow v}{t_1\, t_2 \Downarrow v}.$$

This semantics makes no formal distinction between terms that diverge and those that do not evaluate to anything because of a type error (trying to take the successor of a non-number, or to apply a non-function to an argument), but one could easily define a variant returning an explicit error token in the latter case.

A *(complete) program* is simply a closed term. We say that a program t returns the result n if $t \Downarrow \underline{n}$; programs that evaluate to non-numeric results are considered erroneous, in the same sense as above.

1.3. Two denotational semantics

We now consider two denotational semantics of our sample language, very similar to Reynolds's original ones, but slightly updated and simplified. Most notably, we work in the setting of cpos, which need not contain either $\bot$- or $\top$-elements. We generally use Winskel's notation [37] for domain-theoretic constructs: $\lfloor\cdot\rfloor$ is the inclusion from a cpo A to the lifted $A_\bot$, and for a function $f : A \to D$ (with D pointed, i.e., having a least element), $f^* : A_\bot \to D$ is its strict extension. However, we will *not* follow Winskel's convention of writing the continuous function space as $[A \to B]$, as distinct from the space of all set-theoretic functions $A \to B$; all functions in the following are continuous, unless otherwise qualified.

As already noted, we give two denotational semantics of the language. The first, *direct* one is shown in Fig. 1. It is formulated in terms of a domain (i.e., pointed cpo) D^{d} of term denotations, and a predomain (i.e., bottomless cpo) E^{d} of value denotations. The latter arises as a solution of the evident predomain equation; we write $\iota_{\mathrm{n}}^{\mathrm{d}}$ ($\iota_{\mathrm{f}}^{\mathrm{d}}$) for the composition of the left (right) injection with the solution isomorphism. The equations defining the meanings of terms, $[\![t]\!]^{\mathrm{d}}$, and of complete programs, $[\![t]\!]^{\mathrm{D}}$, are then straightforward.

The intent is that, for a complete program t, $t \Downarrow \underline{n}$ iff $[\![t]\!]^{\mathrm{D}} = \lfloor n \rfloor$. We will not formally prove this property here, but the relevant arguments are essentially those of Plotkin for call-by-name PCF [28], as extended to untyped settings by Pitts [25], and straightforwardly completed with an additional case for call-by-value abstractions. Briefly, the "only-if" direction (soundness) follows a straightforward induction on the derivation of $t \Downarrow v$, but "if" (computational adequacy) requires a generalized logical-relations construction – closely related to the one in the present paper, in fact, but too peripheral to our main topic.

Instead, we will consider a second denotational semantics, based on *continuations*, as shown in Fig. 2. Again, our formulation differs only in immaterial ways from Reynolds's. The domain of term denotations D^{c} is now given in terms of continuations mapping value-denotations from E^{c} to *final answers* from a domain O. This more general view of term behaviors allows us to extend the language with a variety of additional constructs, such as control operators. More immediately, however, it also makes it possible to distinguish formally between divergence and type errors, by

$$
\begin{aligned}
D^{\mathrm{d}} &= E^{\mathrm{d}}_{\perp} \\
E^{\mathrm{d}} &\cong \mathbf{N} + (D^{\mathrm{d}} \to D^{\mathrm{d}}) \quad [\iota^{\mathrm{d}}_{\mathrm{n}} : \mathbf{N} \to E^{\mathrm{d}}, \iota^{\mathrm{d}}_{\mathrm{f}} : (D^{\mathrm{d}} \to D^{\mathrm{d}}) \to E^{\mathrm{d}}] \\
[\![t]\!]^{\mathrm{d}} &\in (\mathbf{V} \to D^{\mathrm{d}}) \to D^{\mathrm{d}} \\
[\![\underline{n}]\!]^{\mathrm{d}} &= \lambda\rho.\lfloor \iota^{\mathrm{d}}_{\mathrm{n}}(n) \rfloor \\
[\![\mathrm{succ}]\!]^{\mathrm{d}} &= \lambda\rho.\lfloor \iota^{\mathrm{d}}_{\mathrm{f}}(\lambda d.(\lambda e.\mathrm{case}\ e\ \mathrm{of} \begin{Bmatrix} \iota^{\mathrm{d}}_{\mathrm{n}}(n) \to \lfloor \iota^{\mathrm{d}}_{\mathrm{n}}(n+1) \rfloor \\ \iota^{\mathrm{d}}_{\mathrm{f}}(f) \to \perp_{D^{\mathrm{d}}} \end{Bmatrix})^* d) \rfloor \\
[\![x]\!]^{\mathrm{d}} &= \lambda\rho.\rho\, x \\
[\![\lambda^{\mathrm{n}} x.t]\!]^{\mathrm{d}} &= \lambda\rho.\lfloor \iota^{\mathrm{d}}_{\mathrm{f}}(\lambda d.[\![t]\!]^{\mathrm{d}}\rho[x \mapsto d]) \rfloor \\
[\![\lambda^{\mathrm{v}} x.t]\!]^{\mathrm{d}} &= \lambda\rho.\lfloor \iota^{\mathrm{d}}_{\mathrm{f}}(\lambda d.(\lambda e.[\![t]\!]^{\mathrm{d}}\rho[x \mapsto \lfloor e \rfloor])^* d) \rfloor \\
[\![t_1\, t_2]\!]^{\mathrm{d}} &= \lambda\rho.(\lambda e.\mathrm{case}\ e\ \mathrm{of} \begin{Bmatrix} \iota^{\mathrm{d}}_{\mathrm{n}}(n) \to \perp_{D^{\mathrm{d}}} \\ \iota^{\mathrm{d}}_{\mathrm{f}}(f) \to f\,([\![t_2]\!]^{\mathrm{d}}\rho) \end{Bmatrix})^* ([\![t_1]\!]^{\mathrm{d}}\rho) \\
[\![t]\!]^{\mathrm{D}} &\in \mathbf{N}_{\perp} \quad [t \text{ closed}] \\
[\![t]\!]^{\mathrm{D}} &= (\lambda e.\mathrm{case}\ e\ \mathrm{of} \begin{Bmatrix} \iota^{\mathrm{d}}_{\mathrm{n}}(n) \to \lfloor n \rfloor \\ \iota^{\mathrm{d}}_{\mathrm{f}}(f) \to \perp_{\mathbf{N}_{\perp}} \end{Bmatrix})^* ([\![t]\!]^{\mathrm{d}}(\lambda x.\perp_{D^{\mathrm{d}}}))
\end{aligned}
$$

Fig. 1. Direct semantics of the sample language.

$$
\begin{aligned}
D^{\mathrm{c}} &= (E^{\mathrm{c}} \to O) \to O \quad [O \text{ pointed}, o_{\mathrm{err}} \in O, k_{\mathrm{init}} \in \mathbf{N} \to O] \\
E^{\mathrm{c}} &\cong \mathbf{N} + (D^{\mathrm{c}} \to D^{\mathrm{c}}) \quad [\iota^{\mathrm{c}}_{\mathrm{n}} : \mathbf{N} \to E^{\mathrm{c}}, \iota^{\mathrm{c}}_{\mathrm{f}} : (D^{\mathrm{c}} \to D^{\mathrm{c}}) \to E^{\mathrm{c}}] \\
[\![t]\!]^{\mathrm{c}} &\in (\mathbf{V} \to D^{\mathrm{c}}) \to D^{\mathrm{c}} \\
[\![\underline{n}]\!]^{\mathrm{c}} &= \lambda\rho.\lambda k.k\,(\iota^{\mathrm{c}}_{\mathrm{n}}(n)) \\
[\![\mathrm{succ}]\!]^{\mathrm{c}} &= \lambda\rho.\lambda k.k\,(\iota^{\mathrm{c}}_{\mathrm{f}}(\lambda d.\lambda k.d\,(\lambda e.\mathrm{case}\ e\ \mathrm{of} \begin{Bmatrix} \iota^{\mathrm{c}}_{\mathrm{n}}(n) \to k\,(\iota^{\mathrm{c}}_{\mathrm{n}}(n+1)) \\ \iota^{\mathrm{c}}_{\mathrm{f}}(f) \to o_{\mathrm{err}} \end{Bmatrix}))) \\
[\![x]\!]^{\mathrm{c}} &= \lambda\rho.\lambda k.\rho\, x\, k \\
[\![\lambda^{\mathrm{n}} x.t]\!]^{\mathrm{c}} &= \lambda\rho.\lambda k.k\,(\iota^{\mathrm{c}}_{\mathrm{f}}(\lambda d.\lambda k.[\![t]\!]^{\mathrm{c}}\rho[x \mapsto d]\, k)) \\
[\![\lambda^{\mathrm{v}} x.t]\!]^{\mathrm{c}} &= \lambda\rho.\lambda k.k\,(\iota^{\mathrm{c}}_{\mathrm{f}}(\lambda d.\lambda k.d\,(\lambda e.[\![t]\!]^{\mathrm{c}}\rho[x \mapsto \lambda k'.k' e]\, k))) \\
[\![t_1\, t_2]\!]^{\mathrm{c}} &= \lambda\rho.\lambda k.[\![t_1]\!]^{\mathrm{c}}\rho\,(\lambda e.\mathrm{case}\ e\ \mathrm{of} \begin{Bmatrix} \iota^{\mathrm{c}}_{\mathrm{n}}(n) \to o_{\mathrm{err}} \\ \iota^{\mathrm{c}}_{\mathrm{f}}(f) \to f([\![t_2]\!]^{\mathrm{c}}\rho)\, k \end{Bmatrix}) \\
[\![t]\!]^{\mathrm{C}} &\in O \quad [t \text{ closed}] \\
[\![t]\!]^{\mathrm{C}} &= [\![t]\!]^{\mathrm{c}}(\lambda x.\lambda k.o_{\mathrm{err}})\,(\lambda e.\mathrm{case}\ e\ \mathrm{of} \begin{Bmatrix} \iota^{\mathrm{c}}_{\mathrm{n}}(n) \to k_{\mathrm{init}}\, n \\ \iota^{\mathrm{c}}_{\mathrm{f}}(f) \to o_{\mathrm{err}} \end{Bmatrix})
\end{aligned}
$$

Actual: $O = (\mathbf{N}+1)_{\perp}$, $o_{\mathrm{err}} = \lfloor in_2(*) \rfloor$, $k_{\mathrm{init}} = \lambda n.\lfloor in_1(n) \rfloor$.
Simplified variant: $O = \mathbf{N}_{\perp}$, $o_{\mathrm{err}} = \perp$, $k_{\mathrm{init}} = \lambda n.\lfloor n \rfloor$.

Fig. 2. Continuation semantics of sample language.

designating a particular answer element $o_{\mathrm{err}} \in O$ as representing an aborted computation. Successful termination is modeled by applying the initial continuation k_{init} to the number intended as the final result, so we normally require that $\forall n \in \mathbf{N}.\, k_{\mathrm{init}}\, n \neq o_{\mathrm{err}}$.

If we had considered the simplified variant of the continuation semantics (i.e., with $O = \mathbf{N}_{\perp}$), which identifies divergence and errors, we would expect a complete agreement between their observable results, i.e., for closed t,

$[\![t]\!]^{\mathrm{D}} = [\![t]\!]^{\mathrm{C}}$. However, we will actually look at a slightly more complicated notion of agreement between the two semantics, where type errors give an observably different result from divergence in the continuation-based one. (We could also have distinguished further between different kinds of errors: using a function as a number and vice versa; the adaptations are straightforward.) To state the agreement in this setting, let $Abs : (\mathbf{N}+1)_\perp \to \mathbf{N}_\perp$ be given by

$$Abs(\perp) = Abs(\lfloor in_2(*)\rfloor) = \perp \qquad Abs(\lfloor in_1(n)\rfloor) = \lfloor n\rfloor\,,$$

i.e., the function that identifies observations of error results with divergence. We then expect to show the following result:

Theorem 1 (*Agreement of Semantics*). *For closed t, $[\![t]\!]^{\mathrm{D}} = Abs([\![t]\!]^{\mathrm{C}})$.*

1.4. Showing agreement

When proving the above theorem, we encounter three primary sources of complications:

(1) The semantics are expressed in terms of reflexive domains, about which we will ultimately need to know more than just that they are solutions of the evident domain equations. While this is a major technical problem, it is essentially orthogonal to the specific relationship between the two semantics.
(2) The two semantics have different observable type-error behavior. We need to make sure that their "agreement up to abstracted observation" is properly maintained also at higher types, i.e., for semantic values representing functions.
(3) The two semantics are organized differently (direct versus continuation-passing). Although this looks like a cross-cutting difference, because it permeates all the semantic equations, a suitable schematization of the two semantics will also let us isolate this aspect from the others.

In short, the problem boils down to formulating a suitable induction hypothesis about the meanings of general terms in the two semantics, from which Theorem 1 can ultimately be obtained. To capture the relationship between direct and continuation meanings of terms and values, Reynolds introduced two relations, $(\sim) \subseteq D^{\mathrm{d}} \times D^{\mathrm{c}}$ and $(\approx) \subseteq E^{\mathrm{d}} \times E^{\mathrm{c}}$, such that

$$\begin{aligned} d \sim d' \iff\ & (d = \perp \wedge (d' = \lambda k.\perp \vee d' = \lambda k.\lfloor in_2(*)\rfloor)) \vee \\ & (\exists e \approx e'.\, d = \lfloor e\rfloor \wedge d' = \lambda k.k\, e') \\ e \approx e' \iff\ & (\exists n.\, e = \iota^{\mathrm{d}}_{\mathrm{n}}(n) \wedge e' = \iota^{\mathrm{c}}_{\mathrm{n}}(n)) \vee \\ & (\exists f, f'.\, e = \iota^{\mathrm{d}}_{\mathrm{f}}(f) \wedge e' = \iota^{\mathrm{c}}_{\mathrm{f}}(f') \wedge \forall d \sim d'.\, f\, d \sim f'\, d'). \end{aligned}$$

The characterization of $\sim$ says that a direct and a continuation term-meaning are related if they either both represent failure to compute a value, or both represent successful computations of related values. On the other hand, two value-meanings are related by $\approx$ if either both are the same number, or both are functions mapping related term-meaning arguments to related results. Given these equivalences, it is then straightforward to show, by structural induction on t, that

$$(\forall x \in \mathbf{V}.\, \rho\, x \sim \rho'\, x) \Rightarrow [\![t]\!]^{\mathrm{d}}\rho \sim [\![t]\!]^{\mathrm{c}}\rho'\,.$$

And from this implication, the desired Theorem 1 follows easily. But does $\sim$ exist at all? Unlike a logical relation, $\approx$ is not defined by induction on the type structure. And since $\sim$ (through $\approx$) occurs negatively in its own specification property, it might be that the specification is actually unsatisfiable, i.e., amounts to requiring that $d \sim d'$ precisely when $d \not\sim d'$. Needless to say, an equivalence proof building implicitly on the existence of such a relation is unhelpful.

It turns out that existence of $\sim$ relies on more than E^{d} and E^{c} being arbitrary solutions to the underlying domain equations: we also exploit that they are *least* solutions. Reynolds's paper goes back to the inverse-limit constructions of E^{d} and E^{c}, and reinforces them to construct the relation $\approx$ at the same time. In this paper we will use a more abstract method, due to Pitts, which establishes the existence (and uniqueness) of $\approx$ based only on a simple, equational characterization of minimality.

1.5. Outline of the paper

In addition to showing the motivating Theorem 1 itself, we also want to develop a general methodology for establishing that two monadic semantics of a language agree. Accordingly, we use the following strategy:

(1) Introduce a formal metalanguage with recursive types and an abstract notion of computational effects. This language will have a straightforward domain-theoretic semantics, parameterized by the interpretations of effects as monads.
(2) Define a single, compositional translation of the sample language (henceforth called the *object* language) to the metalanguage, in such a way that we can recover both the direct and the continuation semantics as particular domain-theoretic interpretations of effects in the metalanguage.
(3) Develop general principles for constructing type-indexed relations between interpretations of the metalanguage, parameterized by relational actions of the effects; and show a general logical-relations lemma.
(4) Instantiate the logical-relations construction with specific relational interpretations of the effects, to obtain the desired equivalence theorem for the object language.

The first two points are covered in Section 2, and the last two in Section 3.

Additionally, the development is structured in such a way that it restricts easily to a setting without recursive types. That is, the parts of Sections 2 and 3 dealing with recursive types and relations between them can be skipped – especially on a first reading – without significant loss of continuity. In particular, if we want to show a property like Theorem 1 in the context of a simply typed object language, we only have to deal with problems (2) and (3) from Section 1.4, and the arguments are somewhat simpler. However, the basic message of the paper is that recursive types are in fact quite benign, and that the effect-specific reasoning from the typed case scales "for free" to an untyped object language.

2. A multi-monadic metalanguage

Even for modeling languages with a single notion of computational effects (i.e., *effect-untyped* ones), it is often useful to structure the semantics and proofs in terms of multiple effects, much like a semantics of an untyped language nevertheless typically makes use of several semantic domains (such as the D and E in our sample semantics). Accordingly, we start our development by introducing a *Multi-Monadic MetaLanguage*, $\mathrm{M}^3\mathrm{L}$.

This language is essentially that of Moggi [18,19], straightforwardly extended with constructs for product, sum, and (nested) recursive types, in the manner of FPC [10,9]. The main novelty is that we generalize the single computation-type constructor T to a whole family of such constructors, indexed by effects. We also relax the conditions on well-formed function spaces: in addition to T-exponentials $\tau \to T\tau'$, we allow the codomain type to be a product or further function space of computation types. The purpose is to syntactically allow total function spaces in places where they do not cause trouble semantically. Thus, e.g., $\mathit{int} \to \mathit{bool} \to T\,\mathit{int}$ would be a well-formed type (with an interpretation isomorphic to that of its uncurried variant, $\mathit{int} \times \mathit{bool} \to T\,\mathit{int}$), but $\mathit{int} \to \mathit{int}$ would not.

On the semantic side, to keep the constructions concrete, we will only consider one specific, domain-theoretic interpretation of the metalanguage (though parameterized by the meanings of base effects, types, and constants), instead of considering interpretations in arbitrary categories with suitable additional structure. This choice in particular ensures that all monads are automatically strong, simplifying many definitions and arguments. Also, to make the characterizations of both recursive types and terms more concise, we assume that all relevant computational effects include divergence as a possible behavior. These specializations are not expected to be essential, however: it should ultimately be possible to reformulate the following development in more abstract terms.

2.1. Syntax

We present the syntax of $\mathrm{M}^3\mathrm{L}$ in two parts: first the types and effects, and then terms. As is conventional, the syntax is parameterized by a *signature* Σ of elementary constructs included in the metalanguage.

Raw syntax [up to renaming of bound type variables (underlined)]

$$\Delta ::= \alpha_1, \ldots, \alpha_n \quad [\text{all } \alpha_i \text{ pairwise distinct}]$$
$$\tau ::= \alpha \mid b \mid 1 \mid \tau_1 \times \tau_2 \mid 0 \mid \tau_1 + \tau_2 \mid \mu\underline{\alpha}.\tau \mid \sigma$$
$$\sigma ::= 1 \mid \sigma_1 \times \sigma_2 \mid \tau \to \sigma \mid \langle e \rangle \tau$$

Kinding judgment, $\boxed{\vdash_\Delta \tau \text{ type}}$:

$$\frac{}{\vdash_\Delta \alpha \text{ type}}(\alpha \in \Delta) \qquad \frac{}{\vdash_\Delta b \text{ type}}((b \text{ type}) \in \Sigma) \qquad \frac{}{\vdash_\Delta 1 \text{ type}}$$

$$\frac{\vdash_\Delta \tau_1 \text{ type} \quad \vdash_\Delta \tau_2 \text{ type}}{\vdash_\Delta \tau_1 \times \tau_2 \text{ type}} \qquad \frac{}{\vdash_\Delta 0 \text{ type}} \qquad \frac{\vdash_\Delta \tau_1 \text{ type} \quad \vdash_\Delta \tau_2 \text{ type}}{\vdash_\Delta \tau_1 + \tau_2 \text{ type}}$$

$$\frac{\vdash_{\Delta,\alpha} \tau \text{ type}}{\vdash_\Delta \mu\alpha.\tau \text{ type}} \qquad \frac{\vdash_\Delta \sigma \; e\text{-type}}{\vdash_\Delta \sigma \text{ type}}$$

Effect-kinding judgment, $\boxed{\vdash_\Delta \sigma \; e\text{-type}}$:

$$\frac{}{\vdash_\Delta 1 \; e\text{-type}} \qquad \frac{\vdash_\Delta \sigma_1 \; e\text{-type} \quad \vdash_\Delta \sigma_2 \; e\text{-type}}{\vdash_\Delta \sigma_1 \times \sigma_2 \; e\text{-type}}$$

$$\frac{\vdash_\Delta \tau \text{ type} \quad \vdash_\Delta \sigma \; e\text{-type}}{\vdash_\Delta \tau \to \sigma \; e\text{-type}} \qquad \frac{\vdash_\Delta \tau \text{ type}}{\vdash_\Delta \langle e' \rangle \tau \; e\text{-type}}((e \preceq e') \in \Sigma)$$

Fig. 3. Well-kinded M^{3}L types.

2.1.1. Types and effects

Definition 2. The types-and-effects part of an M^{3}L signature Σ consists of the following:

- A countable set of *base types* b.
- A countable set of *effects* e, partially ordered by a *subeffect* relation $\preceq$.

We also require that the collection of effects has a *least element*, $\bot$.

Much like a type can be understood as denoting a collection of *values*, an effect denotes a collection of *computation behaviors*. These always include the *pure* behavior, i.e., merely returning a value; and there is a notion of *sequencing* the behaviors of two subcomputations. Effect names need not be atomic: for example e could itself be a set of exceptions that can be raised, or of storage cells that the computation can access.

Subeffecting is conceptually similar to subtyping: it may represent *inclusion* of behaviors (like variant-subtyping, where the supertype contains additional values), but also *identification* of behaviors (like record-subtyping, where the supertype ignores some distinctions between values). We will work with an explicit-coercion semantics of subeffecting in the metalanguage, but this semantics may be used to interpret implicit subeffecting (effect-subsumption) in an object language. The effect $\bot$ contains divergence as the only non-pure behavior. As noted above, we will assume that meanings of all other effects include this behavior as well.

Given a signature, we define well-kinded types in Fig. 3. We distinguish syntactically between *general* (or *value*) types τ and *computation* types σ, the latter being properly included in the former. To conveniently model (nested) type recursion, types may in general contain free type variables from a kinding context Δ; such variables are always considered to stand for value-types.

Terms of e-type σ represent *generalized computations* with e-behaviors. Such a generalized computation may need to be instantiated with further data, before it can proceed. In the simplest case, $\sigma = \langle e \rangle \tau$, no additional data is needed. For $\sigma = \sigma_1 \times \sigma_2$, the data determines whether to compute the first or the second component of the pair; for $\sigma = \tau \to \sigma'$, it is simply the τ-typed argument value. For $\sigma = \langle e' \rangle \tau$ with $e \preceq e'$, it represents whatever information is needed to turn an e'-computation into an e-computation; for example, if e is partiality and e' is partiality and state access, then an e'-computation is still a potentially diverging computation, but needing the initial state as the datum. $\bot$-types correspond

precisely to *pointed* types [17, 2.6.4]. (In the semantics, when $\perp$ is interpreted as the domain-theoretic lifting monad, they will denote pointed cpos.)

The syntax embodies a slight notational ambiguity, in that we are using the same symbol $\times$ for both value- and computation-product types (and similarly for the nullary variant, 1). However, we will ensure that this overloading is always coherent; that is, if we explicitly write $\overline{\sigma}$ for the inclusion of computation-types among general ones, it will always be the case that $[\![\overline{\sigma_1 \times \sigma_2}]\!] = [\![\overline{\sigma_1} \times \overline{\sigma_2}]\!]$ and $[\![\overline{1}]\!] = [\![1]\!]$.

Remark 3. We note that the following rule is evidently admissible:

$$\frac{\vdash_\Delta \sigma\ e'\text{-type}}{\vdash_\Delta \sigma\ e\text{-type}}\ ((e \preceq e') \in \Sigma).$$

If, instead, we took it as a proper rule, we could simply require $e = e'$ in the kinding rule for $\langle e' \rangle \tau$. We prefer the current formulation, however, since it makes the semantics of e-types definable by induction on syntactic phrases, rather than on kinding derivations. □

2.1.2. Terms

Definition 4. A (full) M^{3}L *signature* Σ consists of a type-and-effect part as in Definition 2, together with a countable set of typed constants $c\colon \tau$, where $\vdash \tau$ type according to the signature.

The main purpose of the constants is to allow construction of terms denoting non-trivial effectful computations. For example, a signature containing an exception effect e and a base type *exn* of exception names would typically also contain a constant $raise : exn \to \langle \mathsf{e} \rangle 0$ (where *raise* x raises exception x without returning any result), and a type-indexed family of constants $handle_\alpha : \langle \mathsf{e} \rangle \alpha \times (exn \to \langle \mathsf{e} \rangle \alpha) \to \langle \mathsf{e} \rangle \alpha$ (where $handle_\alpha\ (t, h)$ passes any exception raised by t to the handler h, which may either properly handle or re-raise it).

Given such a signature, the well-typed terms over Σ are shown in Fig. 4. The intended semantics of the various constructs should be mostly self-explanatory, except possibly for the last three forms. $\mathbf{val}^e\ M$ is the pure e-computation that simply returns the value of M. $\mathbf{glet}^e\ x \Leftarrow M_1.\ M_2$ is the generalized e-computation that, once it obtains its required instantiation data, performs the (ordinary) e-computation M_1, binds x to the result, and then continues with the now-instantiated computation M_2. Finally, $\mu x^\sigma.M$ allows recursive definitions of terms of any e-type σ. In particular, we can use it directly for defining function tuples by mutual recursion. (Given recursion on types, term recursion is actually definable by means of the Y-combinator, but we include it as a separate construct, so that the fragment of M^{3}L without μ-types is also computationally interesting.)

Note that, although $\mu\alpha.\cdots$ is the only binding construct for type variables, we do allow terms over a non-empty Δ, e.g., $\vdash_\alpha \lambda x^\alpha.\mathbf{val}^\perp\ x : \alpha \to \langle \perp \rangle \alpha$; such top-level polymorphic terms will be useful for expressing monad definitions inside the metalanguage, as detailed in Section 2.6.

We will frequently omit type tags where they are clear from the context. As syntactic sugar, we will also allow all binding occurrences of variables to be written as product-patterns, e.g., $\lambda(x^{\tau_1}, y^{\tau_2}).M$, with the obvious expansion in terms of projections from a single variable.

Finally, we can define two simpler term constructors in terms of the generalized let:

$$\mathbf{let}^e_\tau\ x \Leftarrow M_1.\ M_2 \equiv \mathbf{glet}^e_{\langle e \rangle \tau}\ x \Leftarrow M_1.\ M_2$$
$$\mathbf{inc}^{e,e'}_\tau\ M \equiv \mathbf{glet}^e_{\langle e' \rangle \tau}\ x \Leftarrow M.\ \mathbf{val}^{e'}\ x$$

with derived typing rules:

$$\frac{\Gamma \vdash_\Delta M_1 : \langle e \rangle \tau \quad \Gamma, x\colon \tau \vdash_\Delta M_2 : \langle e \rangle \tau'}{\Gamma \vdash_\Delta \mathbf{let}^e_\tau\ x \Leftarrow M_1.\ M_2 : \langle e \rangle \tau'} \qquad \frac{\Gamma \vdash_\Delta M : \langle e \rangle \tau}{\Gamma \vdash_\Delta \mathbf{inc}^{e,e'}_\tau\ M : \langle e' \rangle \tau}\ (e \preceq e').$$

Remark 5. M^{3}L is syntactically very similar to the Dependency Core Calculus (DCC) [1], where our poset of effects is specialized to a lattice of *protection levels*. The rules for well-kinded e-types mirror those for the *protected type* relation in DCC, and the rules for **val** and **glet** are like for η and `bind`. The DCC semantics is somewhat different, though, because the levels are not meant to model computational effects, but only restrictions on dependence; thus, they only play a non-trivial role in the relational interpretation. (In Tse and Zdancewic's formulation [34], however,

Raw syntax [up to renaming of bound variables (underlined)]

$$\begin{aligned}
\Gamma &::= x_1{:}\tau_1, \ldots, x_n{:}\tau_n \quad \text{[all } x_i \text{ pairwise distinct]}\\
M &::= x \mid c \mid () \mid (M_1, M_2) \mid \mathbf{fst}(M) \mid \mathbf{snd}(M) \mid \mathbf{inl}_{\tau_2}(M) \mid \mathbf{inr}_{\tau_1}(M)\\
&\quad \mid \mathbf{void}_\tau(M) \mid \mathbf{case}\ M\ \mathbf{of\ inl}\ (\underline{x_1}).\ M_1\ []\ \mathbf{inr}\ (\underline{x_2}).\ M_2 \mid \mathbf{in}_{\mu\alpha.\tau}(M)\\
&\quad \mid \mathbf{out}_{\mu\alpha.\tau}(M) \mid \lambda\underline{x}^\tau.M \mid M_1\,M_2 \mid \mathbf{val}^e\ M \mid \mathbf{glet}^e_\sigma\ \underline{x} \Leftarrow M_1.\ M_2 \mid \mu\underline{x}^\sigma.M
\end{aligned}$$

Well-kinded **type environments** $\boxed{\vdash_\Delta \Gamma\ \text{env}} \Leftrightarrow \forall(x_i{:}\tau_i) \in \Gamma.\ \vdash_\Delta \tau_i$ type.
Assuming $\vdash_\Delta \Gamma$ env and $\vdash_\Delta \tau$ type, a **typing** judgment $\boxed{\Gamma \vdash_\Delta M : \tau}$:

$$\frac{}{\Gamma \vdash_\Delta c : \tau}((c{:}\tau) \in \Sigma) \qquad \frac{}{\Gamma \vdash_\Delta x : \tau}((x{:}\tau) \in \Gamma) \qquad \frac{}{\Gamma \vdash_\Delta () : 1}$$

$$\frac{\Gamma \vdash_\Delta M_1 : \tau_1 \qquad \Gamma \vdash_\Delta M_2 : \tau_2}{\Gamma \vdash_\Delta (M_1, M_2) : \tau_1 \times \tau_2} \qquad \frac{\Gamma \vdash_\Delta M : \tau_1 \times \tau_2}{\Gamma \vdash_\Delta \mathbf{fst}(M) : \tau_1} \qquad \frac{\Gamma \vdash_\Delta M : \tau_1 \times \tau_2}{\Gamma \vdash_\Delta \mathbf{snd}(M) : \tau_2}$$

$$\frac{\Gamma \vdash_\Delta M : 0}{\Gamma \vdash_\Delta \mathbf{void}_\tau(M) : \tau} \qquad \frac{\Gamma \vdash_\Delta M : \tau_1}{\Gamma \vdash_\Delta \mathbf{inl}_{\tau_2}(M) : \tau_1 + \tau_2} \qquad \frac{\Gamma \vdash_\Delta M : \tau_2}{\Gamma \vdash_\Delta \mathbf{inr}_{\tau_1}(M) : \tau_1 + \tau_2}$$

$$\frac{\Gamma \vdash_\Delta M : \tau_1 + \tau_2 \qquad \Gamma, x_1{:}\tau_1 \vdash_\Delta M_1 : \tau \qquad \Gamma, x_2{:}\tau_2 \vdash_\Delta M_2 : \tau}{\Gamma \vdash_\Delta \mathbf{case}\ M\ \mathbf{of\ inl}\ (x_1).\ M_1\ []\ \mathbf{inr}\ (x_2).\ M_2 : \tau}$$

$$\frac{\Gamma \vdash_\Delta M : \tau[(\mu\alpha.\tau)/\alpha]}{\Gamma \vdash_\Delta \mathbf{in}_{\mu\alpha.\tau}(M) : \mu\alpha.\tau} \qquad \frac{\Gamma \vdash_\Delta M : \mu\alpha.\tau}{\Gamma \vdash_\Delta \mathbf{out}_{\mu\alpha.\tau}(M) : \tau[(\mu\alpha.\tau)/\alpha]}$$

$$\frac{\Gamma, x{:}\tau \vdash_\Delta M : \sigma}{\Gamma \vdash_\Delta \lambda x^\tau.M : \tau \to \sigma} \qquad \frac{\Gamma \vdash_\Delta M_1 : \tau \to \sigma \qquad \Gamma \vdash_\Delta M_2 : \tau}{\Gamma \vdash_\Delta M_1\,M_2 : \sigma}$$

$$\frac{\Gamma \vdash_\Delta M : \tau}{\Gamma \vdash_\Delta \mathbf{val}^e\ M : \langle e\rangle\tau} \qquad \frac{\Gamma \vdash_\Delta M_1 : \langle e\rangle\tau \quad \Gamma, x{:}\tau \vdash_\Delta M_2 : \sigma \quad \vdash_\Delta \sigma\ e\text{-type}}{\Gamma \vdash_\Delta \mathbf{glet}^e_\sigma\ x \Leftarrow M_1.\ M_2 : \sigma}$$

$$\frac{\Gamma, x{:}\sigma \vdash_\Delta M : \sigma}{\Gamma \vdash_\Delta \mu x^\sigma.M : \sigma}$$

Fig. 4. Well-typed M^{3}L terms.

levels are modeled in terms of environment monads giving selective access to *keys* that allow otherwise parametric computations to non-trivially depend on data from lower protection levels.)

DCC actually models not only security considerations, but also more general questions of what computations may depend on what data, such as binding-time analysis. A closer investigation is beyond the scope of this paper, but it seems very likely that the commonalities are more than superficial. In particular, we expect that the constructions presented here could be easily instantiated or adapted to give a coherent account of an extension of DCC with recursive types, and possibly also with computational effects beyond partiality.

If we wanted to give a formal operational interpretation of the metalanguage, it might be advantageous to distinguish syntactically between value terms (with τ-types) and computation terms (with σ-types), as is done in Levy's call-by-push-value syntax [12], or perhaps adopt the modal-logic-based syntax of Pfenning and Davies [23]. However, we choose not to pursue such directions here, both in order to remain as close as possible to Moggi's original monadic metalanguage, and because the idea of an operational semantics parameterized by an arbitrary notion of computation conceptually requires parameterizing over the *format* of the evaluation rules, which seems hard to do uniformly in a sufficiently general way. □

2.2. Semantical preliminaries

As already mentioned, we are working in the concrete setting of cpos (more precisely, ω-cpos, i.e., sets equipped with partial orders, closed under limits of ω-chains, but not necessarily containing least elements). A "function" will always mean a total, continuous function (i.e., preserving limits of ω-chains), unless explicitly qualified otherwise. It is expected, though, that the constructions should be fairly robust with respect to the precise domain-theoretic framework used.

Cpos and functions form a category **Cpo**; though we will occasionally mention categorical shorthand for some of the definitions and results, the development is meant to be accessible even to readers without any formal background in category theory.

Definition 6 (*Monad*). A *monad* is a triple, $\mathcal{T} = (T, \eta, \star)$, where for every cpo A, TA is a cpo of *computations* of A-elements; for every A, $\eta_A : A \to TA$ is the *unit* function, and for every A and B, $\star_{A,B} : TA \times (A \to TB) \to TB$ is the *bind* function. These functions must satisfy the following equalities:

$$\eta_A\, a \star_{A,B} f = f\, a \quad [a \in A, f \in A \to TB] \tag{1}$$

$$t \star_{A,A} \eta_A = t \quad [t \in TA] \tag{2}$$

$$(t \star_{A,B} f) \star_{B,C} g = t \star_{A,C} (\lambda a^A.\, f\, a \star_{B,C} g) \quad [t \in TA, f \in A \to TB, g \in B \to TC]. \tag{3}$$

We will often omit the indices on these and other cpo-indexed function families when they are clear from the context.

Remark 7. Technically, the above definition is more accurately called a *Kleisli triple*. However, this formulation is equivalent to the more standard category-theoretic notion of monads, as consisting of an *endofunctor* T on **Cpo** and *natural transformations* $\eta_A : A \to TA$ and $\mu_A : TTA \to TA$ satisfying a couple of additional equations [14, VI], because one can take $T(f) = \lambda t^{TA}.\, t \star_{A,B} (\eta_B \circ f)$ and $\mu_A = \lambda t^{TTA}.\, t \star_{TA,A} id_{TA}$; and, conversely, $t \star_{A,B} f = \mu_B(T(f)\, t)$.

More subtly, we are actually working with *strong* monads [18], in that the $\star$-operation acts on elements of the function space, $f \in A \to TB$, rather than on morphisms $f : A \to TB$. Notationally, this strengthening means that in $t \star \lambda a. \cdots$, we allow the abstraction body to contain variables other than a. In the setting of cpos (where the morphisms and function-space elements are in 1–1 correspondence), the shortcut makes no material difference, but simplifies the notation considerably. However, it should also be possible to rework the following constructions in terms of a *tensorial strength* $\tau_{A,B} : A \times TB \to T(A \times B)$, used to explicitly "bring in" variables under the scope of a computation. (In our formulation, it is trivially definable as $\tau_{A,B} = \lambda(a, t).\, t \star \lambda b.\, \eta\,(a, b)$.) This refinement might prove necessary if we want to extend the results to a more complicated setting, such as a Kripke semantics. Alternatively, one might build uses of τ directly into the $\star$-operation as an *indexed Kleisli triple*, in the manner of Pitts [24]. □

Example 8. The *lifting monad* $\mathcal{T}^\perp = (T^\perp, \eta^\perp, \star^\perp)$ is given by

$$T^\perp A = A_\perp \qquad \eta^\perp a = \lfloor a \rfloor \qquad t \star^\perp f = f^* t.$$

It is immediately checked that $\mathcal{T}^\perp$ is indeed a monad. □

The intuition is that elements of TA represent potentially effectful computations of A-elements. ηa represents the pure computation returning a, while $t \star f$ represents the computation evaluating t, followed by applying f to the result (if any). Elements of TA other than ηa represent computations with various impure behaviors. While η is usually injective, there is no requirement of this; in particular, $TA = \mathbf{1}$ can be uniquely extended to a monad.

Definition 9 (*Monad Morphism*). A *monad morphism* from $(T, \eta, \star)$ to $(T', \eta', \star')$ is a family of functions $i_A : TA \to T'A$, such that

$$i_A\,(\eta_A\, a) = \eta'_A\, a \quad [a \in A] \tag{4}$$

$$i_B\,(t \star_{A,B} f) = i_A\, t \star'_{A,B} (\lambda a.\, i_B\,(f\, a)) \quad [t \in TA, f \in A \to TB]. \tag{5}$$

It is easy to check that the identity and composition of monad morphisms are themselves monad morphisms; indeed, monads and morphisms between them form a category.

A monad morphism can be thought of as transforming between different representations of behaviors; as such, it must preserve the pure behavior and behavior sequencing. Monad morphisms will be used to model subeffecting in the metalanguage.

Definition 10 (*Monad Algebra*). An *algebra* for a monad $(T, \eta, \star)$ is a pair (D, γ), where D is the *carrier* cpo, and $\gamma : TD \to D$ is the *structure map*, such that

$$\gamma\,(\eta_D\, d) = d \quad [d \in D] \tag{6}$$

$$\gamma\,(t \star_{A,D} f) = \gamma\,(t \star_{A,D} \lambda a.\, \eta_D\,(\gamma\,(f\, a))) \quad [t \in TA,\ f \in A \to TD]. \tag{7}$$

Consider an algebra $(D, \gamma : D_\perp \to D)$ for the lifting monad. D must have a least element, namely $\perp_D = \gamma \perp$ (because, by monotonicity of γ and (6), for any $d \in D$, $\perp_D = \gamma \perp \sqsubseteq \gamma \lfloor d \rfloor = \gamma\,(\eta^\perp d) = d$). Conversely, any pointed cpo can be (uniquely) organized as a $\mathcal{T}^\perp$-algebra by taking $\gamma \perp = \perp_D$ and $\gamma \lfloor d \rfloor = d$. Eq. (6) is then immediate, and (7) is clearly satisfied both for $t = \perp$ and $t = \lfloor a \rfloor$.

Algebras will be used for modeling generalized computations in the metalanguage.

Definition 11 (*Algebra Morphism*). An *algebra morphism* or *rigid function* between $\mathcal{T}$-algebras (D, γ) and (D', γ') is a function $h : D \to D'$, such that

$$h\,(\gamma\, t) = \gamma'\,(t \star_{D,D'} \lambda d.\, \eta_{D'}\,(h\, d)) \quad [t \in TD]. \tag{8}$$

In particular, a function between pointed cpos D and D' is a $\mathcal{T}^\perp$-algebra morphism precisely when it is strict, i.e., maps $\perp_D$ to $\perp_{D'}$. More generally, when elements of D and D' represent generalized computations, a rigid function can be thought of as mapping every d into a d' that uses (and performs any behaviors of) d "first".

Monad morphisms naturally relate independently given monads. There is also a different notion of relationship, particularly convenient for constructing more complex monads incrementally, by building on top of simpler ones [8]; we will return to this construction in Section 2.6:

Definition 12 (*Monad Layering*). A *layering* of monad $\mathcal{T}'$ over $\mathcal{T}$ is a family of $\mathcal{T}$-algebras $(T'A, \gamma_A : TT'A \to T'A)$, with respect to which $\lambda t'.\, t' \star'_{A,B} f \in T'A \to T'B$ is rigid for every $f \in A \to T'B$, i.e., such that

$$\gamma_A\, t \star'_{A,B} f = \gamma_B\,(t \star_{T'A,T'B} \lambda t'.\, \eta_{T'B}\,(t' \star'_{A,B} f)) \qquad [t \in TT'A]. \tag{9}$$

It is easy to see that the two notions are in fact equivalent:

Lemma 13. *There is a one-to-one correspondence between monad layerings γ of $\mathcal{T}'$ over $\mathcal{T}$, and monad morphisms i from $\mathcal{T}$ to $\mathcal{T}'$, given by*

$$i_A\, t = \gamma_A\,(t \star_{A,T'A} \lambda a.\, \eta_{T'A}\,(\eta'_A\, a)) \quad [t \in TA] \tag{10}$$

$$\gamma_A\, t = i_{T'A}\, t \star'_{T'A,A} \lambda t'.\, t' \quad [t \in TT'A]. \tag{11}$$

Proof. Assuming that each of $\mathcal{T}$ and $\mathcal{T}'$ satisfies Eqs. (1)–(3), it is straightforward to check that (4) and (5) follow from (6), (7), (9) and (10); and conversely, that (6), (7) and (9) follow from (4), (5) and (11). Moreover, i expanded by (10) and (11) is equal to i by (4) and (5); and conversely, γ expanded by (10) and (11) is equal to γ by (6), (7) and (9). □

2.3. Semantics of types

Let us now return to giving a semantics of M^{3}L. We start with interpreting the type structure:

Definition 14. A *type-and-effect interpretation* of an M^{3}L signature Σ consists of the following assignments:

- To every base type $b \in \Sigma$, a cpo $\mathcal{B}^b$.
- To every effect $e \in \Sigma$, a monad $\mathcal{T}^e = (T^e, \eta^e, \star^e)$.
- To every subeffecting $(e \preceq e') \in \Sigma$, a monad morphism $i^{e,e'}$ from $\mathcal{T}^e$ to $\mathcal{T}^{e'}$, such that $i^{e,e}_A = id_{T^e A}$, and $i^{e,e''}_A = i^{e',e''}_A \circ i^{e,e'}_A$ when $e \preceq e' \preceq e''$.

The distinguished effect $\perp$ is always interpreted as the lifting monad $\mathcal{T}^\perp$.

For well-formed Δ, $\boxed{[\![\Delta]\!]} = \mathbf{Cpo}^{\Delta} = \{\theta \mid \forall \alpha \in \Delta.\theta(\alpha) \text{ cpo}\}$.
Let $\theta \in [\![\Delta]\!]$.

For $\vdash_\Delta \tau$ type, a cpo $\boxed{[\![\tau]\!]_\theta}$:

$$\begin{aligned}
[\![\alpha]\!]_\theta &= \theta(\alpha)\\
[\![b]\!]_\theta &= \mathcal{B}^b\\
[\![1]\!]_\theta &= \mathbf{1} = \{*\}\\
[\![\tau_1 \times \tau_2]\!]_\theta &= [\![\tau_1]\!]_\theta \times [\![\tau_2]\!]_\theta = \{(a_1, a_2) \mid a_1 \in [\![\tau_1]\!]_\theta, a_2 \in [\![\tau_2]\!]_\theta\}\\
[\![0]\!]_\theta &= \mathbf{0} = \emptyset\\
[\![\tau_1 + \tau_2]\!]_\theta &= [\![\tau_1]\!]_\theta + [\![\tau_2]\!]_\theta = \{in_1(a_1) \mid a_1 \in [\![\tau_1]\!]_\theta\} \cup \{in_2(a_2) \mid a_2 \in [\![\tau_2]\!]_\theta\}\\
[\![\mu\alpha.\tau]\!]_\theta &= \mu[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta = \{\phi_{[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta}\, a \mid a \in [\![\tau]\!]_{\theta[\alpha \mapsto [\![\mu\alpha.\tau]\!]_\theta]}\} \quad \text{[see Section 2.4]}\\
[\![\sigma]\!]_\theta &= [\![\sigma]\!]^{\mathrm{c}}_\theta
\end{aligned}$$

For $\vdash_\Delta \sigma$ e-type, a cpo $\boxed{[\![\sigma]\!]^{\mathrm{c}}_\theta}$ and a function $\boxed{[\![\sigma]\!]^{se}_\theta : T^e[\![\sigma]\!]^{\mathrm{c}}_\theta \to [\![\sigma]\!]^{\mathrm{c}}_\theta}$:

$$\begin{aligned}
[\![1]\!]^{\mathrm{c}}_\theta &= \mathbf{1}\\
[\![1]\!]^{se}_\theta\,(t) &= *\\
[\![\sigma_1 \times \sigma_2]\!]^{\mathrm{c}}_\theta &= [\![\sigma_1]\!]^{\mathrm{c}}_\theta \times [\![\sigma_2]\!]^{\mathrm{c}}_\theta\\
[\![\sigma_1 \times \sigma_2]\!]^{se}_\theta\,(t) &= ([\![\sigma_1]\!]^{se}_\theta\,(t \star^e \lambda(d_1, d_2).\eta^e\, d_1), [\![\sigma_2]\!]^{se}_\theta\,(t \star^e \lambda(d_1, d_2).\eta^e\, d_2))\\
[\![\tau \to \sigma]\!]^{\mathrm{c}}_\theta &= [\![\tau]\!]_\theta \to [\![\sigma]\!]^{\mathrm{c}}_\theta\\
[\![\tau \to \sigma]\!]^{se}_\theta\,(t) &= \lambda a.[\![\sigma]\!]^{se}_\theta\,(t \star^e \lambda f.\eta^e\,(f\, a))\\
[\![\langle e'\rangle\tau]\!]^{\mathrm{c}}_\theta &= T^{e'}[\![\tau]\!]_\theta\\
[\![\langle e'\rangle\tau]\!]^{se}_\theta\,(t) &= i^{e,e'}\, t \star^{e'} \lambda t'.t'
\end{aligned}$$

Fig. 5. Semantics of M^{3}L types.

(The last two bullets could be summarized as saying that the interpretation of effects must be a *functor* from the skeletal category of effect names and subeffecting to the category of monads and monad morphisms.)

Given such an interpretation, and an assignment of cpos to all type variables in Δ, we define meanings of all types and e-types over Δ in Fig. 5. (We will elaborate on the meaning of μ-types in Section 2.4.)

We immediately see, since the inclusion from computation to value types merely forgets the structure-map part of the interpretation, that $[\![\overline{\sigma_1 \times \sigma_2}]\!]_\theta = [\![\sigma_1 \times \sigma_2]\!]^{\mathrm{c}}_\theta = [\![\sigma_1]\!]^{\mathrm{c}}_\theta \times [\![\sigma_2]\!]^{\mathrm{c}}_\theta = [\![\overline{\sigma_1}]\!]_\theta \times [\![\overline{\sigma_2}]\!]_\theta = [\![\overline{\sigma_1} \times \overline{\sigma_2}]\!]_\theta$.

Lemma 15. *For $\vdash_\Delta \sigma$ e-type, $([\![\sigma]\!]^{\mathrm{c}}_\theta, [\![\sigma]\!]^{se}_\theta)$ is a $\mathcal{T}^e$-algebra. In particular, since any well-formed σ is a $\perp$-type, the cpo $[\![\sigma]\!]^{\mathrm{c}}_\theta$ is always pointed.*

Proof. Straightforward induction on the structure of σ. □

As noted in Remark 3, any e'-type is also an e-type when $e \preceq e'$. The corresponding algebras are related as follows:

Lemma 16. *If $\vdash_\Delta \sigma$ e'-type and $e \preceq e'$ then $[\![\sigma]\!]^{se}_\theta = [\![\sigma]\!]^{se'}_\theta \circ i^{e,e'}$.*

Proof. Straightforward induction on the structure of σ. □

Remark 17. One might consider extending the metalanguage with a notion of computational base types, to be interpreted by full algebras rather than only cpos, and/or with type variables ranging over computation types, potentially allowing direct recursive definitions of the latter. It is unclear, however, whether any useful object languages would actually require such facilities in the metalanguage; so far, this does not seem to be the case. □

2.4. Recursive types

We still need to give a proper account of recursive types, which may themselves be type-parameterized, i.e., in $\mu\alpha.\tau$, τ may contain type variables other than α, possibly bound by outer μs. As a further challenge, we want the constructions to eventually scale directly to reasoning about programs with (possibly nested) recursive types, in the same style as we would use for reasoning about simply typed ones.

This section can be skipped by readers primarily interested in the semantics of effects, rather than of type recursion. The main result can be summarized as verifying that the second, circular characterization of $[\![\mu\alpha.\tau]\!]_\theta$ in Fig. 5 in fact amounts to a proper definition.

2.4.1. Solving recursive cpo equations

The solutions of recursive domain equations are perhaps more commonly formulated in the category of pointed cpos and strict continuous functions. Since our metalanguage takes plain cpos as the primitive concept, however, it will be more convenient to formulate the results in terms of the equivalent category of cpos and (a representation of) partial continuous functions.

To keep the development compact, we introduce some auxiliary notation. We write $f : A \rightharpoonup B$ for $f : A \to B_\perp$. For any $f : A \to B$, we take $\overline{f} = \eta_B^\perp \circ f = \lambda a.\lfloor f\, a\rfloor : A \rightharpoonup B$ (in particular, $\overline{id}_A = \eta_A^\perp : A \rightharpoonup A$); and when also $g : B \rightharpoonup C$, we define $g \mathbin{\bar\circ} f = \lambda a.\, f\, a \star_{B,C}^\perp g = g^* \circ f : A \rightharpoonup C$. We recognize functions $f : A \rightharpoonup B$ as morphisms in the Kleisli category $\mathbf{Cpo}_{\mathcal{T}^\perp}$ of the lifting monad.

Further, for a pair of functions $f^+ : A \rightharpoonup B$ and $f^- : B \rightharpoonup A$, we write $f^\pm = \binom{f^+}{f^-} : A \rightleftharpoons B$, with $\overline{\boldsymbol{id}}_A = \binom{\overline{id}_A}{\overline{id}_A} : A \rightleftharpoons A$; and when also $g^\pm : B \rightleftharpoons C$, composition is given by $g^\pm \mathbin{\bar\bullet} f^\pm = \binom{g^+ \mathbin{\bar\circ} f^+}{f^- \mathbin{\bar\circ} g^-} : A \rightleftharpoons C$. This determines a category of function pairs, $\mathbf{Cpo}_{\mathcal{T}^\perp}^\pm$. Finally, we write $f^\mp = \binom{f^-}{f^+} : B \rightleftharpoons A$ for the function pair with the components swapped.

To clearly separate the endofunctors on **Cpo** arising from effect-interpreting monads (as outlined in Remark 7) from those used only to interpret recursive types, we introduce a compact name for the latter (the prefix "m" is meant to suggest "mixed" and/or "mu"):

Definition 18 (*m-Functor*). An *m-functor* F is a functor from $\mathbf{Cpo}_{\mathcal{T}^\perp}^\pm$ to $\mathbf{Cpo}_{\mathcal{T}^\perp}$, i.e. mapping every cpo A to a cpo FA; and every function pair $f^\pm : A \rightleftharpoons B$ to a function $F(f^\pm) : FA \rightharpoonup FB$, such that $F(\overline{\boldsymbol{id}}_A) = \overline{id}_{FA}$ and $F(g^\pm \mathbin{\bar\bullet} f^\pm) = F(g^\pm) \mathbin{\bar\circ} F(f^\pm)$.

We can now state a central result of domain theory:

Theorem 19 (*Existence of Minimal Invariants*). *Let F be a* locally continuous *m-functor, i.e., additionally satisfying that $F\binom{\bigsqcup_{i\in\omega} f_i^+}{\bigsqcup_{i\in\omega} f_i^-} = \bigsqcup_{i\in\omega} F\binom{f_i^+}{f_i^-}$. Then there exists a* minimal invariant *for F, i.e., a cpo μF and an isomorphism $\phi_F : F(\mu F) \to \mu F$, such that $\mathit{fix}(\lambda f^{\mu F \to (\mu F)_\perp}.\, \overline{\phi_F} \mathbin{\bar\circ} F\binom{f}{f} \mathbin{\bar\circ} \overline{\phi_F^{-1}}) = \overline{id}_{\mu F}$.*

Proof. It is fairly easy to verify that the standard inverse-limit construction actually constructs solutions that are minimal invariants. See, e.g., Pitts [26] for the formulation in terms of pointed cpos and strict functions; the adaptations to our equivalent category of partial functions are straightforward. □

One can easily show that minimal invariants are unique up to isomorphism. The minimality condition will prove instrumental not only for reasoning about the solution, but also for even constructing well-defined meanings of nested μ-types. (Working in the category $\mathbf{Cpo}_{\mathcal{T}^\perp}^\pm$ instead of $\mathbf{Cpo}_{\mathcal{T}^\perp}^{\mathrm{op}} \times \mathbf{Cpo}_{\mathcal{T}^\perp}$, i.e., where the components of a function pair need not go between the same two cpos, will allow us to use Theorem 19 as stated, without having to generalize it explicitly to *simultaneous* minimal invariants; cf. [26, Remark 7.1].)

Example 20. Consider the type $\mu\alpha.1 + \alpha$, which we expect to denote a cpo A with an isomorphism $\phi : \mathbf{1} + A \to A$. A possible choice is of course $A = \mathbf{N}$, with $\phi(in_1(*)) = 0$ and $\phi(in_2(n)) = n + 1$ for all $n \in \mathbf{N}$. However, we could also have taken, e.g., $A = \mathbf{N}' = \mathbf{N} \cup \{\infty\}$ (still ordered discretely); then, with the additional clause $\phi(in_2(\infty)) = \infty$, $\phi : \mathbf{1} + \mathbf{N}' \to \mathbf{N}'$ remains an isomorphism.

However, the minimal-invariant condition turns out to say that the "copy function", $f : A \to A_\perp$, recursively defined in the least-fixed-point sense by

$$f(a) = \text{case } \phi^{-1}(a) \text{ of } \left\{\begin{array}{l} in_1(*) \to \lfloor \phi(in_1(*)) \rfloor \\ in_2(a') \to (\lambda a''.\, \lfloor \phi(in_2(a'')) \rfloor)^*(f(a')) \end{array}\right\}$$

must satisfy $f(a) = \lfloor a \rfloor$ for all $a \in A$. This is in fact the case for all $a \in \mathbf{N}$, but we evidently have $f\infty = \bot$, so the alternative solution fails to be a minimal invariant.

Note, though, that $\mathbf{N}$ and $\mathbf{N}'$ are still isomorphic as sets, and hence also as discrete cpos. Thus, being a minimal invariant is inherently a property of the whole pair $(\mu F, \phi_F)$, not just of μF itself. (We will return this example when considering recursively defined relations.) □

2.4.2. M-functorial semantics of types

To use Theorem 19, we need to associate with each type constructor $\alpha.\tau$, an m-functorial action. And to define this action by induction on τ, which may itself contain μs, we need to generalize our notion of an m-functor a bit, making it take as arguments a whole sequence of objects or function pairs, indexed by a sequence of type variables Δ:

Let $\theta, \theta' \in [\![\Delta]\!]$. We write $\varphi : \theta \rightleftharpoons \theta'$ for $\forall \alpha \in \Delta.\, \varphi(\alpha) : \theta(\alpha) \rightleftharpoons \theta'(\alpha)$. In particular, $\overline{\boldsymbol{id}}_\theta : \theta \rightleftharpoons \theta$ is defined componentwise, and for $\varphi_1 : \theta \rightleftharpoons \theta'$ and $\varphi_2 : \theta' \rightleftharpoons \theta''$, so is $\varphi_2 \mathbin{\bar\bullet} \varphi_1 : \theta \rightleftharpoons \theta''$. This determines a category $(\mathbf{Cpo}^{\pm}_{\mathcal{T}^\perp})^\Delta$. Finally, we define $\varphi^\dagger \in \theta' \rightleftharpoons \theta$ by $\varphi^\dagger(\alpha) = f^\mp$, when $\varphi(\alpha) = f^\pm$, for each $\alpha \in \Delta$.

We give the m-functorial interpretations of types in Fig. 6.

Again, we first check that the meanings of product types are coherent:

$$\begin{aligned}
[\![\overline{\sigma_1 \times \sigma_2}]\!]^{\mathrm{f}}(\varphi) &= \lambda d.\, \eta^\perp([\![\sigma_1 \times \sigma_2]\!]^{\mathrm{cf}}(\varphi)\, d) \\
&= \lambda(d_1, d_2).\, \eta^\perp([\![\sigma_1]\!]^{\mathrm{cf}}(\varphi)\, d_1, [\![\sigma_2]\!]^{\mathrm{cf}}(\varphi)\, d_2) \\
&= \lambda(d_1, d_2).\, \eta^\perp([\![\sigma_1]\!]^{\mathrm{cf}}(\varphi)\, d_1) \star^\perp \lambda d_1'.\, \eta^\perp([\![\sigma_2]\!]^{\mathrm{cf}}(\varphi)\, d_2) \star^\perp \lambda d_2'.\, \eta^\perp(d_1', d_2') \\
&= \lambda(d_1, d_2).\, [\![\overline{\sigma_1}]\!]^{\mathrm{f}}(\varphi)\, d_1 \star^\perp \lambda d_1'.\, [\![\overline{\sigma_2}]\!]^{\mathrm{f}}(\varphi)\, d_2 \star^\perp \lambda d_2'.\, \eta^\perp(d_1', d_2') \\
&= [\![\overline{\sigma_1} \times \overline{\sigma_2}]\!]^{\mathrm{f}}(\varphi).
\end{aligned}$$

Lemma 21. *For $\vdash_\Delta \sigma$ e-type, and $\varphi : \theta \rightleftharpoons \theta'$, $[\![\sigma]\!]^{\mathrm{cf}}(\varphi)$ is strict (i.e., a $\mathcal{T}^\perp$-algebra morphism):*

$$[\![\sigma]\!]^{\mathrm{cf}}(\varphi)\,([\![\sigma]\!]^{\mathrm{s}\perp}_\theta\, t) = [\![\sigma]\!]^{\mathrm{s}\perp}_{\theta'}\,(t \star^\perp \lambda d.\, \eta^\perp([\![\sigma]\!]^{\mathrm{cf}}(\varphi)\, d)) \quad [t \in T^\perp([\![\sigma]\!]_\theta)]. \tag{12}$$

Proof. It is easy to check, by induction on σ, that $[\![\sigma]\!]^{\mathrm{cf}}(\varphi) \perp_{[\![\sigma]\!]_\theta} = \perp_{[\![\sigma]\!]_{\theta'}}$. In fact, with a little care, we can show that if $\vdash_\Delta \sigma$ e-type then $[\![\sigma]\!]^{\mathrm{cf}}(\varphi)$ is also a $\mathcal{T}^e$-algebra morphism, as long as $\mathcal{T}^e$ is a *commutative* monad (commutativity is needed in the function-type case). □

The functor $[\![\sigma]\!]^{\mathrm{cf}}(-)$ thus effectively goes to the Eilenberg–Moore category of the lifting monad, $\mathbf{Cpo}^{\mathcal{T}^\perp}$, which we can think of as the subcategory of **Cpo** with pointed cpos as objects and (total) strict functions as morphisms.

Formally, since the proper definition of $[\![\tau]\!]_\theta$ when $\tau = \mu\alpha.\tau_0$ relies on Theorem 19, the fact that $[\![\tau]\!]_\theta$ is well defined has to be shown by induction on τ together with the following result:

Lemma 22. *The semantics $[\![\tau]\!]^{\mathrm{f}}(\varphi)$ and $[\![\sigma]\!]^{\mathrm{cf}}(\varphi)$ are functorial in the following senses: let $\theta, \theta', \theta'' \in [\![\Delta]\!]$ and $\varphi_1 : \theta \rightleftharpoons \theta'$, $\varphi_2 : \theta' \rightleftharpoons \theta''$. Then*

(1) *If $\vdash_\Delta$ τ type then $([\![\tau]\!]_-, [\![\tau]\!]^{\mathrm{f}}(-))$ form a functor from $(\mathbf{Cpo}^{\pm}_{\mathcal{T}^\perp})^\Delta$ to $\mathbf{Cpo}_{\mathcal{T}^\perp}$, i.e., $[\![\tau]\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta) = \overline{id}_{[\![\tau]\!]_\theta}$ and $[\![\tau]\!]^{\mathrm{f}}(\varphi_2 \mathbin{\bar\bullet} \varphi_1) = [\![\tau]\!]^{\mathrm{f}}(\varphi_2) \mathbin{\bar\circ} [\![\tau]\!]^{\mathrm{f}}(\varphi_1)$.*

(2) *If $\vdash_\Delta$ σ e-type then $([\![\sigma]\!]^{\mathrm{c}}_-, [\![\sigma]\!]^{\mathrm{cf}}(-))$ form a functor from $(\mathbf{Cpo}^{\pm}_{\mathcal{T}^\perp})^\Delta$ to **Cpo**, i.e., $[\![\sigma]\!]^{\mathrm{cf}}(\overline{\boldsymbol{id}}_\theta) = id_{[\![\sigma]\!]^{\mathrm{c}}_\theta}$ and $[\![\sigma]\!]^{\mathrm{cf}}(\varphi_2 \mathbin{\bar\bullet} \varphi_1) = [\![\sigma]\!]^{\mathrm{cf}}(\varphi_2) \circ [\![\sigma]\!]^{\mathrm{cf}}(\varphi_1)$.*

In particular, for $\vdash_{\Delta,\alpha} \tau$ type, $[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta$ is indeed a locally continuous m-functor.

Proof. By mutual structural induction on τ and σ. Most cases for part (1) are completely straightforward, noting that the case for $\tau_1 \times \tau_2$, like Lemma 21, relies on $\mathcal{T}^\perp$ being commutative. The only non-obvious case is for $\tau = \mu\alpha.\tau_0$. For the first half (identity), let $F = [\![\alpha.\tau_0]\!]^{\mathrm{m}}_\theta$, take $\phi^\pm = \binom{\phi_F}{\phi_F^{-1}}$, and define the functionals,

$$H(h^\pm) = \begin{pmatrix} \overline{\phi^+} \mathbin{\bar\circ} [\![\tau_0]\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta[\alpha \mapsto h^\pm]) \mathbin{\bar\circ} \overline{\phi^-} \\ \overline{\phi^+} \mathbin{\bar\circ} [\![\tau_0]\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta[\alpha \mapsto h^\mp]) \mathbin{\bar\circ} \overline{\phi^-} \end{pmatrix} = \begin{pmatrix} \overline{\phi^+} \mathbin{\bar\circ} F(h^\pm) \mathbin{\bar\circ} \overline{\phi^-} \\ \overline{\phi^+} \mathbin{\bar\circ} F(h^\mp) \mathbin{\bar\circ} \overline{\phi^-} \end{pmatrix}$$

Let $\theta, \theta' \in [\![\Delta]\!]$; and $\varphi : \theta \rightleftharpoons \theta'$.
For $\vdash_\Delta \tau$ type, a function $\boxed{[\![\tau]\!]^{\mathrm{f}}(\varphi) : [\![\tau]\!]_\theta \rightharpoonup [\![\tau]\!]_{\theta'}}$:

$$
\begin{aligned}
[\![\alpha]\!]^{\mathrm{f}}(\varphi) &= f^+ \text{ where } \varphi(\alpha) = f^\pm \\
[\![b]\!]^{\mathrm{f}}(\varphi) &= \overline{id}_{\mathcal{B}^b} = \eta^\perp \\
[\![1]\!]^{\mathrm{f}}(\varphi) &= \overline{id}_{\mathbf{1}} = \lambda u.\, \eta^\perp * \\
[\![\tau_1 \times \tau_2]\!]^{\mathrm{f}}(\varphi) &= \lambda(a_1, a_2).\, [\![\tau_1]\!]^{\mathrm{f}}(\varphi)\, a_1 \star^\perp \lambda a_1'.\, [\![\tau_2]\!]^{\mathrm{f}}(\varphi)\, a_2 \star^\perp \lambda a_2'.\, \eta^\perp (a_1', a_2') \\
[\![0]\!]^{\mathrm{f}}(\varphi) &= \emptyset \quad \text{[empty function from } \emptyset \text{ to } \emptyset_\perp] \\
[\![\tau_1 + \tau_2]\!]^{\mathrm{f}}(\varphi) &= \lambda s.\, \text{case } s \text{ of } \begin{Bmatrix} in_1(a_1) \to [\![\tau_1]\!]^{\mathrm{f}}(\varphi)\, a_1 \star^\perp \lambda a_1'.\, \eta^\perp (in_1(a_1')) \\ in_2(a_2) \to [\![\tau_2]\!]^{\mathrm{f}}(\varphi)\, a_2 \star^\perp \lambda a_2'.\, \eta^\perp (in_2(a_2')) \end{Bmatrix} \\
[\![\mu\alpha.\tau]\!]^{\mathrm{f}}(\varphi) &= h^+ \text{ where, in the least-fixed-point sense,} \\
h^\pm &= \begin{pmatrix} \overline{\phi_{[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta'}}} \,\bar{\circ}\, [\![\tau]\!]^{\mathrm{f}}(\varphi[\alpha \mapsto h^\pm]) \,\bar{\circ}\, \overline{\phi^{-1}_{[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta}}} \\ \overline{\phi_{[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta}}} \,\bar{\circ}\, [\![\tau]\!]^{\mathrm{f}}(\varphi^\dagger[\alpha \mapsto h^\mp]) \,\bar{\circ}\, \overline{\phi^{-1}_{[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta'}}} \end{pmatrix} \\
&= \begin{pmatrix} \lambda m^{\mu[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta}}.\, [\![\tau]\!]^{\mathrm{f}}(\varphi[\alpha \mapsto h^\pm])\, (\phi^{-1}_{[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta}} m) \star^\perp \lambda a.\, \eta^\perp (\phi_{[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta'}} a) \\ \lambda m^{\mu[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta'}}.\, [\![\tau]\!]^{\mathrm{f}}(\varphi^\dagger[\alpha \mapsto h^\mp])\, (\phi^{-1}_{[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta'}} m) \star^\perp \lambda a.\, \eta^\perp (\phi_{[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta}} a) \end{pmatrix} \\
[\![\sigma]\!]^{\mathrm{f}}(\varphi) &= \overline{[\![\sigma]\!]^{\mathrm{cf}}(\varphi)} = \lambda d.\, \eta^\perp ([\![\sigma]\!]^{\mathrm{cf}}(\varphi)\, d)
\end{aligned}
$$

For $\vdash_\Delta \sigma$ e-type, a function $\boxed{[\![\sigma]\!]^{\mathrm{cf}}(\varphi) : [\![\sigma]\!]^{\mathrm{c}}_\theta \to [\![\sigma]\!]^{\mathrm{c}}_{\theta'}}$:

$$
\begin{aligned}
[\![1]\!]^{\mathrm{cf}}(\varphi) &= id_{\mathbf{1}} = \lambda u^{\mathbf{1}}.\, * \\
[\![\sigma_1 \times \sigma_2]\!]^{\mathrm{cf}}(\varphi) &= [\![\sigma_1]\!]^{\mathrm{cf}}(\varphi) \times [\![\sigma_2]\!]^{\mathrm{cf}}(\varphi) = \lambda(d_1, d_2).\, ([\![\sigma_1]\!]^{\mathrm{cf}}(\varphi)\, d_1, [\![\sigma_2]\!]^{\mathrm{cf}}(\varphi)\, d_2) \\
[\![\tau \to \sigma]\!]^{\mathrm{cf}}(\varphi) &= \lambda f.\, [\![\sigma]\!]^{\mathrm{s}\perp}_{\theta'} \circ (\overline{[\![\sigma]\!]^{\mathrm{cf}}(\varphi) \circ f} \,\bar{\circ}\, [\![\tau]\!]^{\mathrm{f}}(\varphi^\dagger)) \\
&= \lambda f.\, \lambda a'^{[\![\tau]\!]_{\theta'}}.\, [\![\sigma]\!]^{\mathrm{s}\perp}_{\theta'} ([\![\tau]\!]^{\mathrm{f}}(\varphi^\dagger)\, a' \star^\perp \lambda a^{[\![\tau]\!]_\theta}.\, \eta^\perp ([\![\sigma]\!]^{\mathrm{cf}}(\varphi)\, (f\, a))) \\
[\![\langle e' \rangle \tau]\!]^{\mathrm{cf}}(\varphi) &= \lambda t^{T^{e'}[\![\tau]\!]_\theta}.\, t \star^{e'} \lambda a^{[\![\tau]\!]_\theta}.\, i^{\perp, e'} ([\![\tau]\!]^{\mathrm{f}}(\varphi)\, a)
\end{aligned}
$$

For $\theta \in [\![\Delta]\!]$ and $\vdash_{\Delta,\alpha} \tau$ type, an m-functor $\boxed{[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta}$:

$$
[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta(A) = [\![\tau]\!]_{\theta[\alpha \mapsto A]} \qquad [\![\alpha.\tau]\!]^{\mathrm{m}}_\theta(f^\pm) = [\![\tau]\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta[\alpha \mapsto f^\pm])\,.
$$

Fig. 6. M-functorial actions of M^{3}L types.

$$
K(h) = \overline{\phi^+} \,\bar{\circ}\, [\![\tau_0]\!]^{\mathrm{f}} \left(\overline{\boldsymbol{id}}_\theta \left[\alpha \mapsto \begin{pmatrix} h \\ h \end{pmatrix} \right] \right) \,\bar{\circ}\, \overline{\phi^-} = \overline{\phi^+} \,\bar{\circ}\, F \begin{pmatrix} h \\ h \end{pmatrix} \,\bar{\circ}\, \overline{\phi^-}.
$$

Then $H\binom{h}{h} = \binom{K(h)}{K(h)}$. By induction, we also get that for all $n \in \omega$, $H^n\binom{\perp}{\perp} = \binom{K^n(\perp)}{K^n(\perp)}$: the case $n = 0$ is immediate, and

$$
H^{n+1}\begin{pmatrix} \perp \\ \perp \end{pmatrix} = H\left(H^n \begin{pmatrix} \perp \\ \perp \end{pmatrix} \right) \overset{\text{ih}}{=} H \begin{pmatrix} K^n(\perp) \\ K^n(\perp) \end{pmatrix} = \begin{pmatrix} K(K^n(\perp)) \\ K(K^n(\perp)) \end{pmatrix} = \begin{pmatrix} K^{n+1}(\perp) \\ K^{n+1}(\perp) \end{pmatrix}.
$$

Hence,

$$
\begin{aligned}
[\![\mu\alpha.\tau_0]\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta) &= \pi_+(fix\, H) = \pi_+ \left(\bigsqcup_{n \in \omega} H^n \begin{pmatrix} \perp \\ \perp \end{pmatrix} \right) = \pi_+ \left(\bigsqcup_{n \in \omega} \begin{pmatrix} K^n(\perp) \\ K^n(\perp) \end{pmatrix} \right) = \bigsqcup_n K^n(\perp) = fix\, K \\
&= fix(\lambda h.\, \overline{\phi_F} \,\bar{\circ}\, F \begin{pmatrix} h \\ h \end{pmatrix} \,\bar{\circ}\, \overline{\phi_F^{-1}}) = \overline{id}_{\mu F} = \overline{id}_{[\![\mu\alpha.\tau_0]\!]_\theta}
\end{aligned}
$$

where $\pi_+(f^\pm) = f^+$, and the penultimate equation follows because μF and ϕ_F constitute a minimal invariant for F.

The second half (composition) proceeds similarly. For $\varphi : \theta \rightleftharpoons \theta'$, let

$$H_\varphi(h^\pm) = \overline{\phi}^{\pm}_{[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta'}} \bar{\bullet} \begin{pmatrix} [\![\tau_0]\!]^{\mathrm{f}}(\varphi[\alpha \mapsto h^\pm]) \\ [\![\tau_0]\!]^{\mathrm{f}}(\varphi^\dagger[\alpha \mapsto h^\mp]) \end{pmatrix} \bar{\bullet} \overline{\phi}^{\mp}_{[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta}}.$$

Then, by the induction hypothesis for τ_0 and the equation $\overline{\phi}^{\mp}_{[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta'}} \bar{\bullet} \overline{\phi}^{\pm}_{[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta'}} = \overline{\boldsymbol{id}}$, it follows directly that

$$H_{\varphi_2 \bar{\bullet} \varphi_1}(h_2^\pm \bar{\bullet} h_1^\pm) = H_{\varphi_2}(h_2^\pm) \bar{\bullet} H_{\varphi_1}(h_1^\pm).$$

Again, by induction on n, we get $H^n_{\varphi_2 \bar{\bullet} \varphi_1}\binom{\perp}{\perp} = H^n_{\varphi_2}\binom{\perp}{\perp} \bar{\bullet} H^n_{\varphi_1}\binom{\perp}{\perp}$: the case $n = 0$ is because $\perp \bar{\circ} \perp = \perp$, and

$$\begin{aligned} H^{n+1}_{\varphi_2 \bar{\bullet} \varphi_1}\begin{pmatrix}\perp\\\perp\end{pmatrix} &= H_{\varphi_2 \bar{\bullet} \varphi_1}\left(H^n_{\varphi_2 \bar{\bullet} \varphi_1}\begin{pmatrix}\perp\\\perp\end{pmatrix}\right) \overset{\text{ih}}{=} H_{\varphi_2 \bar{\bullet} \varphi_1}\left(H^n_{\varphi_2}\begin{pmatrix}\perp\\\perp\end{pmatrix} \bar{\bullet} H^n_{\varphi_1}\begin{pmatrix}\perp\\\perp\end{pmatrix}\right) \\ &= H_{\varphi_2}\left(H^n_{\varphi_2}\begin{pmatrix}\perp\\\perp\end{pmatrix}\right) \bar{\bullet} H_{\varphi_1}\left(H^n_{\varphi_1}\begin{pmatrix}\perp\\\perp\end{pmatrix}\right) = H^{n+1}_{\varphi_2}\begin{pmatrix}\perp\\\perp\end{pmatrix} \bar{\bullet} H^{n+1}_{\varphi_1}\begin{pmatrix}\perp\\\perp\end{pmatrix}. \end{aligned}$$

Hence,

$$\begin{aligned} [\![\mu\alpha.\tau_0]\!]^{\mathrm{f}}(\varphi_2 \bar{\bullet} \varphi_1) &= \pi_+(\mathit{fix}\, H_{\varphi_2 \bar{\bullet} \varphi_1}) = \pi_+\left(\bigsqcup_n H^n_{\varphi_2 \bar{\bullet} \varphi_1}\begin{pmatrix}\perp\\\perp\end{pmatrix}\right) \\ &= \pi_+\left(\bigsqcup_n H^n_{\varphi_2}\begin{pmatrix}\perp\\\perp\end{pmatrix} \bar{\bullet} H^n_{\varphi_1}\begin{pmatrix}\perp\\\perp\end{pmatrix}\right) = \pi_+\left(\left(\bigsqcup_n H^n_{\varphi_2}\begin{pmatrix}\perp\\\perp\end{pmatrix}\right) \bar{\bullet} \left(\bigsqcup_n H^n_{\varphi_1}\begin{pmatrix}\perp\\\perp\end{pmatrix}\right)\right) \\ &= \pi_+(\mathit{fix}\, H_{\varphi_2} \bar{\bullet} \mathit{fix}\, H_{\varphi_1}) = \pi_+(\mathit{fix}\, H_{\varphi_2}) \bar{\circ} \pi_+(\mathit{fix}\, H_{\varphi_1}) \\ &= [\![\mu\alpha.\tau_0]\!]^{\mathrm{f}}(\varphi_2) \bar{\circ} [\![\mu\alpha.\tau_0]\!]^{\mathrm{f}}(\varphi_1). \end{aligned}$$

The cases for part (2) are again easy, with the case for $\tau \to \sigma$ also exploiting the $\mathcal{T}^\perp$-rigidity of $[\![\sigma]\!]^{\mathrm{cf}}(\varphi)$ (Lemma 21). Finally, the "in particular" is a direct consequence of part (1), with local continuity following from the explicit construction of the morphism action in terms of continuous operations. □

We also check that the m-functorial semantics behaves as expected with respect to substitution:

Lemma 23 (*Substitution, Types*). *For* $\vdash_\Delta \tau'$ type*;* $\theta, \theta' \in [\![\Delta]\!]$*; and* $\varphi \in \theta \rightleftharpoons \theta'$*, let* $A' = [\![\tau']\!]_\theta$ *and* $f'^\pm = \binom{[\![\tau']\!]^{\mathrm{f}}(\varphi)}{[\![\tau']\!]^{\mathrm{f}}(\varphi^\dagger)}$*. Then,*

(1) *If* $\vdash_{\Delta,\alpha'} \tau$ type *then* $[\![\tau[\tau'/\alpha']]\!]_\theta = [\![\tau]\!]_{\theta[\alpha' \mapsto A']}$*, and* $[\![\tau[\tau'/\alpha']]\!]^{\mathrm{f}}(\varphi) = [\![\tau]\!]^{\mathrm{f}}(\varphi[\alpha' \mapsto f'^\pm])$.
(2) *If* $\vdash_{\Delta,\alpha'} \sigma$ *e*-type *then* $[\![\sigma[\tau'/\alpha']]\!]^{\mathrm{c}}_\theta = [\![\sigma]\!]^{\mathrm{c}}_{\theta[\alpha' \mapsto A']}$, $[\![\sigma[\tau'/\alpha']]\!]^{se}_\theta = [\![\sigma]\!]^{se}_{\theta[\alpha' \mapsto A']}$, *and* $[\![\sigma[\tau'/\alpha']]\!]^{\mathrm{cf}}(\varphi) = [\![\sigma]\!]^{\mathrm{cf}}(\varphi[\alpha' \mapsto f'^\pm])$.

Proof. First, it is easy to check that if $\theta(\alpha) = \theta'(\alpha)$ for all $\alpha \in FTV(\tau')$, then $[\![\tau']\!]_\theta = [\![\tau']\!]_{\theta'}$, and similarly, $[\![\tau']\!]^{\mathrm{f}}(\varphi) = [\![\tau']\!]^{\mathrm{f}}(\varphi')$ when $\varphi(\alpha) = \varphi'(\alpha)$ for all such α.

The main proof is by mutual induction on the structure of τ and σ. The only interesting case is for part (1) with $\tau = \mu\alpha.\tau_0$, where we can assume that $\alpha \neq \alpha'$ and $\alpha \notin FTV(\tau')$. We first check that $[\![\alpha.\tau_0[\tau'/\alpha']]\!]^{\mathrm{m}}_\theta = [\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta[\alpha' \mapsto A']}$:

$$\begin{aligned} [\![\alpha.\tau_0[\tau'/\alpha']]\!]^{\mathrm{m}}_\theta(A) &= [\![\tau_0[\tau'/\alpha']]\!]_{\theta[\alpha \mapsto A]} \overset{\text{ih}}{=} [\![\tau_0]\!]_{\theta[\alpha \mapsto A][\alpha' \mapsto [\![\tau']\!]_{\theta[\alpha \mapsto A]}]} = [\![\tau_0]\!]_{\theta[\alpha \mapsto A][\alpha' \mapsto A']} \\ &= [\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta[\alpha' \mapsto A']}(A) \end{aligned}$$

$$\begin{aligned} [\![\alpha.\tau_0[\tau'/\alpha']]\!]^{\mathrm{m}}_\theta(f^\pm) &= [\![\tau_0[\tau'/\alpha']]\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta[\alpha \mapsto f^\pm]) \\ &\overset{\text{ih}}{=} [\![\tau_0]\!]^{\mathrm{f}}\left(\overline{\boldsymbol{id}}_\theta[\alpha \mapsto f^\pm]\left[\alpha' \mapsto \begin{pmatrix} [\![\tau']\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta[\alpha \mapsto f^\pm]) \\ [\![\tau']\!]^{\mathrm{f}}((\overline{\boldsymbol{id}}_\theta[\alpha \mapsto f^\pm])^\dagger) \end{pmatrix}\right]\right) \end{aligned}$$

$$
\begin{aligned}
&= [\![\tau_0]\!]^{\mathrm{f}}\left(\overline{\boldsymbol{id}}_\theta[\alpha \mapsto f^{\pm}]\left[\alpha' \mapsto \begin{pmatrix}[\![\tau']\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta)\\ [\![\tau']\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_\theta)\end{pmatrix}\right]\right)\\
&\overset{*}{=} [\![\tau_0]\!]^{\mathrm{f}}\left(\overline{\boldsymbol{id}}_\theta[\alpha \mapsto f^{\pm}]\left[\alpha' \mapsto \begin{pmatrix}\overline{id}_{[\![\tau']\!]_\theta}\\ \overline{id}_{[\![\tau']\!]_\theta}\end{pmatrix}\right]\right) = [\![\tau_0]\!]^{\mathrm{f}}(\overline{\boldsymbol{id}}_{\theta[\alpha' \mapsto A']}[\alpha \mapsto f^{\pm}])\\
&= [\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta[\alpha' \mapsto A']}(f^{\pm}).
\end{aligned}
$$

(The equation marked (*) uses Lemma 22(1), which again relies on the minimal-invariant property if τ' may contain a μ-type.)

Then, for the first half of part (1),

$$[\![(\mu\alpha.\tau_0)[\tau'/\alpha']]\!]_\theta = [\![\mu\alpha.\tau_0[\tau'/\alpha']]\!]_\theta = \mu[\![\alpha.\tau_0[\tau'/\alpha']]\!]^{\mathrm{m}}_\theta = \mu[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta[\alpha' \mapsto A']} = [\![\mu\alpha.\tau_0]\!]_{\theta[\alpha' \mapsto A']}.$$

For the second half, let $F = [\![\alpha.\tau_0[\tau'/\alpha']]\!]^{\mathrm{m}}_\theta = [\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta[\alpha' \mapsto A']}$ and $F' = [\![\alpha.\tau_0[\tau'/\alpha']]\!]^{\mathrm{m}}_{\theta'} = [\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta'[\alpha' \mapsto A']}$, and write $\phi^{\pm} = \binom{\phi_F}{\phi_F^{-1}}$ and $\phi'^{\pm} = \binom{\phi_{F'}}{\phi_{F'}^{-1}}$. Then

$$
\begin{aligned}
[\![(\mu\alpha.\tau_0)[\tau'/\alpha']]\!]^{\mathrm{f}}(\varphi) &= [\![\mu\alpha.\tau_0[\tau'/\alpha']]\!]^{\mathrm{f}}(\varphi)\\
&= \pi_+\left(fix\left(\lambda h^{\pm}.\overline{\phi}'^{\pm} \bullet \begin{pmatrix}[\![\tau_0[\tau'/\alpha']]\!]^{\mathrm{f}}(\varphi[\alpha \mapsto h^{\pm}])\\ [\![\tau_0[\tau'/\alpha']]\!]^{\mathrm{f}}(\varphi^{\dagger}[\alpha \mapsto h^{\mp}])\end{pmatrix} \bullet \overline{\phi}^{\mp}\right)\right)\\
&\overset{\text{ih}}{=} \pi_+\left(fix\left(\lambda h^{\pm}.\overline{\phi}'^{\pm} \bullet \begin{pmatrix}[\![\tau_0]\!]^{\mathrm{f}}(\varphi[\alpha \mapsto h^{\pm}][\alpha' \mapsto f'^{\pm}])\\ [\![\tau_0]\!]^{\mathrm{f}}(\varphi^{\dagger}[\alpha \mapsto h^{\mp}][\alpha' \mapsto f'^{\mp}])\end{pmatrix} \bullet \overline{\phi}^{\mp}\right)\right)\\
&= \pi_+\left(fix\left(\lambda h^{\pm}.\overline{\phi}'^{\pm} \bullet \begin{pmatrix}[\![\tau_0]\!]^{\mathrm{f}}(\varphi[\alpha' \mapsto f'^{\pm}][\alpha \mapsto h^{\pm}])\\ [\![\tau_0]\!]^{\mathrm{f}}((\varphi[\alpha' \mapsto f'^{\pm}])^{\dagger}[\alpha \mapsto h^{\mp}])\end{pmatrix} \bullet \overline{\phi}^{\mp}\right)\right)\\
&= [\![\mu\alpha.\tau_0]\!]^{\mathrm{f}}(\varphi[\alpha' \mapsto f^{\pm}]). \quad \square
\end{aligned}
$$

2.5. Semantics of terms

Given meanings for all well-formed types, we can complete the definition of interpretations:

Definition 24. An *interpretation* of an M^{3}L signature Σ consists of a type-and-effect interpretation from Definition 14, together with, for every constant $(c\colon \tau) \in \Sigma$, an element $\mathcal{C}^c \in [\![\tau]\!]_{[]}$.

Given such an interpretation, we define meanings of well-typed terms in Fig. 7. The meaning of **void** (M) is given in terms of the (unique) empty function from $[\![0]\!]_\theta = \emptyset$ to $[\![\tau]\!]_\theta$. (M will usually be a 0-typed variable, representing the nominal return value of a non-returning computation; cf. c_{err} and M_{err} in Section 2.7.)

The clauses for **in** and **out** are type-correct because, by Lemma 23,

$$[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta([\![\mu\alpha.\tau]\!]_\theta) = [\![\tau]\!]_{\theta[\alpha \mapsto [\![\mu\alpha.\tau]\!]_\theta]} = [\![\tau[(\mu\alpha.\tau)/\alpha]]\!]_\theta$$

and thus $\phi_{[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta}\colon [\![\tau[(\mu\alpha.\tau)/\alpha]]\!]_\theta \to [\![\mu\alpha.\tau]\!]_\theta$, and conversely for $\phi^{-1}_{[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta}$.

We also state, for reference, a standard result:

Lemma 25 (*Substitution, Terms*). *If* $\Gamma \vdash_\Delta M_1 : \tau_1$ *and* $\Gamma, x\colon \tau_1 \vdash_\Delta M_2 : \tau_2$ *then* $[\![M_2[M_1/x]]\!]\rho = [\![M_2]\!]\rho[x \mapsto [\![M_1]\!]\rho]$.

The derived forms have the expected meanings:

$$
\begin{aligned}
[\![\mathbf{let}^e_\tau\ x \Leftarrow M_1.\, M_2]\!]_\theta\,\rho &= [\![M_1]\!]_\theta\,\rho \star^e \lambda a.[\![M_2]\!]\rho[x \mapsto a]\\
[\![\mathbf{inc}^{e,e'}_\tau\ M]\!]_\theta\,\rho &= i^{e,e'}([\![M]\!]\rho)
\end{aligned}
$$

(a straightforward calculation in both cases).

Let $\theta \in [\![\Delta]\!]$.
For $\vdash_\Delta \Gamma$ env, a cpo $\boxed{[\![\Gamma]\!]_\theta} = \prod_{(x:\tau)\in\Gamma} [\![\tau]\!]_\theta = \{\rho \mid \forall (x:\tau) \in \Gamma. \rho\, x \in [\![\tau]\!]_\theta\}$
For $\Gamma \vdash_\Delta M : \tau$, a function $\boxed{[\![M]\!]_\theta : [\![\Gamma]\!]_\theta \to [\![\tau]\!]_\theta}$:

$$
\begin{aligned}
[\![x]\!]_\theta\, \rho &= \rho\, x \\
[\![c]\!]_\theta\, \rho &= \mathcal{C}^c \\
[\![()]\!]_\theta\, \rho &= * \\
[\![(M_1, M_2)]\!]_\theta\, \rho &= ([\![M_1]\!]_\theta\, \rho, [\![M_2]\!]_\theta\, \rho) \\
[\![\mathbf{fst}(M)]\!]_\theta\, \rho &= \pi_1([\![M_1]\!]_\theta\, \rho) \\
[\![\mathbf{snd}(M)]\!]_\theta\, \rho &= \pi_2([\![M_2]\!]_\theta\, \rho) \\
[\![\mathbf{inl}\,(M)]\!]_\theta\, \rho &= in_1([\![M]\!]_\theta\, \rho) \\
[\![\mathbf{inr}\,(M)]\!]_\theta\, \rho &= in_2([\![M]\!]_\theta\, \rho) \\
[\![\mathbf{case}\ M\ \mathbf{of\ inl}\,(x_1).\, M_1\ []\ \mathbf{inr}\,(x_2).\, M_2]\!]_\theta\, \rho & \\
&= \text{case } [\![M]\!]_\theta\, \rho \text{ of } \left\{\begin{array}{l} in_1(a_1) \to [\![M_1]\!]_\theta\, \rho[x_1 \mapsto a_1] \\ in_2(a_2) \to [\![M_2]\!]_\theta\, \rho[x_2 \mapsto a_2] \end{array}\right\} \\
[\![\mathbf{void}\,(M)]\!]_\theta\, \rho &= \emptyset([\![M]\!]_\theta\, \rho) \\
[\![\mathbf{in}_{\mu\alpha.\tau}(M)]\!]_\theta\, \rho &= \phi_{[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta}([\![M]\!]_\theta\, \rho) \\
[\![\mathbf{out}_{\mu\alpha.\tau}(M)]\!]_\theta\, \rho &= \phi^{-1}_{[\![\alpha.\tau]\!]^{\mathrm{m}}_\theta}([\![M]\!]_\theta\, \rho) \\
[\![\lambda x^\tau.\, M]\!]_\theta\, \rho &= \lambda a.\, [\![M]\!]_\theta\, \rho[x \mapsto a] \\
[\![M_1\, M_2]\!]_\theta\, \rho &= [\![M_1]\!]_\theta\, \rho\, ([\![M_2]\!]_\theta\, \rho) \\
[\![\mathbf{val}^e\ M]\!]_\theta\, \rho &= \eta^e([\![M]\!]_\theta\, \rho) \\
[\![\mathbf{glet}^e_\sigma\ x \Leftarrow M_1.\, M_2]\!]_\theta\, \rho &= [\![\sigma]\!]^{\mathrm{se}}_\theta([\![M_1]\!]_\theta\, \rho \star^e \lambda a.\, \eta^e([\![M_2]\!]_\theta\, \rho[x \mapsto a])) \\
[\![\mu x^\sigma.M]\!]_\theta\, \rho &= fix(\lambda d.\, [\![M]\!]_\theta\, \rho[x \mapsto d])
\end{aligned}
$$

Fig. 7. Semantics of $\mathrm{M}^3\mathrm{L}$ terms.

The following equations are also easily seen to hold semantically; they generalize the usual let-reasoning rules from the monadic metalanguage:

$$
\begin{aligned}
\mathbf{glet}^e_\sigma\ x \Leftarrow \mathbf{val}^e\ M_1.\, M_2 &= M_2[M_1/x] \\
\mathbf{glet}^e_{\langle e\rangle\tau}\ x \Leftarrow M.\, \mathbf{val}^e\ x &= M \\
\mathbf{glet}^{e_2}_\sigma\ y \Leftarrow (\mathbf{glet}^{e_1}_{\langle e_2\rangle\tau_2}\ x \Leftarrow M_1.\, M_2).\, M_3 &= \mathbf{glet}^{e_1}_\sigma\ x \Leftarrow M_1.\, \mathbf{glet}^{e_2}_\sigma\ y \Leftarrow M_2.\, M_3 \quad [x \notin FV(M_3)].
\end{aligned}
$$

We also have equations for decomposing the **glet**-form according to the type of its body:

$$
\begin{aligned}
\mathbf{glet}^e_1\ x \Leftarrow M_1.\, M_2 &= () \\
\mathbf{glet}^e_{\sigma_1\times\sigma_2}\ x \Leftarrow M_1.\, M_2 &= (\mathbf{glet}^e_{\sigma_1}\ x \Leftarrow M_1.\, \mathbf{fst}(M_2), \mathbf{glet}^e_{\sigma_2}\ x \Leftarrow M_1.\, \mathbf{snd}(M_2)) \\
\mathbf{glet}^e_{\tau\to\sigma}\ x \Leftarrow M_1.\, M_2 &= \lambda a^\tau.\, \mathbf{glet}^e_\sigma\ x \Leftarrow M_1.\, M_2\, a.
\end{aligned}
$$

It is straightforward to extend these laws to a sound equational theory for the metalanguage, by adding the usual strong reasoning principles for the remaining constructs, e.g., β- and η-laws for function-typed terms.

2.6. *Defining monads syntactically*

Most monads proposed for modeling computational effects [18], with the notable exception of powerdomain constructions, can actually be defined explicitly in the metalanguage. (This will later allow us to define their canonical relational interpretations in Section 3.5.1.)

Definition 26. For notational convenience, let χ be some distinguished type variable; then we write $\sigma(\tau)$ for $\sigma[\tau/\chi]$. A *formal monad over an effect e* consists of a triple $(\sigma_T, M_\eta, M_\star)$ such that

- $\vdash_\alpha \sigma_T(\alpha)$ e-type
- $\vdash_\alpha M_\eta : \alpha \to \sigma_T(\alpha)$
- $\vdash_{\alpha,\beta} M_\star : \sigma_T(\alpha) \times (\alpha \to \sigma_T(\beta)) \to \sigma_T(\beta)$.

Given such a formal monad and an effect name e', we can consider a possible interpretation of e' as a monad layered over $\mathcal{T}^e$:

$$T^{e'}A = [\![\sigma_T]\!]^{\mathrm{c}}_{[\chi\mapsto A]} \qquad \eta^{e'}_A a = [\![M_\eta]\!]_{[\alpha\mapsto A]}[]\,a$$
$$\gamma^{e,e'}_A = [\![\sigma_T]\!]^{se}_{[\chi\mapsto A]} \qquad t \star^{e'}_{A,B} f = [\![M_\star]\!]_{[\alpha\mapsto A,\beta\mapsto B]}[]\,(t, f).$$

Suppose these components actually satisfy the monad laws, as well as the rigidity condition (9) for layering, which can be expressed in M^{3}L syntax as

$$[\![M_\star\,(\mathbf{glet}^e_{\sigma_T(\alpha)}\ d \Leftarrow t.\,d,\ f)]\!] = [\![\mathbf{glet}^e_{\sigma_T(\beta)}\ d \Leftarrow t.\,M_\star(d, f)]\!] \quad [t : \langle e\rangle\sigma_T(\alpha),\ f : \alpha \to \sigma_T(\beta)].$$

Then by Lemma 13 we can also take

$$i^{e,e'}_A = \lambda t.\,\gamma^{e,e'}_A\,(t \star^e_{A,T'A} \lambda a.\,\eta^e_{T'A}(\eta^{e'}_A a)) = [\![\lambda t^{\langle e\rangle\alpha}.\,\mathbf{glet}^e_{\sigma_T(\alpha)}\ a \Leftarrow t.\,M_\eta\,a]\!]^{[\alpha\mapsto A]}[]. \tag{13}$$

Moreover, this interpretation of $e \preceq e'$ is compatible with all effects below e: suppose $e^\setminus \preceq e \preceq e'$; then, by Lemma 16 we have $[\![\sigma]\!]^{se^\setminus}_\theta = [\![\sigma]\!]^{se}_\theta \circ i^{e^\setminus,e}$. Hence, for $\theta = [\chi \mapsto A]$,

$$\begin{aligned} i^{e^\setminus,e'} &= \lambda t^\setminus.[\![\sigma_T]\!]^{se^\setminus}_\theta\,(t^\setminus \star^{e^\setminus} \lambda a.\,\eta^{e^\setminus}(\eta^{e'} a)) = \lambda t^\setminus.[\![\sigma_T]\!]^{se}_\theta\,(i^{e^\setminus,e}\,(t^\setminus \star^{e^\setminus} \lambda a.\,\eta^{e^\setminus}(\eta^{e'} a))) \\ &= \lambda t^\setminus.[\![\sigma_T]\!]^{se}_\theta\,(i^{e^\setminus,e} t^\setminus \star^e \lambda a.\,i^{e^\setminus,e}(\eta^{e^\setminus}(\eta^{e'} a))) = \lambda t^\setminus.[\![\sigma_T]\!]^{se}_\theta\,(i^{e^\setminus,e} t^\setminus \star^e \lambda a.\,\eta^e(\eta^{e'} a)) = \lambda t^\setminus.i^{e,e'}\,(i^{e^\setminus,e} t^\setminus) \\ &= i^{e,e'} \circ i^{e^\setminus,e}. \end{aligned}$$

Example 27. Let σ_{o} be a closed e-type. Then we define the formal *continuation monad with answer type* σ_{o} by

$$\begin{aligned} \sigma^{\mathsf{k}}_T(\alpha) &\equiv (\alpha \to \sigma_{\mathrm{o}}) \to \sigma_{\mathrm{o}} \\ M^{\mathsf{k}}_\eta &\equiv \lambda a^\alpha.\lambda k^{\alpha\to\sigma_{\mathrm{o}}}.k\,a \\ M^{\mathsf{k}}_\star &\equiv \lambda(t^{(\alpha\to\sigma_{\mathrm{o}})\to\sigma_{\mathrm{o}}},\ f^{\alpha\to(\beta\to\sigma_{\mathrm{o}})\to\sigma_{\mathrm{o}}}).\lambda k^{\beta\to\sigma_{\mathrm{o}}}.t\,(\lambda a^\alpha.\,f\,a\,k). \end{aligned}$$

Let $O = [\![\sigma_{\mathrm{o}}]\!]^{\mathrm{c}}$ and $\gamma_{\mathrm{o}} = [\![\sigma_{\mathrm{o}}]\!]^{se}$. Then the formal monad determines a semantic monad $\mathcal{T}^{\mathsf{k}}$ layered above $\mathcal{T}^e$:

$$T^{\mathsf{k}}A = (A \to O) \to O \qquad \eta^{\mathsf{k}} a = \lambda k.\,k\,a$$
$$\gamma^{e,\mathsf{k}} t = \lambda k.\,\gamma_{\mathrm{o}}\,(t \star^e \lambda g.\,\eta^e\,(g\,k)) \qquad t' \star^{\mathsf{k}} f = \lambda k.\,t'\,(\lambda a.\,f\,a\,k).$$

It is easy to check that η^{k} and $\star^{\mathsf{k}}$ satisfy the three monad laws and the rigidity requirement.

In the common special case where $\sigma_{\mathrm{o}} = \langle e\rangle\tau'$ for some τ', we get $\gamma_{\mathrm{o}}\,t = t \star^e id$ and $i^{e,\mathsf{k}} t = \lambda k.\,t \star^e k$. □

An important reason for allowing nested recursive types in the metalanguage is that they allow us to consider recursively typed object programs using effects that are themselves recursively defined, e.g., by the formal list monad with $\sigma_T(\chi) = \langle\bot\rangle\mu\alpha.1 + \chi \times \alpha$.

2.7. Interpreting the object language

Let us now construct a translation of our sample object language into the metalanguage. We first need to fix a sufficiently rich M^{3}L signature. This should contain at least a type of natural numbers $b_{\mathbf{N}}$, with constants c_n for all numerals, and an addition operation c_+.[1] It also includes two effects. The first, o, captures the *observable* behavior of

[1] Since we have chosen to model numbers as a base type, the addition constant has a functional type, which formally requires its codomain to be pointed, making the translation a bit more complicated. We could also have taken $\tau_{\mathbf{N}} \equiv \mu\alpha.1 + \alpha$; then the successor of $n : \tau_{\mathbf{N}}$ could be expressed directly as $n + 1 \equiv \mathbf{in}_{\tau_N}(\mathbf{inr}\,(n)) : \tau_{\mathbf{N}}$.

Signature:

- Base types: $b_{\mathbf{N}}$.
- Effects: $\bot \preceq \mathtt{o} \preceq \mathtt{c}$.
- Constants: $c_n : b_{\mathbf{N}}$ $[\forall n \in \mathbf{N}]$, $c_+ : b_{\mathbf{N}} \times b_{\mathbf{N}} \to \langle\bot\rangle b_{\mathbf{N}}$, $c_{\mathrm{err}} : \langle\mathtt{o}\rangle 0$, $c_{\mathrm{run}} : \langle\mathtt{c}\rangle b_{\mathbf{N}} \to \langle\mathtt{o}\rangle b_{\mathbf{N}}$.

Abbreviations:

- Types: $\tau_E \equiv \mu\alpha.b_{\mathbf{N}} + (\langle\mathtt{c}\rangle\alpha \to \langle\mathtt{c}\rangle\alpha)$, $\sigma_D \equiv \langle\mathtt{c}\rangle\tau_E$.
- Terms: for $\vdash \sigma$ o-type, $M^{\sigma}_{\mathrm{err}} \equiv \mathbf{glet}^{\mathtt{o}}_{\sigma}\ v \Leftarrow c_{\mathrm{err}}.\ \mathbf{void}_{\sigma}\,(v) : \sigma$

Translation of terms: When $FV(t) \subseteq \{x_1, \ldots, x_n\}$, define $x_1 : \sigma_D, \ldots, x_n : \sigma_D \vdash (\!|t|\!)^{\mathrm{s}} : \sigma_D$ by

$$
\begin{aligned}
(\!|x|\!)^{\mathrm{s}} &= x \\
(\!|\underline{n}|\!)^{\mathrm{s}} &= \mathbf{val}^{\mathtt{c}}\ \mathbf{in}_{\tau_E}(\mathbf{inl}\,(c_n)) \\
(\!|\mathrm{succ}|\!)^{\mathrm{s}} &= \mathbf{val}^{\mathtt{c}}\ \mathbf{in}_{\tau_E}(\mathbf{inr}(\lambda d.\,\mathbf{let}^{\mathtt{c}}_{\tau_E}\ e \Leftarrow d.\,\mathbf{case}\ \mathbf{out}_{\tau_E}(e)\ \mathbf{of} \\
&\qquad\qquad \mathbf{inl}\,(n).\,\mathbf{glet}^{\bot}_{\sigma_D}\ n' \Leftarrow c_+(n, c_1).\mathbf{val}^{\mathtt{c}}\ \mathbf{in}_{\tau_E}(\mathbf{inl}\,(n')). \\
&\qquad\qquad [\!]\ \mathbf{inr}\,(f).\,M^{\sigma_D}_{\mathrm{err}})) \\
(\!|\lambda^{\mathrm{n}} x.t|\!)^{\mathrm{s}} &= \mathbf{val}^{\mathtt{c}}\ \mathbf{in}_{\tau_E}(\mathbf{inr}\,(\lambda x.\,(\!|t|\!)^{\mathrm{s}})) \\
(\!|\lambda^{\mathrm{v}} x.t|\!)^{\mathrm{s}} &= \mathbf{val}^{\mathtt{c}}\ \mathbf{in}_{\tau_E}(\mathbf{inr}\,(\lambda d.\,\mathbf{glet}^{\mathtt{c}}_{\sigma_D}\ e \Leftarrow d.\,(\lambda x.\,(\!|t|\!)^{\mathrm{s}})\,(\mathbf{val}^{\mathtt{c}}\ e))) \\
(\!|t_1\ t_2|\!)^{\mathrm{s}} &= \mathbf{let}^{\mathtt{c}}_{\tau_E}\ e \Leftarrow (\!|t_1|\!)^{\mathrm{s}}.\,\mathbf{case}\ \mathbf{out}_{\tau_E}(e)\ \mathbf{of}\ \mathbf{inl}\,(n).\,M^{\sigma_D}_{\mathrm{err}}\ [\!]\ \mathbf{inr}\,(f).\,f\,(\!|t_2|\!)^{\mathrm{s}}
\end{aligned}
$$

where variable names introduced by the translation are assumed distinct from those occurring in t.

Translation of programs: When $FV(t) = \emptyset$, define $\vdash (\!|t|\!)^{\mathrm{S}} : \langle\mathtt{o}\rangle b_{\mathbf{N}}$ by

$$(\!|t|\!)^{\mathrm{S}} = c_{\mathrm{run}}\,(\mathbf{let}^{\mathtt{c}}_{b_{\mathbf{N}}}\ e \Leftarrow (\!|t|\!)^{\mathrm{s}}.\,\mathbf{case}\ \mathbf{out}_{\tau_E}(e)\ \mathbf{of}\ \mathbf{inl}\,(n).\,\mathbf{val}^{\mathtt{c}}\ n\ [\!]\ \mathbf{inr}\,(f).\,M^{\langle\mathtt{c}\rangle b_{\mathbf{N}}}_{\mathrm{err}})$$

Fig. 8. Translation of object language into $\mathrm{M}^3\mathrm{L}$.

complete programs, which includes at least divergence, and may or may not include an explicit error abort. We capture the notion of type errors as a constant $c_{\mathrm{err}} : \langle\mathtt{o}\rangle 0$, representing a (necessarily) failed computation of an element of type 0. From this constant, we can uniformly build representations of failure at other o-types, using the **glet**-construct.

The second effect, c, is the effect used to model computation in the semantics itself. It must cover at least the externally observable behaviors, $\mathtt{o} \preceq \mathtt{c}$, but may be richer, such as including general control- or state-affecting behaviors. However, ultimately all such computational behaviors must be mappable to something observable. Since we expect all programs to return numeric output, we include just a single constant $c_{\mathrm{run}} : \langle\mathtt{c}\rangle b_{\mathbf{N}} \to \langle\mathtt{o}\rangle b_{\mathbf{N}}$.

The complete translation is shown in Fig. 8. Note that we map (typeless) variables in the object program to variables of type σ_D in the metalanguage.

Let us now formally relate the translation to the original two denotational semantics. We consider two complete domain-theoretic interpretations of the effects, base types, and constants, which we call the *primary* and *secondary* ones. Each will induce a semantics of $\mathrm{M}^3\mathrm{L}$ types and terms, $'[\![-]\!]$ and $''[\![-]\!]$. We choose the interpretations in such a way that the primary semantics of a translated object program coincides with the original direct semantics of the program, while the secondary semantics yields the original continuation semantics.

In both semantics, we will always interpret $b_{\mathbf{N}}$ as the natural numbers, with the standard meanings of the associated constants:

$$
\begin{aligned}
&\mathcal{B}^{b_{\mathbf{N}}}_{\mathrm{I}} = \mathcal{B}^{b_{\mathbf{N}}}_{\mathrm{II}} = \mathbf{N} \qquad \mathcal{C}^{c_n}_{\mathrm{I}} = \mathcal{C}^{c_n}_{\mathrm{II}} = n \in \mathbf{N} \\
&\mathcal{C}^{c_+}_{\mathrm{I}} = \mathcal{C}^{c_+}_{\mathrm{II}} = \lambda(n_1, n_2).\lfloor n_1 + n_2 \rfloor \in \mathbf{N} \times \mathbf{N} \to \mathbf{N}_{\bot}.
\end{aligned}
$$

This leaves the effects o and c, and the constants c_{err} and c_{run}.

2.7.1. Primary semantics

Here, we interpret both observable and internal effects as simple domain-theoretic lifting (with the trivial effect-inclusion), and failure as divergence:

$$\mathcal{T}_{\mathrm{I}}^{\mathsf{O}} = \mathcal{T}_{\mathrm{I}}^{\mathsf{C}} = \mathcal{T}^{\perp} \qquad \mathcal{C}_{\mathrm{I}}^{c_{\mathrm{err}}} = \perp \in \emptyset_{\perp} \qquad \mathcal{C}_{\mathrm{I}}^{c_{\mathrm{run}}} = id \in \mathbf{N}_{\perp} \to \mathbf{N}_{\perp}.$$

Since minimal-invariant solutions of recursive domain equations (including the standard inverse-limit construction) are unique up to isomorphism, we can assume wlog that $E^{\mathrm{d}} = {}^{\mathrm{I}}[\![\tau_E]\!][]$, with $\iota_{\mathrm{n}}^{\mathrm{d}}(n) = \phi(in_1(n))$ and $\iota_{\mathrm{f}}^{\mathrm{d}}(f) = \phi(in_2(f))$. We then also get $D^{\mathrm{d}} = {}^{\mathrm{I}}[\![\sigma_D]\!][]$. Further, clearly

$${}^{\mathrm{I}}[\![M_{\mathrm{err}}^{\sigma}]\!][] = \perp \star^{\perp} \lambda v.\emptyset\, v = \perp_{{}^{\mathrm{I}}[\![\sigma]\!]}.$$

And then it is easy to verify that

$$[\![t]\!]^{\mathrm{d}}\rho = {}^{\mathrm{I}}[\![\langle\!|t|\!\rangle^{\mathrm{S}}]\!]\rho|_X \quad [FV(t) \subseteq X] \quad \text{and} \quad [\![t]\!]^{\mathrm{D}} = {}^{\mathrm{I}}[\![\langle\!|t|\!\rangle^{\mathrm{S}}]\!][] \quad [FV(t) = \emptyset].$$

2.7.2. Secondary semantics

In the secondary semantics, the observable behaviors are those of diverging, returning a result, and aborting with an error; these can be modeled with the *divergence-and-errors* monad $\mathcal{T}^{\mathsf{e}}$, also definable syntactically as

$$\begin{aligned}
\sigma_T^{\mathsf{e}}(\alpha) &\equiv \langle\perp\rangle(\alpha+1)\\
M_{\eta}^{\mathsf{e}} &\equiv \lambda a^{\alpha}.\,\mathbf{val}^{\perp}\,\mathbf{inl}\,(a)\\
M_{\star}^{\mathsf{e}} &\equiv \lambda(t^{\langle\perp\rangle(\alpha+1)},\, f^{\alpha\to\langle\perp\rangle(\beta+1)}).\,\mathbf{let}^{\perp}\, s \Leftarrow t.\,\mathbf{case}\, s\, \mathbf{of}\, \mathbf{inl}\,(a).\, f\, a\, []\, \mathbf{inr}\,(u).\, \mathbf{val}^{\perp}\, \mathbf{inr}\,(u).
\end{aligned}$$

We then take $\mathcal{T}_{\mathrm{II}}^{\mathsf{O}} = \mathcal{T}^{\mathsf{e}}$. The monad morphism from lifting is given by

$$i_{\mathrm{II}}^{\perp,\mathsf{O}} = {}^{\mathrm{II}}[\![\lambda t.\,\mathbf{let}^{\perp}\, a \Leftarrow t.\, M_{\eta}^{\mathsf{e}}\, a]\!][] = \lambda t.\, t \star^{\perp} \eta_{\mathrm{II}}^{\mathsf{O}}.$$

The unit and bind operations capture the usual semantics of error propagation in *direct* style. It would have been perfectly reasonable to simply take $\mathcal{T}_{\mathrm{II}}^{\mathsf{C}} = \mathcal{T}_{\mathrm{II}}^{\mathsf{O}}$, resulting in an explicitly error-checking direct semantics. However, we take a more general monad for C, both to capture the original continuation semantics, and to illustrate how continuation models other effects. Accordingly, we interpret C as the continuation monad (Example 27) with answer type $\langle\mathsf{O}\rangle b_{\mathbf{N}}$. Then we again have $E^{\mathrm{c}} = {}^{\mathrm{II}}[\![\tau_E]\!][]$ and $D^{\mathrm{c}} = {}^{\mathrm{II}}[\![\sigma_D]\!][]$.

For constants, we interpret c_{err} as an explicit abort, and c_{run} as evaluation with the initial continuation:

$$\begin{aligned}
\mathcal{C}_{\mathrm{II}}^{c_{\mathrm{err}}} &= \lfloor in_2(*)\rfloor \in (\emptyset + \mathbf{1})_{\perp} = T^{\mathsf{O}}\emptyset\\
\mathcal{C}_{\mathrm{II}}^{c_{\mathrm{run}}} &= \lambda t.\, t\, \eta_{\mathrm{II}}^{\mathsf{O}} = \lambda t.\, t\, (\lambda n.\, \lfloor in_1(n)\rfloor) \in ((\mathbf{N} \to O) \to O) \to O.
\end{aligned}$$

Then

$${}^{\mathrm{II}}[\![M_{\mathrm{err}}^{\langle\mathsf{C}\rangle\tau}]\!][] = \lambda k.\, \lfloor in_2(*)\rfloor \star_{\mathrm{II}}^{\mathsf{O}} \lambda v.\emptyset\, v\, k = \lambda k.\, \lfloor in_2(*)\rfloor,$$

and we again get

$$[\![t]\!]^{\mathrm{c}}\rho = {}^{\mathrm{II}}[\![\langle\!|t|\!\rangle^{\mathrm{S}}]\!]\rho|_X \quad [FV(t) \subseteq X] \quad \text{and} \quad [\![t]\!]^{\mathrm{C}} = {}^{\mathrm{II}}[\![\langle\!|t|\!\rangle^{\mathrm{S}}]\!][] \quad [FV(t) = \emptyset].$$

Remark 28. We could have specified the original semantics of by-value abstractions slightly differently, by defining $[\![-]\!]^{\mathrm{d}'}$ and $[\![-]\!]^{\mathrm{c}'}$ exactly like the non-primed variants, except for the following clauses:

$$\begin{aligned}
[\![\lambda^{\mathrm{v}} x.\, t]\!]^{\mathrm{d}'} &= \lambda\rho.\lfloor \iota_{\mathrm{f}}^{\mathrm{d}}(\lambda d.(\lambda e.[\![t]\!]^{\mathrm{d}'}\rho[x \mapsto d])^{*} d)\rfloor\\
[\![\lambda^{\mathrm{v}} x.\, t]\!]^{\mathrm{c}'} &= \lambda\rho.\lambda k.k\,(\iota_{\mathrm{f}}^{\mathrm{c}}(\lambda d.\lambda k.d\,(\lambda e.[\![t]\!]^{\mathrm{c}'}\rho[x \mapsto d]\,k))).
\end{aligned}$$

That is, we force evaluation of the function argument, but then throw away the result, and behave like the by-name abstraction. These semantics evidently correspond, in the same sense as above, to the following translation into the metalanguage:

$$\langle\!|\lambda^{\mathrm{v}} x.\, t|\!\rangle^{\mathrm{s}'} = \mathbf{val}^{\mathsf{C}}\, \mathbf{in}_{\tau_E}(\mathbf{inr}\,(\lambda x.\, \mathbf{glet}_{\sigma_D}^{\mathsf{C}}\, e \Leftarrow x.\, \langle\!|t|\!\rangle^{\mathrm{s}'})) \quad [e \notin FV(t)].$$

Now, we clearly have $[\![t]\!]^{\mathrm{d}} = [\![t]\!]^{\mathrm{d}'}$ (because partiality is an *idempotent* effect), so it does not matter which formulation we use for the direct-style variant. On the other hand, in general $[\![t]\!]^{\mathrm{c}} \neq [\![t]\!]^{\mathrm{c}'}$. However, for the purpose of evaluating complete programs, the two variants are still indistinguishable. Indeed, the arguments in the following require no modification to show observational agreement between (either formulation of) the direct semantics and the alternative continuation semantics. □

3. Relating interpretations of the metalanguage

Having defined the semantics of the metalanguage with respect to interpretations of base types, effects, and constants, we now shift our attention to establishing correspondences between multiple such interpretations. We will do so as an instance of the logical-relations principle: for every well-formed metalanguage type, we define a relation between the cpos interpreting that type, and then show that the various meanings of every well-typed term are related at the term's type. However, whereas a standard logical relation is fully determined from relational interpretations of the base types, now we will also need interpretations as *relational actions* of all the effects.

We will formulate all results in terms of relating arbitrary primary and secondary semantics, $'[\![-]\!]$ and $''[\![-]\!]$. Often, the former will act as a reasoning-oriented *specification* semantics, while the latter will be an efficiency-oriented *implementation* semantics, but the framework imposes no particular such interpretations: all constructions are completely symmetric with respect to the roles of the primary and secondary semantics.

To visually distinguish pairs arising from working with exactly two semantics, from other two-component constructions, we will use angle brackets ($\langle, \rangle$) exclusively to group the former kind. It is completely straightforward to generalize all of the following results to n-ary relations, especially unary ones. With a little more work, many of the constructions should also adapt to syntax–semantics relations (such as are used for establishing a correspondence between an operational and a denotational semantics [28,25]), or even syntax–syntax, as might be used to relate two operational semantics.

3.1. Semantical preliminaries

Definition 29 (*Admissible Relation*). A relation $R \subseteq A_{\mathrm{I}} \times A_{\mathrm{II}}$ is called *admissible* (a.k.a. *inclusive*) if for any pair of chains $\langle (a_{n\iota})_{n\in\omega}\rangle^{\iota\in\mathrm{I},\mathrm{II}}$, such that $\forall n \in \omega.\langle a_{n\iota}\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R$, also $\langle \bigsqcup_{n\in\omega} a_{n\iota}\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R$. We write $\mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$ for the set of all such relations.

The immediate importance of admissible relations is due to the following lemma, which will allow us to relate recursively defined terms. Later we will see that – much like continuity not only allows us to give a general semantics of recursive terms, but also plays a key role in interpreting recursive types – admissibility is also crucial for constructing relations between recursive types.

Lemma 30 (*Fixed-point Induction*). *Let $\langle D_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$ be pointed cpos, and $R \in \mathbf{ARel}\langle D_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$ such that $\langle \bot_{D_\iota}\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R$. Then for any pair of functions $\langle f_\iota : D_\iota \to D_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$, such that $\forall\langle d_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R.\langle f_\iota\, d_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R$, we also have $\langle \mathit{fix}\, f_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R$.*

Proof. Follows immediately from admissibility of R, and the defining equation $\mathit{fix}\, f = \bigsqcup_{n\in\omega} f^n(\bot)$. □

Like for continuity, we have a number of general principles for constructing new admissible relations from others:

Lemma 31. *Admissible relations are closed under* (1) *arbitrary intersections,* (2) *finite unions,* (3) *inverse images by arbitrary (continuous, as usual) functions, and* (4) *direct images by order-monics:*

(1) *If for all $j \in J$, $R_j \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$, then $\{\langle a_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \forall j \in J.\langle a_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R_j\} \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$.*
(2) *If for all $j \in J$, $R_j \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$, and J is* finite*, then $\{\langle a_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \exists j \in J.\langle a_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R_j\} \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$.*
(3) *If $S \in \mathbf{ARel}\langle B_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$, and $\langle f_\iota : A_\iota \to B_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$ are functions, then $\{\langle a_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle f_\iota\, a_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in S\} \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$.*
(4) *If $R \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$, and $\langle f_\iota : A_\iota \to B_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$ are* order-monics *(i.e., for each $\iota \in \{\mathrm{I},\mathrm{II}\}$, and all $a, a' \in A_\iota$, $f_\iota(a) \sqsubseteq_{B_\iota} f_\iota(a') \Rightarrow a \sqsubseteq_{A_\iota} a'$), then $\{\langle f_\iota\, a_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle a_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in R\} \in \mathbf{ARel}\langle B_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$.*

Proof. All parts are straightforward and standard (e.g., [37, 10.2]). □

The following is a central concept in the construction:

Definition 32 (*Monadic Relational Action*). A *relational action* T_r for a pair of monads $\langle (T_\iota, \eta_\iota, \star_\iota)\rangle^{\iota\in\mathsf{I},\mathsf{II}}$, assigns to any $\langle A_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}$ and $R \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}$, a relation $T_\mathsf{r}(R) \in \mathbf{ARel}\langle T_\iota A_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}$, such that

(1) If $\langle a_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in R$, then $\langle \eta_\iota\, a_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in T_\mathsf{r}(R)$.
(2) If $\langle t_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in T_\mathsf{r}(R)$ and $\forall\langle a_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in R.\langle f_\iota\, a_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in T_\mathsf{r}(S)$, then $\langle t_\iota \star_\iota f_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in T_\mathsf{r}(S)$.

Example 33. When $\mathcal{T}_\mathsf{I}$ and $\mathcal{T}_\mathsf{II}$ are both the lifting monad $\mathcal{T}^\perp$ from Example 8, we can take

$$T_\mathsf{r}^\perp(R) = \{\langle\perp\rangle^{\iota\in\mathsf{I},\mathsf{II}}\} \cup \{\langle\lfloor a_\iota\rfloor\rangle^{\iota\in\mathsf{I},\mathsf{II}} \mid \langle a_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in R\}.$$

It is straightforward to check (e.g., using Lemma 31(2, 4)) that $T_\mathsf{r}^\perp$ is indeed a monadic relational action, i.e., that $T_\mathsf{r}^\perp(R)$ is admissible when R is, and that the mapping satisfies the conditions of Definition 32. □

When $R \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}$ and $S \in \mathbf{ARel}\langle B_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}$, we write $R \to^\mathsf{r} S$ for the set of relation-preserving function pairs:

$$R \to^\mathsf{r} S = \{\langle f_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \mid \forall\langle a_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in R.\langle f_\iota\, a_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in S\} \in \mathbf{ARel}\langle A_\iota \to B_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}.$$

Definition 34 (*Algebra Relation*). Let T_r be a relational action for a pair of monads $\langle\mathcal{T}_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}$. A T_r-*relation* between $\mathcal{T}_\mathsf{I}$-algebra $(D_\mathsf{I}, \gamma_\mathsf{I})$ and $\mathcal{T}_\mathsf{II}$-algebra $(D_\mathsf{II}, \gamma_\mathsf{II})$ is a relation $R \in \mathbf{ARel}\langle D_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}$ such that $\langle\gamma_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in T_\mathsf{r}(R) \to^\mathsf{r} R$.

In particular, a relation between pointed cpos $\langle D_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}$ is a $T_\mathsf{r}^\perp$-relation precisely when it is pointed, i.e., includes $\langle\perp_{D_\iota}\rangle^{\iota\in\mathsf{I},\mathsf{II}}$.

3.2. Relational semantics of types

Much like we previously interpreted types as cpos, we can now interpret them as admissible relations between those interpretations. We start with the relational counterpart of Definition 14:

Definition 35. A *relational type-and-effect interpretation* (with respect to two ordinary interpretations) of an $\mathrm{M}^3\mathrm{L}$ signature consists of the following:

- For every base type b, a relation $\mathcal{B}_\mathsf{r}^b \in \mathbf{ARel}\langle\mathcal{B}_\iota^b\rangle^{\iota\in\mathsf{I},\mathsf{II}}$.
- For every effect e, a relational action T_r^e for the monads $\mathcal{T}_\mathsf{I}^e$ and $\mathcal{T}_\mathsf{II}^e$.
- For every subeffecting $e \preceq e'$, a guarantee that $\langle i_\iota^{e,e'}\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in T_\mathsf{r}^e(R) \to^\mathsf{r} T_\mathsf{r}^{e'}(R)$.

The distinguished effect $\perp$ is always interpreted as the lifting action from Example 33.

Given such an interpretation, we can construct relational meanings of all types and e-types in a straightforward way. For technical reasons, to properly account for relational meanings of recursive types in Section 3.3 we map every type variable to a *pair* of relations, to be used for positive and negative occurrences of the variable, respectively. More precisely, let $\mathbf{ARel}^\pm\langle A_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} = \{\binom{R^+}{R^-} \mid R^+, R^- \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}}\}$. We then define a notion of relation environments: for $\theta_\mathsf{I}, \theta_\mathsf{II} \in [\![\Delta]\!]$, we write

$$\mathbf{ARel}^\pm\langle\theta_\iota\rangle^{\iota\in\mathsf{I},\mathsf{II}} = \{\theta_\mathsf{r} \mid \forall\alpha \in \Delta.\theta_\mathsf{r}(\alpha) \in \mathbf{ARel}^\pm\langle\theta_\iota(\alpha)\rangle^{\iota\in\mathsf{I},\mathsf{II}}\}.$$

The operation $\theta_\mathsf{r}^\dagger$ swaps the components of each pair in θ_r: if $\theta_\mathsf{r}(\alpha) = R^\pm$, then $\theta_\mathsf{r}^\dagger(\alpha) = R^\mp$. (When dealing with non-recursive types only, we can require that both relations in a pair are always equal, so that, in particular, $\theta_\mathsf{r}^\dagger = \theta_\mathsf{r}$.) The full relational semantics is shown in Fig. 9. As expected, we use monadic relational actions to interpret effects. All clauses should be straightforward, except the one for recursive types, which we will return to in the next section.

We need to check that the assignment is actually meaningful:

Lemma 36. *For each* τ, ${}^\mathsf{r}[\![\tau]\!]_{\theta_\mathsf{r}}$ *is indeed admissible.*

Proof. Follows easily by structural induction, using Lemma 31(1–4). □

We also get a relational variant of Lemma 15:

Lemma 37. *Let* $\vdash_\Delta \sigma$ e-type. *Then* ${}^\mathsf{r}[\![\sigma]\!]^\mathsf{c}_{\theta_\mathsf{r}}$ *is a* T_r^e*-relation between the algebra-meanings of* σ, *i.e.,* $\langle{}^\iota[\![\sigma]\!]^{\mathsf{S}e}_{\theta_\iota}\rangle^{\iota\in\mathsf{I},\mathsf{II}} \in T_\mathsf{r}^e({}^\mathsf{r}[\![\sigma]\!]^\mathsf{c}_{\theta_\mathsf{r}}) \to^\mathsf{r} {}^\mathsf{r}[\![\sigma]\!]^\mathsf{c}_{\theta_\mathsf{r}}$. *In particular, since any well-formed* σ *is a* $\perp$*-type, the relation* ${}^\mathsf{r}[\![\sigma]\!]^\mathsf{c}_{\theta_\mathsf{r}}$ *is always pointed.*

Proof. Straightforward induction on the structure of σ. □

Let $\theta_\iota, \theta_{\shortparallel} \in [\![\Delta]\!]$, and $\theta_{\mathsf{r}} \in \mathbf{ARel}^{\pm}\langle\theta_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$.
For $\vdash_\Delta \tau$ type, a relation $\boxed{{}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}} \in \mathbf{ARel}\langle{}^{\iota}[\![\tau]\!]_{\theta_\iota}\rangle^{\iota\in\{\iota,\shortparallel\}}}$:

$$
\begin{aligned}
{}^{\mathsf{r}}[\![\alpha]\!]_{\theta_{\mathsf{r}}} &= R^{+} \text{ where } \theta_{\mathsf{r}}(\alpha) = R^{\pm}\\
{}^{\mathsf{r}}[\![b]\!]_{\theta_{\mathsf{r}}} &= \mathcal{B}^{b}_{\mathsf{r}}\\
{}^{\mathsf{r}}[\![1]\!]_{\theta_{\mathsf{r}}} &= \{\langle *\rangle^{\iota\in\{\iota,\shortparallel\}}\}\\
{}^{\mathsf{r}}[\![\tau_1 \times \tau_2]\!]_{\theta_{\mathsf{r}}} &= \{\langle (a_{1\iota}, a_{2\iota})\rangle^{\iota\in\{\iota,\shortparallel\}} \mid \langle a_{1\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\tau_1]\!]_{\theta_{\mathsf{r}}} \wedge \langle a_{2\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\tau_2]\!]_{\theta_{\mathsf{r}}}\}\\
{}^{\mathsf{r}}[\![0]\!]_{\theta_{\mathsf{r}}} &= \emptyset\\
{}^{\mathsf{r}}[\![\tau_1 + \tau_2]\!]_{\theta_{\mathsf{r}}} &= \{\langle in_1(a_{1\iota})\rangle^{\iota\in\{\iota,\shortparallel\}} \mid \langle a_{1\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\tau_1]\!]_{\theta_{\mathsf{r}}}\} \cup \{\langle in_2(a_{2\iota})\rangle^{\iota\in\{\iota,\shortparallel\}} \mid \langle a_{2\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\tau_2]\!]_{\theta_{\mathsf{r}}}\}\\
{}^{\mathsf{r}}[\![\mu\alpha.\tau]\!]_{\theta_{\mathsf{r}}} &= \mu^{\mathsf{r}}\begin{pmatrix}{}^{\mathsf{r}}[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta_{\mathsf{r}}}\\ {}^{\mathsf{r}}[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta^{\dagger}_{\mathsf{r}}}\end{pmatrix} = \{\langle \phi_{{}^{\mathsf{r}}[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta_\iota}} a_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \mid \langle a_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}[\alpha\mapsto({}^{\mathsf{r}}[\![\mu\alpha.\tau]\!]_{\theta_{\mathsf{r}}}, {}^{\mathsf{r}}[\![\mu\alpha.\tau]\!]_{\theta^{\dagger}_{\mathsf{r}}})]}\} \quad \text{[See Section 3.3]}\\
{}^{\mathsf{r}}[\![\sigma]\!]_{\theta_{\mathsf{r}}} &= {}^{\mathsf{r}}[\![\sigma]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}}
\end{aligned}
$$

For $\vdash_\Delta \sigma$ e-type, a relation $\boxed{{}^{\mathsf{r}}[\![\sigma]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}} \in \mathbf{ARel}\langle{}^{\iota}[\![\sigma]\!]^{\mathrm{c}}_{\theta_\iota}\rangle^{\iota\in\{\iota,\shortparallel\}}}$:

$$
\begin{aligned}
{}^{\mathsf{r}}[\![1]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}} &= \{\langle *\rangle^{\iota\in\{\iota,\shortparallel\}}\}\\
{}^{\mathsf{r}}[\![\sigma_1 \times \sigma_2]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}} &= \{\langle (d_{1\iota}, d_{2\iota})\rangle^{\iota\in\{\iota,\shortparallel\}} \mid \langle d_{1\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\sigma_1]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}} \wedge \langle d_{2\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\sigma_2]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}}\}\\
{}^{\mathsf{r}}[\![\tau \to \sigma]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}} &= \{\langle f_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \mid \forall\langle a_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\tau]\!]_{\theta^{\dagger}_{\mathsf{r}}}.\langle f_\iota\, a_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in {}^{\mathsf{r}}[\![\sigma]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}}\}\\
{}^{\mathsf{r}}[\![\langle e'\rangle\tau]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}} &= T^{e'}_{\mathsf{r}}({}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}})
\end{aligned}
$$

Fig. 9. Relational semantics of M^{3}L types.

3.3. Relations on recursive types

Again, this section can be skipped; we will formally establish that the circular specification of the relation ${}^{\mathsf{r}}[\![\mu\alpha.\tau]\!]_{\theta_{\mathsf{r}}}$ in Fig. 9 does in fact define a unique relation between the cpos interpreting $\mu\alpha.\tau$ in the two semantics.

3.3.1. Solving recursive relation equations

Let us first extend our partial-function notation to relations. Let relations $R \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$ and $S \in \mathbf{ARel}\langle B_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$ be given. For functions $\langle f_\iota : A_\iota \rightharpoonup B_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$, we write $\langle f_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R \rightharpoonup^{\mathsf{r}} S$ when $\langle f_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R \to^{\mathsf{r}} T^{\perp}_{\mathsf{r}}(S)$. Clearly, $\langle \overline{id}_{A_\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in R \rightharpoonup^{\mathsf{r}} R$; and if $\langle f_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R \rightharpoonup^{\mathsf{r}} S$ and $\langle g_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in S \rightharpoonup^{\mathsf{r}} T$ then $\langle g_\iota \mathbin{\overline{\circ}} f_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R \rightharpoonup^{\mathsf{r}} T$. We also have $\langle \overline{f_\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in R \rightharpoonup^{\mathsf{r}} S$ precisely when $\langle f_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R \to^{\mathsf{r}} S$. In particular, for $R, R' \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$,

$$R \subseteq R' \Leftrightarrow \langle \overline{id}_{A_\iota}\rangle^{\iota\in\{\iota,\shortparallel\}} \in R \rightharpoonup^{\mathsf{r}} R'. \tag{14}$$

We also write $\langle f^{\pm}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} S^{\pm}$ for $\langle f^{+}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R^{+} \rightharpoonup^{\mathsf{r}} S^{+} \wedge \langle f^{-}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in S^{-} \rightharpoonup^{\mathsf{r}} R^{-}$. Then $\langle \overline{\boldsymbol{id}}\rangle^{\iota\in\{\iota,\shortparallel\}} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} R^{\pm}$; and if $\langle f^{\pm}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} S^{\pm}$ and $\langle g^{\pm}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in S^{\pm} \rightleftharpoons^{\mathsf{r}} T^{\pm}$, then $\langle g^{\pm}_\iota \mathbin{\overline{\bullet}} f^{\pm}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} T^{\pm}$. Also, if $\langle f^{\pm}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} S^{\pm}$, then $\langle f^{\mp}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in S^{\mp} \rightleftharpoons^{\mathsf{r}} R^{\mp}$.

Definition 38 (*m-Functorial Relational Action*). A *relational action* F_{r} for a pair of m-functors $\langle F_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$, assigns to every pair of relations $R^{\pm} \in \mathbf{ARel}^{\pm}\langle A_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$, a relation $F_{\mathsf{r}}(R^{\pm}) \in \mathbf{ARel}\langle F_\iota A_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$, such that for every $R^{\pm} \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$ and $S^{\pm} \in \mathbf{ARel}^{\pm}\langle B_\iota\rangle^{\iota\in\{\iota,\shortparallel\}}$, it holds that

$$\langle f^{\pm}_\iota\rangle^{\iota\in\{\iota,\shortparallel\}} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} S^{\pm} \Rightarrow \langle F_\iota(f^{\pm}_\iota)\rangle^{\iota\in\{\iota,\shortparallel\}} \in F_{\mathsf{r}}(R^{\pm}) \rightharpoonup^{\mathsf{r}} F_{\mathsf{r}}(S^{\pm}). \tag{15}$$

We formulate the following, central theorem in a slightly more general way than Pitts [26, Theorem 4.16]; this generalization will allow us to establish existence of invariant relations at nested recursive types as well, without having to explicitly consider parameterized relational actions.

Theorem 39 (*Existence of Invariant Relations*). *Let F_{r}^{+} and F_{r}^{-} both be relational actions for m-functors $\langle F_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$. Then there exists a* unique *pair of admissible relations R^+, $R^-\in\mathbf{ARel}\langle\mu F_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$, such that*

$$\begin{aligned} R^+ &= \{\langle\phi_{F_\iota} m_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle m_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in F_{\mathsf{r}}^{+}(R^{\pm})\} \\ R^- &= \{\langle\phi_{F_\iota} m_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle m_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in F_{\mathsf{r}}^{-}(R^{\mp})\}. \end{aligned} \tag{16}$$

We write $\mu^{\mathsf{r}}\binom{F_{\mathsf{r}}^{+}}{F_{\mathsf{r}}^{-}}$ for R^+; by uniqueness, $\mu^{\mathsf{r}}\binom{F_{\mathsf{r}}^{-}}{F_{\mathsf{r}}^{+}}$ must denote R^-. In particular, when $F_{\mathsf{r}}^{+}=F_{\mathsf{r}}^{-}$, we must also have $R^+=R^-$.

Proof. Consider the set of relations $\mathbf{ARel}\langle\mu F_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$ to be ordered by set-theoretic inclusion, $\subseteq$. Since admissible relations are closed under arbitrary intersections (Lemma 31(1)), this ordered set is in fact a complete lattice (with $\bigwedge\{R_j \mid j\in J\}=\bigcap\{R_j \mid j\in J\}$, while $\bigvee\{R_j \mid j\in J\}=\bigcap\{R\in\mathbf{ARel}\langle\mu F_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \forall j\in J.R_j\subseteq R\}$ is the least *admissible* relation containing all of the R_j). By symmetry, $\mathbf{ARel}\langle\mu F_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$ ordered by reverse inclusion, $\supseteq$, is also a complete lattice, as is the product of these two lattices, which we will use as the ordering on $\mathbf{ARel}^{\pm}\langle\mu F_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$.

An m-functorial relational action is easily seen to be monotonic in its positive argument and antimonotonic in the negative. For suppose $R^+\subseteq S^+$ and $S^-\subseteq R^-$. Then by (14), $\langle\overline{\boldsymbol{id}}_{\mu F_\iota}\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in R^{\pm}\rightleftharpoons^{\mathsf{r}} S^{\pm}$, and thus $\langle F_\iota(\overline{\boldsymbol{id}}_{\mu F_\iota})\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in F_{\mathsf{r}}(R^{\pm})\rightharpoonup^{\mathsf{r}} F_{\mathsf{r}}(S^{\pm})$. But $F_\iota(\overline{\boldsymbol{id}}_{\mu F_\iota})=\overline{id}_{F_\iota(\mu F_\iota)}$, so we have $\langle\overline{id}_{F_\iota(\mu F_\iota)}\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in F_{\mathsf{r}}(R^{\pm})\rightharpoonup^{\mathsf{r}} F_{\mathsf{r}}(S^{\pm})$, which says precisely that $F_{\mathsf{r}}(R^{\pm})\subseteq F_{\mathsf{r}}(S^{\pm})$.

Since isomorphisms are necessarily order-monics, the operator

$$R^{\pm}\mapsto\begin{pmatrix}\{\langle\phi_{F_\iota} m_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle m_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in F_{\mathsf{r}}^{+}(R^{\pm})\}\\ \{\langle\phi_{F_\iota} m_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle m_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in F_{\mathsf{r}}^{-}(R^{\mp})\}\end{pmatrix}.$$

on $\mathbf{ARel}^{\pm}\langle\mu F_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}}$ is also well defined (by Lemma 31(4)) and monotonic, and hence has a fixed point by the Knaster–Tarski theorem. This gives us existence of at least one solution for (16). (Unlike Pitts's proof, ours does not exploit the fact that the Knaster–Tarski construction actually finds the *least* (pre)fixed point of the operator.)

For uniqueness, suppose both $R_1^{\pm}$ and $R_2^{\pm}$ satisfy (16). We aim to show that $R_1^+\subseteq R_2^+$ and $R_2^-\subseteq R_1^-$; the converse inclusions (i.e., with R_1 and R_2 swapped) follow symmetrically. Consider the relation

$$R_1^{\pm}\rightleftharpoons^{\mathsf{r}} R_2^{\pm}\in\mathbf{ARel}\left\langle\begin{matrix}(\mu F_\iota\to(\mu F_\iota)_\perp)\times\\ (\mu F_\iota\to(\mu F_\iota)_\perp)\end{matrix}\right\rangle^{\iota\in\mathrm{I},\mathrm{II}}.$$

We evidently have $\langle\binom{\lambda m^{\mu F_\iota}.\perp}{\lambda m^{\mu F_\iota}.\perp}\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in R_1^{\pm}\rightleftharpoons^{\mathsf{r}} R_2^{\pm}$. Further, assume $\langle f_\iota^{\pm}\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in R_1^{\pm}\rightleftharpoons^{\mathsf{r}} R_2^{\pm}$. Then by (15) for the action F_{r}^{+}, $\langle F_\iota(f_\iota^{\pm})\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in F_{\mathsf{r}}^{+}(R_1^{\pm})\rightharpoonup^{\mathsf{r}} F_{\mathsf{r}}^{+}(R_2^{\pm})$. Also, by the assumption on $R_2^{\pm}$, $\langle\phi_{F_\iota}\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in F_{\mathsf{r}}^{+}(R_2^{\pm})\to^{\mathsf{r}} R_2^{+}$; and from the assumption on $R_1^{\pm}$ and $\phi_{F_\iota}^{-1}\circ\phi_{F_\iota}=id$, $\langle\phi_{F_\iota}^{-1}\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in R_1^{+}\to^{\mathsf{r}} F_{\mathsf{r}}^{+}(R_1^{\pm})$, so

$$\langle\overline{\phi_{F_\iota}}\mathbin{\bar\circ} F_\iota(f_\iota^{\pm})\mathbin{\bar\circ}\overline{\phi_{F_\iota}^{-1}}\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in R_1^{+}\rightharpoonup^{\mathsf{r}} R_2^{+}.$$

Analogously, $\langle F_\iota^{-}(f_\iota^{\mp})\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in F_{\mathsf{r}}^{-}(R_2^{\mp})\rightharpoonup^{\mathsf{r}} F_{\mathsf{r}}^{-}(R_1^{\mp})$, so we get

$$\left\langle\begin{pmatrix}\overline{\phi_{F_\iota}}\mathbin{\bar\circ} F_\iota(f_\iota^{\pm})\mathbin{\bar\circ}\overline{\phi_{F_\iota}^{-1}}\\ \overline{\phi_{F_\iota}}\mathbin{\bar\circ} F_\iota(f_\iota^{\mp})\mathbin{\bar\circ}\overline{\phi_{F_\iota}^{-1}}\end{pmatrix}\right\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in R_1^{\pm}\rightleftharpoons^{\mathsf{r}} R_2^{\pm}.$$

Hence, by fixed-point induction, also

$$\left\langle fix\left(\lambda f^{\pm}.\begin{pmatrix}\overline{\phi_{F_\iota}}\mathbin{\bar\circ} F_\iota(f^{\pm})\mathbin{\bar\circ}\overline{\phi_{F_\iota}^{-1}}\\ \overline{\phi_{F_\iota}}\mathbin{\bar\circ} F_\iota(f^{\mp})\mathbin{\bar\circ}\overline{\phi_{F_\iota}^{-1}}\end{pmatrix}\right)\right\rangle^{\iota\in\mathrm{I},\mathrm{II}}\in R_1^{\pm}\rightleftharpoons^{\mathsf{r}} R_2^{\pm}.$$

Like in the proof of the μ-case in Lemma 22(1), we then exploit the minimal-invariant property to conclude that, for $\iota\in\{\mathrm{I},\mathrm{II}\}$, and $\pi\in\{\pi_+,\pi_-\}$,

$$\pi\left(fix\left(\lambda f^{\pm}.\begin{pmatrix}\overline{\phi_{F_\iota}}\mathbin{\bar\circ} F_\iota(f^{\pm})\mathbin{\bar\circ}\overline{\phi_{F_\iota}^{-1}}\\ \overline{\phi_{F_\iota}}\mathbin{\bar\circ} F_\iota(f^{\mp})\mathbin{\bar\circ}\overline{\phi_{F_\iota}^{-1}}\end{pmatrix}\right)\right)=fix\left(\lambda f.\overline{\phi_{F_\iota}}\mathbin{\bar\circ} F_\iota\binom{f}{f}\mathbin{\bar\circ}\overline{\phi_{F_\iota}^{-1}}\right)=\overline{id}_{\mu F_\iota}.$$

Thus, $\langle \overline{\boldsymbol{id}}_{\mu F_\iota} \rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in R_1^{\pm} \rightleftharpoons^{\mathsf{r}} R_2^{\pm}$, which means precisely that $R_1^+ \subseteq R_2^+$ and $R_2^- \subseteq R_1^-$. □

Example 40. Consider again the type $\mu\alpha.1+\alpha$ from Example 20. Let $\phi_{\mathrm{I}} : \mathbf{1} + A_{\mathrm{I}} \to A_{\mathrm{I}}$ and $\phi_{\mathrm{II}} : \mathbf{1} + A_{\mathrm{II}} \to A_{\mathrm{II}}$ be isomorphisms (but not necessarily minimal invariants), and suppose ${}^{\mathrm{I}}[\![\mu\alpha.1+\alpha]\!]_{[]} = A_{\mathrm{I}}$ and ${}^{\mathrm{II}}[\![\mu\alpha.1+\alpha]\!]_{[]} = A_{\mathrm{II}}$. A candidate solution R for ${}^{\mathsf{r}}[\![\mu\alpha.1+\alpha]\!]_{[]} \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}}$ must then satisfy that

$$R = \{\langle \phi_\iota(a_\iota)\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \mid \langle a_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in {}^{\mathsf{r}}[\![1+\alpha]\!]_{[\alpha\mapsto\binom{R}{R}]}\} = \{\langle \phi_\iota(in_1(*))\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}}\} \cup \{\langle \phi_\iota(in_2(a_\iota))\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \mid \langle a_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in R\}.$$

If we take both the primary and the secondary interpretation to be the non-standard solution $\mathbf{N}'$, the characterization of R leaves it unspecified whether $\langle\infty\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in R$: both possibilities are consistent. (Indeed, the Knaster–Tarski construction (for least prefixed points) in the proof of Theorem 39 would give us that $\langle\infty\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \notin R^+$ but $\langle\infty\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in R^-$.) But when both interpretations are minimal invariants, they contain no other elements beyond (isomorphic copies of) the standard naturals, and R is uniquely specified by the equation above. □

3.3.2. *Functoriality of standard relational actions*

To use Theorem 39, we must show that the m-functorial action we have previously associated with each type constructor $\alpha.\tau$ matches its natural relational interpretation.

Let us write $\langle\varphi_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in \theta_{\mathsf{r}} \rightleftharpoons^{\mathsf{r}} \theta'_{\mathsf{r}}$ for $\forall\alpha\in\Delta.\ \langle\varphi_\iota(\alpha)\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in \theta_{\mathsf{r}}(\alpha) \rightleftharpoons^{\mathsf{r}} \theta'_{\mathsf{r}}(\alpha)$. We also formally define the candidate relational action of type constructors as follows:

$${}^{\mathsf{r}}[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta_{\mathsf{r}}}(R^{\pm}) = {}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}[\alpha\mapsto R^{\pm}]}$$

(The fact that R^+ and R^- are not necessarily equal is what requires us to generally define ${}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}}$ for θ_{r} not necessarily satisfying $\theta_{\mathsf{r}} = \theta_{\mathsf{r}}^{\dagger}$.)

Well definedness of the relational semantics (for μ-types) then has to be argued together with the following lemma, corresponding to the type-directed definitions of m-functorial action of types in Fig. 6.

Lemma 41 (*m-Functoriality of Relational Semantics*). *For each $\iota\in\{\mathrm{I},\mathrm{II}\}$, let $\theta_\iota, \theta'_\iota \in [\![\Delta]\!]$ and $\varphi_\iota : \theta_\iota \rightleftharpoons \theta'_\iota$. Also, assume $\theta_{\mathsf{r}} \in \mathbf{ARel}^{\pm}\langle\theta_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}}$, and $\theta'_{\mathsf{r}} \in \mathbf{ARel}^{\pm}\langle\theta'_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}}$, such that $\langle\varphi_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in \theta_{\mathsf{r}} \rightleftharpoons^{\mathsf{r}} \theta'_{\mathsf{r}}$. Then,*

(1) *If $\vdash_\Delta \tau$ type, then $\langle {}^{\iota}[\![\tau]\!]^{\mathrm{f}}(\varphi_\iota)\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in {}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}} \rightharpoonup^{\mathsf{r}} {}^{\mathsf{r}}[\![\tau]\!]_{\theta'_{\mathsf{r}}}$.*
(2) *If $\vdash_\Delta \sigma$ e-type, then $\langle {}^{\iota}[\![\sigma]\!]^{\mathrm{cf}}(\varphi_\iota)\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in {}^{\mathsf{r}}[\![\sigma]\!]^{\mathrm{c}}_{\theta_{\mathsf{r}}} \to^{\mathsf{r}} {}^{\mathsf{r}}[\![\sigma]\!]^{\mathrm{c}}_{\theta'_{\mathsf{r}}}$.*

In particular, ${}^{\mathsf{r}}[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta_{\mathsf{r}}}$ is indeed a relational action for the m-functors ${}^{\mathrm{I}}[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta_{\mathrm{I}}}$ and ${}^{\mathrm{II}}[\![\alpha.\tau]\!]^{\mathrm{m}}_{\theta_{\mathrm{II}}}$.

Proof. By mutual structural induction on τ and σ. Again, for (1), the only challenging case is:

- Case $\tau = \mu\alpha.\tau_0$. For each $\iota\in\{\mathrm{I},\mathrm{II}\}$, let $\phi_\iota^{\pm} = \binom{\phi_\iota}{\phi_\iota^{-1}}_{{}^{\iota}[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta_\iota}}$. By the IH on τ_0, we know that $F_{\mathsf{r}}^+ = {}^{\mathsf{r}}[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta_{\mathsf{r}}}$ and $F_{\mathsf{r}}^- = {}^{\mathsf{r}}[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta_{\mathsf{r}}^{\dagger}}$ are proper relational actions, so $R^+ = \mu^{\mathsf{r}}(F_{\mathsf{r}}^{\pm})$ and $R^- = \mu^{\mathsf{r}}(F_{\mathsf{r}}^{\mp})$ satisfy:

$$R^{\pm} = \begin{pmatrix}\{\langle \phi_\iota^+ m_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \mid \langle m_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in F_{\mathsf{r}}^+(R^{\pm})\} \\ \{\langle \phi_\iota^+ m_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \mid \langle m_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in F_{\mathsf{r}}^-(R^{\mp})\}\end{pmatrix}$$

or, equivalently,

$$R^{\pm} = \begin{pmatrix}\{\langle m_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \mid \langle \phi_\iota^- m_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in F_{\mathsf{r}}^+(R^{\pm})\} \\ \{\langle m_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \mid \langle \phi_\iota^- m_\iota\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in F_{\mathsf{r}}^-(R^{\mp})\}\end{pmatrix}.$$

In other words, $\langle \overline{\phi}_\iota^{\pm}\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in \binom{F_{\mathsf{r}}^+(R^{\pm})}{F_{\mathsf{r}}^-(R^{\mp})} \rightleftharpoons^{\mathsf{r}} R^{\pm}$ and $\langle \overline{\phi}_\iota^{\mp}\rangle^{\iota\in\{\mathrm{I},\mathrm{II}\}} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} \binom{F_{\mathsf{r}}^+(R^{\pm})}{F_{\mathsf{r}}^-(R^{\mp})}$. Analogous results hold for $\phi'^{\pm}_{\mathrm{I}}$, $\phi'^{\pm}_{\mathrm{II}}$, $F'^{\pm}_{\mathsf{r}}$, and $R'^{\pm}$ in terms of θ'_{r}.

Now, for $\iota\in\{\mathrm{I},\mathrm{II}\}$, let

$$H_\iota(h^{\pm}) = \overline{\phi'^{\pm}_\iota} \bullet \begin{pmatrix}{}^{\iota}[\![\tau_0]\!]^{\mathrm{f}}(\varphi_\iota[\alpha\mapsto h^{\pm}]) \\ {}^{\iota}[\![\tau_0]\!]^{\mathrm{f}}(\varphi_\iota^{\dagger}[\alpha\mapsto h^{\mp}])\end{pmatrix} \bullet \overline{\phi'^{\mp}_\iota}.$$

The required result is then precisely the $+$-half of the statement

$$\langle \mathit{fix}\, H_\iota \rangle^{\iota\in\mathsf{I},\shortparallel} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} R'^{\pm},$$

which we show by fixed-point induction. Evidently

$$R^{\pm} \rightleftharpoons^{\mathsf{r}} R'^{\pm} \in \mathbf{ARel}\left\langle \begin{matrix} (\mu F_\iota^{+} \to (\mu F_\iota'^{+})_\perp) \times \\ (\mu F_\iota'^{-} \to (\mu F_\iota^{-})_\perp) \end{matrix} \right\rangle^{\iota\in\mathsf{I},\shortparallel}$$

and $\langle \binom{\lambda m.\perp}{\lambda m'.\perp} \rangle^{\iota\in\mathsf{I},\shortparallel} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} R'^{\pm}$. Assume therefore that $\langle h_\iota^{\pm}\rangle^{\iota\in\mathsf{I},\shortparallel} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} R'^{\pm}$. Together with the assumption that $\langle \varphi_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in \theta_{\mathsf{r}} \rightleftharpoons^{\mathsf{r}} \theta'_{\mathsf{r}}$, this gives

$$\langle \varphi_\iota[\alpha \mapsto h_\iota^{\pm}]\rangle^{\iota\in\mathsf{I},\shortparallel} \in \theta_{\mathsf{r}}[\alpha \mapsto R^{\pm}] \rightleftharpoons^{\mathsf{r}} \theta'_{\mathsf{r}}[\alpha \mapsto R'^{\pm}].$$

By IH(1) on τ_0 we therefore get

$$\langle {}^{\iota}[\![\tau_0]\!]^{\mathrm{f}}(\varphi_\iota[\alpha \mapsto h_\iota^{\pm}])\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\tau_0]\!]_{\theta_{\mathsf{r}}[\alpha\mapsto R^{\pm}]} \rightharpoonup^{\mathsf{r}} {}^{\mathsf{r}}[\![\tau_0]\!]_{\theta'_{\mathsf{r}}[\alpha\mapsto R'^{\pm}]} = F_{\mathsf{r}}^{+}(R^{\pm}) \rightharpoonup^{\mathsf{r}} F_{\mathsf{r}}'^{+}(R'^{\pm})$$

and, symmetrically,

$$\langle {}^{\iota}[\![\tau_0]\!]^{\mathrm{f}}(\varphi_\iota^{\dagger}[\alpha \mapsto h_\iota^{\mp}])\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\tau_0]\!]_{\theta_{\mathsf{r}}'^{\dagger}[\alpha\mapsto R'^{\mp}]} \rightharpoonup^{\mathsf{r}} {}^{\mathsf{r}}[\![\tau_0]\!]_{\theta_{\mathsf{r}}^{\dagger}[\alpha\mapsto R^{\mp}]} = F_{\mathsf{r}}'^{-}(R'^{\mp}) \rightharpoonup^{\mathsf{r}} F_{\mathsf{r}}^{-}(R^{\mp}).$$

But, together with the properties of the isomorphisms, this says precisely that

$$\langle H_\iota(h_\iota^{\pm})\rangle^{\iota\in\mathsf{I},\shortparallel} \in R^{\pm} \rightleftharpoons^{\mathsf{r}} R'^{\pm},$$

as required for the fixed-point induction step.

The cases for part (2) are all straightforward. A representative one shows (together with the trivial case for $\tau = \sigma$) that monadic relational actions do indeed induce m-functorial ones:

- Case $\sigma = \langle e'\rangle\tau$. Suppose $\langle t_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e'}({}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}})$; we must show that

$$\langle t_\iota \star_\iota^{e'} \lambda a.\, i_\iota^{\perp,e'}({}^{\iota}[\![\tau]\!]^{\mathrm{f}}(\varphi_\iota)\, a)\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e'}({}^{\mathsf{r}}[\![\tau]\!]_{\theta'_{\mathsf{r}}}).$$

By the assumption on $\langle t_\iota\rangle^{\iota\in\mathsf{I},\shortparallel}$ and Definition 32(2), it suffices to show that, for all $\langle a_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}}$,

$$\langle i_\iota^{\perp,e'}({}^{\iota}[\![\tau]\!]^{\mathrm{f}}(\varphi_\iota)\, a_\iota)\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e'}({}^{\mathsf{r}}[\![\tau]\!]_{\theta'_{\mathsf{r}}}).$$

Now, by the requirement on $i^{\perp,e'}$ from Definition 35, we need only show that

$$\langle {}^{\iota}[\![\tau]\!]^{\mathrm{f}}(\varphi_\iota)\, a_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{\perp}({}^{\mathsf{r}}[\![\tau]\!]_{\theta'_{\mathsf{r}}}),$$

which follows from the assumption on $\langle a_\iota\rangle^{\iota\in\mathsf{I},\shortparallel}$ and IH(1) on τ.

Finally, the "in particular" follows straightforwardly from part (1). □

The relational semantics also behaves predictably with respect to substitution:

Lemma 42 (*Substitution, Relations*). *Let θ_{r} be a relational environment for Δ, and let $\vdash_{\Delta,\alpha'} \tau$ type and $\vdash_{\Delta} \tau'$ type. Then*

$${}^{\mathsf{r}}[\![\tau[\tau'/\alpha']]\!]_{\theta_{\mathsf{r}}} = {}^{\mathsf{r}}[\![\tau]\!]_{\theta_{\mathsf{r}}[\alpha'\mapsto R'^{\pm}]} \quad \textit{where } R'^{\pm} = \begin{pmatrix} {}^{\mathsf{r}}[\![\tau']\!]_{\theta_{\mathsf{r}}} \\ {}^{\mathsf{r}}[\![\tau']\!]_{\theta_{\mathsf{r}}^{\dagger}} \end{pmatrix}.$$

Proof. We first note that ${}^{\mathsf{r}}[\![\tau']\!]_{\theta_{\mathsf{r}}} = {}^{\mathsf{r}}[\![\tau']\!]_{\theta'_{\mathsf{r}}}$ when $\theta_{\mathsf{r}}(\alpha) = \theta'_{\mathsf{r}}(\alpha)$ for all $\alpha \in FTV(\tau')$. The result then follows by induction on the structure of τ. Again, the only interesting case is for $\tau = \mu\alpha.\tau_0$. Suppose $\alpha \neq \alpha'$ and $\alpha \notin FTV(\tau')$. Then

$$\begin{aligned} {}^{\mathsf{r}}[\![(\mu\alpha.\tau_0)[\tau'/\alpha']]\!]_{\theta_{\mathsf{r}}} &= {}^{\mathsf{r}}[\![\mu\alpha.\tau_0[\tau'/\alpha']]\!]_{\theta_{\mathsf{r}}} = \mu^{\mathsf{r}}\begin{pmatrix} R^{\pm} \mapsto {}^{\mathsf{r}}[\![\tau_0[\tau'/\alpha']]\!]_{\theta_{\mathsf{r}}[\alpha\mapsto R^{\pm}]} \\ R^{\pm} \mapsto {}^{\mathsf{r}}[\![\tau_0[\tau'/\alpha']]\!]_{\theta_{\mathsf{r}}^{\dagger}[\alpha\mapsto R^{\pm}]} \end{pmatrix} \\ &\overset{\text{ih}}{=} \mu^{\mathsf{r}}\begin{pmatrix} R^{\pm} \mapsto {}^{\mathsf{r}}[\![\tau_0]\!]_{\theta_{\mathsf{r}}[\alpha\mapsto R^{\pm}][\alpha'\mapsto R'^{\pm}]} \\ R^{\pm} \mapsto {}^{\mathsf{r}}[\![\tau_0]\!]_{\theta_{\mathsf{r}}^{\dagger}[\alpha\mapsto R^{\pm}][\alpha'\mapsto R'^{\mp}]} \end{pmatrix} = \mu^{\mathsf{r}}\begin{pmatrix} R^{\pm} \mapsto {}^{\mathsf{r}}[\![\tau_0]\!]_{\theta_{\mathsf{r}}[\alpha'\mapsto R'^{\pm}][\alpha\mapsto R^{\pm}]} \\ R^{\pm} \mapsto {}^{\mathsf{r}}[\![\tau_0]\!]_{(\theta_{\mathsf{r}}[\alpha'\mapsto R'^{\pm}])^{\dagger}[\alpha\mapsto R^{\pm}]} \end{pmatrix} = {}^{\mathsf{r}}[\![\mu\alpha.\tau_0]\!]_{\theta_{\mathsf{r}}[\alpha'\mapsto R'^{\pm}]}. \end{aligned}$$

□

3.4. Main theorem

We can again complete our definition of relational interpretations, analogously to the case for Definition 24.

Definition 43. A *relational interpretation* of Σ (with respect to two ordinary interpretations) consists of relational interpretations of the base types and effects from Definition 35, such that, for every constant $(c:\tau) \in \Sigma$, its interpretations are related by $\langle \mathcal{C}_\iota^c \rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\tau]\!]_{[]}$.

Again, we extend the relational semantics of individual types to entire type environments: Let $\theta_\mathsf{I}, \theta_\shortparallel \in [\![\Delta]\!]$, and $\theta_\mathsf{r} \in \mathbf{ARel}^{\pm}\langle\theta_\iota\rangle^{\iota\in\mathsf{I},\shortparallel}$. For $\vdash_\Delta \Gamma$ env, we then take

$${}^{\mathsf{r}}[\![\Gamma]\!]_{\theta_\mathsf{r}} = \{\langle\rho_\iota \in {}^{\iota}[\![\Gamma]\!]_{\theta_\iota}\rangle^{\iota\in\mathsf{I},\shortparallel} \mid \forall(x:\tau)\in\Gamma.\langle\rho_\iota(x)\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\tau]\!]_{\theta_\mathsf{r}}\}.$$

Now, instead of a compositional *semantics* of terms like in Fig. 7, we get a *theorem*, provable by straightforward structural induction:

Theorem 44 (*Fundamental Lemma*). *Let $\theta_\mathsf{I}, \theta_\shortparallel \in [\![\Delta]\!]$ be given, and suppose $\theta_\mathsf{r} \in \mathbf{ARel}^{\pm}\langle\theta_\iota\rangle^{\iota\in\mathsf{I},\shortparallel}$ satisfies $\theta_\mathsf{r} = \theta_\mathsf{r}^\dagger$. Then for every M with $\Gamma \vdash_\Delta M : \tau$, if $\langle\rho_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\Gamma]\!]_{\theta_\mathsf{r}}$, then $\langle {}^{\iota}[\![M]\!]_{\theta_\iota}\,\rho_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\tau]\!]_{\theta_\mathsf{r}}$.*

Proof. By induction on the structure of M. All cases are quite simple; we show a few representative ones:

- Case $M = \mathbf{glet}^e_\sigma\ x \Leftarrow M_1.\, M_2$. To show:

  $$\langle {}^{\iota}[\![\sigma]\!]^{\mathrm{se}}_{\theta_\iota}\,({}^{\iota}[\![M_1]\!]_{\theta_\iota}\,\rho_\iota \star^e_\iota \lambda a.\,\eta^e_\iota\,({}^{\iota}[\![M_2]\!]_{\theta_\iota}\,\rho_\iota[x\mapsto a]))\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\sigma]\!]_{\theta_\mathsf{r}}.$$

  By Lemma 37, it suffices to show

  $$\langle {}^{\iota}[\![M_1]\!]_{\theta_\iota}\,\rho_\iota \star^e_\iota \lambda a.\,\eta^e_\iota\,({}^{\iota}[\![M_2]\!]_{\theta_\iota}\,\rho_\iota[x\mapsto a]))\rangle^{\iota\in\mathsf{I},\shortparallel} \in T^e_{\mathsf{r}}({}^{\mathsf{r}}[\![\sigma]\!]_{\theta_\mathsf{r}})\,,$$

  which follows straightforwardly from IH on M_1 and M_2, and Definition 32(1,2).
- Case $M = \mathbf{in}_{\mu\alpha.\tau_0}(M_0)$. To show:

  $$\langle \phi_{{}^{\iota}[\![\mu\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta_\iota}}\,({}^{\iota}[\![M_0]\!]_{\theta_\iota}\,\rho_\iota)\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\mu\alpha.\tau_0]\!]_{\theta_\mathsf{r}} = \mu^{\mathsf{r}}\begin{pmatrix}{}^{\mathsf{r}}[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta_\mathsf{r}}\\ {}^{\mathsf{r}}[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta_\mathsf{r}^\dagger}\end{pmatrix}.$$

  By the defining property of μ^{r} (Theorem 39), and the substitution property for the relational semantics (Lemma 42), this amounts to showing

  $$\langle {}^{\iota}[\![M_0]\!]_{\theta_\iota}\,\rho_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\alpha.\tau_0]\!]^{\mathrm{m}}_{\theta_\mathsf{r}}\begin{pmatrix}{}^{\mathsf{r}}[\![\mu\alpha.\tau_0]\!]_{\theta_\mathsf{r}}\\ {}^{\mathsf{r}}[\![\mu\alpha.\tau_0]\!]_{\theta_\mathsf{r}^\dagger}\end{pmatrix} = {}^{\mathsf{r}}[\![\tau_0]\!]_{\theta_\mathsf{r}[\alpha\mapsto({}^{\mathsf{r}}[\![\mu\alpha.\tau_0]\!]_{\theta_\mathsf{r}},\,{}^{\mathsf{r}}[\![\mu\alpha.\tau_0]\!]_{\theta_\mathsf{r}^\dagger})]} = {}^{\mathsf{r}}[\![\tau_0[(\mu\alpha.\tau_0)/\alpha]]\!]_{\theta_\mathsf{r}}\,.$$

  But that is precisely what the IH on M_0 gives us.
- Case $M = \mu x^\sigma.M_0$. To show:

  $$\langle \mathit{fix}\,(\lambda d.\,{}^{\iota}[\![M_0]\!]_{\theta_\iota}\rho_\iota[x\mapsto d])\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\sigma]\!]_{\theta_\mathsf{r}}.$$

  Since $\langle\bot\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\sigma]\!]_{\theta_\mathsf{r}}$ by Lemma 37, to use fixed-point induction (Lemma 30), we just need to establish that for all $\langle d_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\sigma]\!]_{\theta_\mathsf{r}}$,

  $$\langle {}^{\iota}[\![M_0]\!]_{\theta_\iota}\rho_\iota[x\mapsto d_\iota]\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\sigma]\!]_{\theta_\mathsf{r}}.$$

  And this follows immediately from the IH on M_0, since $\langle\rho_\iota[x\mapsto d_\iota]\rangle^{\iota\in\mathsf{I},\shortparallel} \in {}^{\mathsf{r}}[\![\Gamma, x:\sigma]\!]_{\theta_\mathsf{r}}$.

The remaining cases are analogous or standard. (The cases for λ-abstractions and applications rely on $\theta_\mathsf{r}^\dagger = \theta_\mathsf{r}$, so that ${}^{\mathsf{r}}[\![\tau\to\sigma]\!]_{\theta_\mathsf{r}} = {}^{\mathsf{r}}[\![\tau]\!]_{\theta_\mathsf{r}} \to^{\mathsf{r}} {}^{\mathsf{r}}[\![\sigma]\!]_{\theta_\mathsf{r}}$, as usual.) □

3.5. Constructing relational actions for effects

We have two general principles for constructing monadic relational actions.

3.5.1. Syntax-directed relational actions

Suppose an effect is defined by the same *formal* monad in both the primary and the secondary semantics. (This does not mean that it is interpreted by the same monad in both, because the interpretations of base types, effects, and constants can be different in the two semantics.) Then we can construct a relational action for the effect directly from the syntactic structure of the monad's type constructor:

Lemma 45 (*Bottom-up Relational Action*). *Let effect e' be defined by the formal monad $(\sigma_T, M_\eta, M_\star)$ in both semantics. Then, given relational actions for all effects occurring in σ_T, $T_{\mathsf{r}}^{e'}(R) = {}^{\mathsf{r}}[\![\sigma_T]\!]_{[\chi\mapsto\binom{R}{R}]}$ is a proper relational action for $\langle \mathcal{T}_\iota^{e'}\rangle^{\iota\in\mathrm{I},\shortparallel}$. Moreover, this choice is compatible with relational actions of those $e \preceq e'$ for which the monad morphisms are also constructed syntactically by Eq.* (13)*.*

Proof. First, $T_{\mathsf{r}}^{e'}(R)$ is admissible by construction (Lemma 36). The conditions of Definition 32 then follow directly from the Fundamental Lemma: For part (1), Theorem 44 gives us, when $R \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\shortparallel}$,

$$\langle {}^{\iota}[\![M_\eta]\!]_{[\alpha\mapsto A_\iota]}\,[]\rangle^{\iota\in\mathrm{I},\shortparallel} \in {}^{\mathsf{r}}[\![\alpha\to\sigma_T(\alpha)]\!]_{[\alpha\mapsto\binom{R}{R}]}.$$

Hence, for any $\langle a_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \in R$,

$$\langle \eta_\iota^{e'}\, a_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} = \langle {}^{\iota}[\![M_\eta]\!]_{[\alpha\mapsto A_\iota]}\,[]\, a_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \in {}^{\mathsf{r}}[\![\sigma_T(\alpha)]\!]_{[\alpha\mapsto\binom{R}{R}]} = T_{\mathsf{r}}^{e'}(R).$$

The verification of condition (2) is analogous.

We must also check that the action respects effect-inclusions. Suppose $R \in \mathbf{ARel}\langle A_\iota\rangle^{\iota\in\mathrm{I},\shortparallel}$ and $\langle t_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \in T_{\mathsf{r}}^{e}(R)$; we must check that

$$\langle i_\iota^{e,e'}\, t_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} = \langle {}^{\iota}[\![\sigma_T(\alpha)]\!]^{\mathrm{se}}_{[\alpha\mapsto A_\iota]}\,(t \star_\iota^e \lambda a.\, \eta_\iota^e\,(\eta_\iota^{e'}\, a))\rangle^{\iota\in\mathrm{I},\shortparallel} \in T_{\mathsf{r}}^{e'}(R).$$

By Lemma 37, it suffices to show that

$$\langle t \star_\iota^e \lambda a.\, \eta_\iota^e\,(\eta_\iota^{e'}\, a)\rangle^{\iota\in\mathrm{I},\shortparallel} \in T_{\mathsf{r}}^{e}(T_{\mathsf{r}}^{e'}(R))$$

and by Definition 32(1, 2), this follows if for all $\langle a_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \in R$, $\langle \eta_\iota^{e'}\, a\rangle^{\iota\in\mathrm{I},\shortparallel} \in T_{\mathsf{r}}^{e'}(R)$, which was already established for part (1) above. □

In other words, if an effect is defined the same way in the two semantics, there is no need to do anything special to define its relational action. (This situation is analogous to not having to provide relational interpretations for definable base types, i.e., those that are already denotations of some other type phrase in the language.)

3.5.2. Inverse-image relational actions

As an alternative, we can construct a relational action for an effect from the action of its supereffect, in terms of the monad morphisms between the interpretations:

Lemma 46 (*Top-down Relational Action*). *Given effects $e \preceq e'$ and relational action $T_{\mathsf{r}}^{e'}$ for e', the following is a proper relational action for e:*

$$T_{\mathsf{r}}^{e}(R) = \{\langle t_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \mid \langle i_\iota^{e,e'}\, t_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \in T_{\mathsf{r}}^{e'}(R)\}.$$

Moreover, this choice is compatible with all possible relational actions of effects below e.

Proof. First, we get that $T_{\mathsf{r}}^{e}(R)$ is admissible from admissibility of $T_{\mathsf{r}}^{e'}(R)$ and Lemma 31(3). That it satisfies the two conditions from Definition 32 follows immediately from the monad-morphism laws and the fact that $T_{\mathsf{r}}^{e'}(R)$ is a monadic relational action:

(1) If $\langle a_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \in R$, then

$$\langle i_\iota^{e,e'}\,(\eta_\iota^e\, a_\iota)\rangle^{\iota\in\mathrm{I},\shortparallel} = \langle \eta_\iota^{e'}\, a_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \in T_{\mathsf{r}}^{e'}(R)\,,$$

so $\langle \eta_\iota^e\, a_\iota\rangle^{\iota\in\mathrm{I},\shortparallel} \in T_{\mathsf{r}}^{e}(R)$.

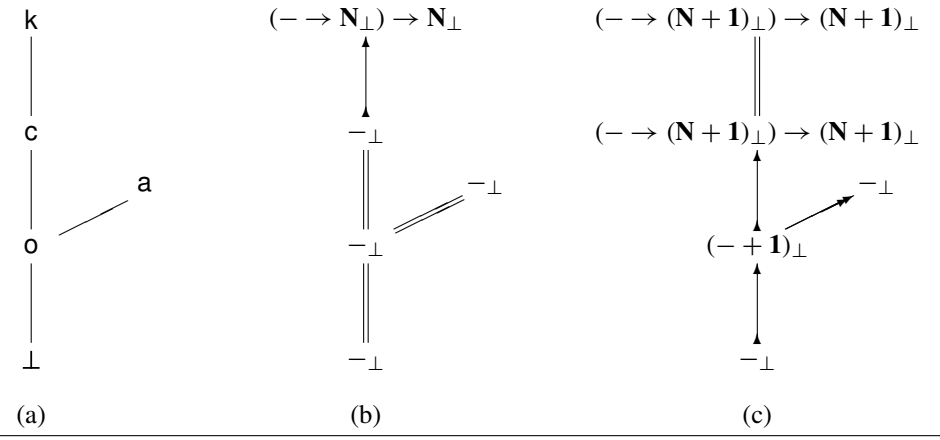


Fig. 10. Effect ordering for the object-language signature (a), with interpretations in primary (b) and secondary (c) semantics.

(2) If $\langle t_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e}(R)$ and $\forall\langle a_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in R.\langle f_\iota\, a_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e}(R)$, then $\langle i_\iota^{e,e'}\, t_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e'}(R)$ and $\forall\langle a_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in R.\langle i_\iota^{e,e'}\,(f_\iota\, a_\iota)\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e'}(R)$, so

$$\langle i_\iota^{e,e'}\,(t_\iota \star_\iota^{e} f_\iota)\rangle^{\iota\in\mathsf{I},\shortparallel} = \langle i_\iota^{e,e'}\, t_\iota \star_\iota^{e'} (i_\iota^{e,e'} \circ f_\iota)\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e'}(R)$$

i.e., $\langle t_\iota \star_\iota^{e} f_\iota\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e}(R)$.

Moreover, if $e^\backprime \preceq e$ and $\langle t_\iota^\backprime\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e^\backprime}(R)$ then

$$\langle i_\iota^{e,e'}\,(i_\iota^{e^\backprime,e}\, t_\iota^\backprime)\rangle^{\iota\in\mathsf{I},\shortparallel} = \langle i_\iota^{e^\backprime,e'}\, t_\iota^\backprime\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e'}(R)$$

by Definition 35 for $e^\backprime \preceq e'$, so $\langle i_\iota^{e^\backprime,e}\, t_\iota^\backprime\rangle^{\iota\in\mathsf{I},\shortparallel} \in T_{\mathsf{r}}^{e}(R)$. □

(More generally, we can take the action of e as the intersection of the inverse-image actions of all (immediate) successors of e in the ordering.)

3.6. Relating interpretations of the object language

In Section 2.7, we constructed two interpretations of the signature used to translate the object language into the metalanguage, such that the meanings of a translated object term coincided with its direct and continuation semantics, respectively. We now aim to construct a relational interpretation of the signature that will allow us to show that the two semantics agree.

Although we could construct the required relational interpretations of o and c directly, it will be illustrative and more convenient to first extend the signature with two additional effects k and a, ordered as in Fig. 10(a). The intended meanings of all effects are now:

- o is still the notion of observable behaviors. We have already chosen to interpret it as divergence (*error-absorbing*) in the primary semantics and divergence-and-errors (*error-reporting*) in the secondary one.
- k is the notion of continuation-passing with respect to observations. It will be interpreted as the denotation of the formal continuation monad with answer type $\langle\mathsf{o}\rangle b_{\mathbf{N}}$ (as in Example 27) in each semantics. (Note these do not actually determine the same *monad*, because o is interpreted differently.) For both interpretations, we also get the inclusion morphism $i^{\mathsf{o},\mathsf{k}}$ as in the example.
- c is the notion of computation used when interpreting computations in the object language. It is taken the same as o in the primary semantics, and as k in secondary one, so that one of the inclusions is trivial in each semantics:

$$\mathcal{T}_{\shortparallel}^{\mathsf{c}} = \mathcal{T}_{\shortparallel}^{\mathsf{o}} \qquad i_{\shortparallel}^{\mathsf{o},\mathsf{c}} = id \qquad i_{\shortparallel}^{\mathsf{c},\mathsf{k}} = i_{\shortparallel}^{\mathsf{o},\mathsf{k}} = \lambda t.\lambda k.\, t \star_{\shortparallel}^{\mathsf{o}} k$$
$$\mathcal{T}_{\shortparallel}^{\mathsf{c}} = \mathcal{T}_{\shortparallel}^{\mathsf{k}} \qquad i_{\shortparallel}^{\mathsf{o},\mathsf{c}} = i_{\shortparallel}^{\mathsf{o},\mathsf{k}} = \lambda t.\lambda k.\, t \star_{\shortparallel}^{\mathsf{o}} k \qquad i_{\shortparallel}^{\mathsf{c},\mathsf{k}} = id.$$

(Had we taken instead $\mathcal{T}_{\mathrm{I}}^{\mathsf{c}} = \mathcal{T}_{\mathrm{I}}^{\mathsf{k}}$, we could have formally related two different *continuation* semantics of the object language: an error-absorbing and an error-reporting one.)

- a is the notion of *abstracted observations*, where both an error return and divergence are understood as failure to deliver a result, like in the original operational semantics. We interpret this effect as simple lifting in both semantics: The monad morphism from observations is the identity in the primary semantics, while it reinterprets errors as divergence in the secondary one:

$$i_{\mathrm{I}}^{\mathsf{o},\mathsf{a}} = id \qquad i_{\mathrm{II}}^{\mathsf{o},\mathsf{a}} = \lambda t.\, t \star^{\perp} \lambda o.\, \mathrm{case}\ o\ \mathrm{of} \begin{Bmatrix} in_1(a) \to \eta^{\perp} a \\ in_2(*) \to \perp \end{Bmatrix}.$$

(It is easy to check that $i_{\mathrm{II}}^{\mathsf{o},\mathsf{a}}$ is indeed a monad morphism.)

Note that, unlike the subeffect coercions we have seen so far, $i_{\mathrm{II}}^{\mathsf{o},\mathsf{a}}$ is not injective: it is an example of projection-style *behavior identification* rather than embedding-style *behavior inclusion*.

These interpretations are summarized in Fig. 10(b,c).

For relations, we take the obvious relation at the base type of natural numbers:

$$\mathcal{B}_{\mathsf{r}}^{b_{\mathbf{N}}} = \{\langle n_{\mathrm{I}} \in \mathbf{N}, n_{\mathrm{II}} \in \mathbf{N}\rangle \mid n_{\mathrm{I}} = n_{\mathrm{II}}\}.$$

We also need to specify relational actions for all the non-$\perp$ effects. It is important that we do this in the proper sequence, as some actions will be derived from earlier ones:

(1) The relational action for a is lifting from Example 33:

$$T_{\mathsf{r}}^{\mathsf{a}}(R) = T_{\mathsf{r}}^{\perp}(R) = \{\langle \perp, \perp\rangle\} \cup \{\langle \lfloor a_{\mathrm{I}} \rfloor, \lfloor a_{\mathrm{II}} \rfloor\rangle \mid \langle a_{\mathrm{I}}, a_{\mathrm{II}}\rangle \in R\}.$$

In particular, if R is the equality relation on A, then $T_{\mathsf{r}}^{\mathsf{a}}(R)$ is equality on $A_{\perp}$.

(2) The relational action for o is the inverse image of the a-action (Lemma 46). That is,

$$\begin{aligned} T_{\mathsf{r}}^{\mathsf{o}}(R) &= \{\langle t_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle i_\iota^{\mathsf{o},\mathsf{a}} t_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in T_{\mathsf{r}}^{\mathsf{a}}(R)\} = \{\langle t_{\mathrm{I}}, t_{\mathrm{II}}\rangle \mid \langle t_{\mathrm{I}}, i_{\mathrm{II}}^{\mathsf{o},\mathsf{a}} t_{\mathrm{II}}\rangle \in T_{\mathsf{r}}^{\perp}(R)\} \\ &= \{\langle \perp, \perp\rangle, \langle \perp, \lfloor in_2(*)\rfloor\rangle\} \cup \{\langle \lfloor a_{\mathrm{I}}\rfloor, \lfloor in_1(a_{\mathrm{II}})\rfloor\rangle \mid \langle a_{\mathrm{I}}, a_{\mathrm{II}}\rangle \in R\}. \end{aligned}$$

(3) The relational action for k is syntactic (based on the action of o), Lemma 45. We first compute the relation on answers:

$$R_{\mathrm{o}} = {}^{\mathsf{r}}[\![\langle \mathsf{o}\rangle b_{\mathbf{N}}]\!]_{[]} = \{\langle \perp, \perp\rangle, \langle \perp, \lfloor in_2(*)\rfloor\rangle\} \cup \{\langle \lfloor n\rfloor, \lfloor in_1(n)\rfloor\rangle \mid n \in \mathbf{N}\} = \{\langle t_{\mathrm{I}}, t_{\mathrm{II}}\rangle \mid t_{\mathrm{I}} = Abs(t_{\mathrm{II}})\}.$$

Then $T_{\mathsf{r}}^{\mathsf{k}}$ is given by

$$\begin{aligned} T_{\mathsf{r}}^{\mathsf{k}}(R) &= {}^{\mathsf{r}}[\![(\alpha \to \langle \mathsf{o}\rangle b_{\mathbf{N}}) \to \langle \mathsf{o}\rangle b_{\mathbf{N}}]\!]_{[\alpha \mapsto \binom{R}{R}]} \\ &= \{\langle t_{\mathrm{I}}, t_{\mathrm{II}}\rangle \mid \forall k_{\mathrm{I}}, k_{\mathrm{II}}.(\forall \langle a_{\mathrm{I}}, a_{\mathrm{II}}\rangle \in R.\langle k_{\mathrm{I}} a_{\mathrm{I}}, k_{\mathrm{II}} a_{\mathrm{II}}\rangle \in R_{\mathrm{o}}) \Rightarrow \langle t_{\mathrm{I}} k_{\mathrm{I}}, t_{\mathrm{II}} k_{\mathrm{II}}\rangle \in R_{\mathrm{o}}\}. \end{aligned}$$

(4) The relational action for c is the inverse image of the k-action (Lemma 46 again):

$$\begin{aligned} T_{\mathsf{r}}^{\mathsf{c}}(R) &= \{\langle t_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle i_\iota^{\mathsf{c},\mathsf{k}} t_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in T_{\mathsf{r}}^{\mathsf{k}}(R)\} = \{\langle t_{\mathrm{I}}, t_{\mathrm{II}}\rangle \mid \langle i_{\mathrm{I}}^{\mathsf{o},\mathsf{k}} t_{\mathrm{I}}, t_{\mathrm{II}}\rangle \in T_{\mathsf{r}}^{\mathsf{k}}(R)\} \\ &= \{\langle t_{\mathrm{I}}, t_{\mathrm{II}}\rangle \mid \forall k_{\mathrm{I}}, k_{\mathrm{II}}.(\forall \langle a_{\mathrm{I}}, a_{\mathrm{II}}\rangle \in R.\langle k_{\mathrm{I}} a_{\mathrm{I}}, k_{\mathrm{II}} a_{\mathrm{II}}\rangle \in R_{\mathrm{o}}) \Rightarrow \langle t_{\mathrm{I}} \star_{\mathrm{I}}^{\mathsf{o}} k_{\mathrm{I}}, t_{\mathrm{II}} k_{\mathrm{II}}\rangle \in R_{\mathrm{o}}\}. \end{aligned}$$

In particular, the relations on σ_D and τ_E become

$$\begin{aligned} {}^{\mathsf{r}}[\![\sigma_D]\!]_{[]} &= T_{\mathsf{r}}^{\mathsf{c}}({}^{\mathsf{r}}[\![\tau_E]\!]_{[]}) \\ &= \{\langle d, d'\rangle \mid \forall k, k'.(\forall \langle e, e'\rangle \in {}^{\mathsf{r}}[\![\tau_E]\!]_{[]}.k\, e = Abs(k'\, e')) \Rightarrow k^*\, d = Abs(d'\, k')\} \end{aligned}$$

$$\begin{aligned} {}^{\mathsf{r}}[\![\tau_E]\!]_{[]} &= \{\langle \phi_\iota s_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle s_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in {}^{\mathsf{r}}[\![b_{\mathbf{N}} + (\langle \mathsf{c}\rangle \tau_E \to \langle \mathsf{c}\rangle \tau_E)]\!]_{[]}\} \\ &= \{\langle \phi_\iota(in_1(n_\iota))\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle n_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in {}^{\mathsf{r}}[\![b_{\mathbf{N}}]\!]_{[]}\} \cup \{\langle \phi_\iota(in_2(f_\iota))\rangle^{\iota\in\mathrm{I},\mathrm{II}} \mid \langle f_\iota\rangle^{\iota\in\mathrm{I},\mathrm{II}} \in {}^{\mathsf{r}}[\![\sigma_D \to \sigma_D]\!]_{[]}\} \\ &= \{\langle \iota_{\mathrm{n}}^{\mathrm{d}}(n), \iota_{\mathrm{n}}^{\mathrm{c}}(n)\rangle \mid n \in \mathbf{N}\} \cup \{\langle \iota_{\mathrm{f}}^{\mathrm{d}}(f), \iota_{\mathrm{f}}^{\mathrm{c}}(f')\rangle \mid \forall \langle d, d'\rangle \in {}^{\mathsf{r}}[\![\sigma_D]\!]_{[]}.\langle f\, d, f'\, d'\rangle \in {}^{\mathsf{r}}[\![\sigma_D]\!]_{[]}\}. \end{aligned}$$

These almost, but not quite, agree with the original specifications for $\sim$ and $\approx$ in Section 1.4; we will return to the difference in Remark 47 below.

To apply the main theorem, we need to check that the interpretations of constants are related:

- It is immediate that, for all $n \in \mathbf{N}$,

$$\langle \mathcal{C}_\iota^{c_n} \rangle^{\iota \in \mathsf{I},\mathsf{II}} = \langle n \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in \mathcal{B}_\mathsf{r}^{b_\mathbf{N}} = {}^\mathsf{r}[\![b_\mathbf{N}]\!]_{[]}.$$

- For c_+, we must check that $\langle \mathcal{C}^{c_+} \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in {}^\mathsf{r}[\![b_\mathbf{N} \times b_\mathbf{N} \to \langle \bot \rangle b_\mathbf{N}]\!]$. So suppose $\langle (n_{1\iota}, n_{2\iota}) \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in {}^\mathsf{r}[\![b_\mathbf{N} \times b_\mathbf{N}]\!]_{[]}$, i.e., $n_{1\mathsf{I}} = n_{1\mathsf{II}}$ and $n_{2\mathsf{I}} = n_{2\mathsf{II}}$. Then also

$$\langle \mathcal{C}_\iota^{c_+}(n_{1\iota}, n_{2\iota}) \rangle^{\iota \in \mathsf{I},\mathsf{II}} = \langle \lfloor n_{1\iota} + n_{2\iota} \rfloor \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in T_\mathsf{r}^\bot(=_\mathbf{N}).$$

- For c_{err}, we must check that $\langle \mathcal{C}_\iota^{c_{\mathrm{err}}} \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in {}^\mathsf{r}[\![\langle \mathsf{o} \rangle 0]\!]_{[]}$. This holds, since

$$\langle \mathcal{C}_\mathsf{I}^{c_{\mathrm{err}}}, \mathcal{C}_\mathsf{II}^{c_{\mathrm{err}}} \rangle = \langle \bot, \lfloor in_2(*) \rfloor \rangle \in \{ \langle \bot, \bot \rangle, \langle \bot, \lfloor in_2(*) \rfloor \rangle \} = T_\mathsf{r}^\mathsf{o}(\emptyset) = {}^\mathsf{r}[\![\langle \mathsf{o} \rangle 0]\!]_{[]}.$$

- For c_{run}, we must check that $\langle \mathcal{C}^{c_{\mathrm{run}}} \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in {}^\mathsf{r}[\![\langle \mathsf{c} \rangle b_\mathbf{N} \to \langle \mathsf{o} \rangle b_\mathbf{N}]\!]$. Assume $\langle t_\iota \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in {}^\mathsf{r}[\![\langle \mathsf{c} \rangle b_\mathbf{N}]\!]_{[]} = T_\mathsf{r}^\mathsf{c}(=_\mathbf{N})$. Take $k_\mathsf{I} = \eta_\mathsf{I}^\mathsf{o}$ and $k_\mathsf{II} = \eta_\mathsf{II}^\mathsf{o}$; then $\forall n \in \mathbf{N}. \langle k_\iota\, n \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in T_\mathsf{r}^\mathsf{o}(=_\mathbf{N}) = R_\mathrm{o}$. Thus, by definition of T_r^c,

$$\langle \mathcal{C}^{c_{\mathrm{run}}}\, t_\iota \rangle^{\iota \in \mathsf{I},\mathsf{II}} = \langle t_\mathsf{I}, t_\mathsf{II}\, \eta_\mathsf{II}^\mathsf{o} \rangle = \langle t_\mathsf{I} \star_\mathsf{I}^\mathsf{o} \eta_\mathsf{I}^\mathsf{o}, t_\mathsf{II}\, \eta_\mathsf{II}^\mathsf{o} \rangle \in R_\mathrm{o} = {}^\mathsf{r}[\![\langle \mathsf{o} \rangle b_\mathbf{N}]\!]_{[]}$$

  as required.

  Finally, applying Theorem 44 to the closed term $\vdash \langle\!| t |\!\rangle^\mathsf{S} : \langle \mathsf{o} \rangle b_\mathbf{N}$, we get

$$\langle [\![t]\!]^\mathrm{D}, [\![t]\!]^\mathrm{C} \rangle = \langle {}^\iota[\![\langle\!| t |\!\rangle^\mathsf{S}]\!][] \rangle^{\iota \in \mathsf{I},\mathsf{II}} \in {}^\mathsf{r}[\![\langle \mathsf{o} \rangle b_\mathbf{N}]\!] = R_\mathrm{o},$$

i.e. $[\![t]\!]^\mathrm{D} = Abs([\![t]\!]^\mathrm{C})$. But that is precisely the statement of Theorem 1, so we are done.

Let us finish by briefly sketching two technical differences between our results and those of the original paper [30]:

Remark 47. The monadic relational action conceptually underlying Reynolds's construction of $\sim$ from $\approx$ in Section 1.4 can be written as follows:

$$T_\mathsf{r}^{\mathsf{c}\prime}(R) = \{ \langle \bot, \lambda k. \bot \rangle, \langle \bot, \lambda k. \lfloor in_2(*) \rfloor \rangle \} \cup \{ \langle \lfloor a_\mathsf{I} \rfloor, \lambda k. k\, a_\mathsf{II} \rangle \mid \langle a_\mathsf{I}, a_\mathsf{II} \rangle \in R \}.$$

This action characterizes continuation-computations *intensionally* by their form, as opposed to the *extensional*, behavior-based characterization used in $T_\mathsf{r}^\mathsf{c}(R)$. Nevertheless, our general construction of the corresponding invariant relation still applies:

We first need to verify that $T_\mathsf{r}^{\mathsf{c}\prime}(R)$ is indeed always an admissible relation when R is admissible. Given Lemma 31, the only non-immediate part is checking that the function $\eta^\mathsf{k} = \lambda a. \lambda k. k\, a$ is an order-monic: suppose $\eta^\mathsf{k} a \sqsubseteq_{(A \to O) \to O} \eta^\mathsf{k} a'$, i.e., $\forall k \in A \to O. k\, a \sqsubseteq_O k\, a'$; we must show that $a \sqsubseteq_A a'$. Let d be an arbitrary element of O such that $d \neq \bot$ (the existence of d relies on O containing at least two elements), and choose k as the (a priori only set-theoretic, but evidently also monotonic) function:

$$k\, x = \begin{cases} \bot & \text{if } x \sqsubseteq_A a' \\ d & \text{otherwise.} \end{cases}$$

We first check that k is actually continuous: consider a chain $(a_i)_{i \in \omega}$ and distinguish whether $\forall i. a_i \sqsubseteq a'$ or not; in both cases, clearly $k(\bigsqcup_i a_i) = \bigsqcup_i k\, a_i$. But then $k\, a \sqsubseteq k\, a' = \bot$, so also $k\, a = \bot$, i.e., $a \sqsubseteq a'$, as required.

It is immediately verified that the action also satisfies the closure conditions of Definition 32, so it can also be used as a basis for recursive definitions of relations, and the proof remains the same. Reynolds's action is a bit tighter: clearly $T_\mathsf{r}^{\mathsf{c}\prime}(R) \subseteq T_\mathsf{r}^\mathsf{c}(R)$, and the inclusion is proper: for example, the pair $\langle \bot, \lambda k. (\lambda a. \lfloor in_2(*) \rfloor)^*(k\, \iota_\mathrm{n}^\mathrm{c}(0)) \rangle$ (i.e., where a failed computation may depend on the continuation) belongs only to the looser relation, but not the tighter one. However, the intensional approach does not seem to generalize easily to effects more complicated than partiality, if one wants to include constants denoting impure functions. □

Remark 48. Reynolds's article goes on to exploit the relations $\sim$ and $\approx$ to construct a *retraction* between the two semantics; that is, it generalizes our function *Abs* to an *abs* on denotations of general terms, and shows that for any t (including open ones), $[\![t]\!]^\mathrm{d} = abs([\![t]\!]^\mathrm{c})$. This construction, however, requires that the domain of final answers O is large enough to contain not only all observable outputs, but also all explicit values E^d, including functional ones.

A similar construction, but restricted to a simply typed setting, is presented by Meyer and Wand [15], and generalized to other notions of effects by Kucan [11]. Unlike Reynolds, however, these use a unary logical relation

(characterizing "well-behaved" continuation-passing meanings independently) rather than a binary one (relating continuation-passing and direct meanings to each other).

A retraction between the two semantics is a powerful result, but arguably stronger than necessary: as long as the semantics agree on observable outputs (possibly modulo further abstraction) of a program, there is no particular reason to require that the relationship between meanings of all subterms also be expressible equationally. And again, the purely retraction-based characterizations do not seem to scale very well to more complicated effect structures and operations.

On the other hand, the idea of choosing a large enough answer type for the continuation monad is actually independent of retractions, but applies more generally when the "answer" returned by a continuation is not necessarily the final, observable outcome of the entire computation, but just an intermediate result, that must be further processed. With this generalization of the continuation concept [5,4] (now usually referred to as *delimited continuations*) one can actually show that *any* definable monad in the primary semantics can be modeled, up to agreement of observables, by a suitable continuation monad in the secondary one [6].

For example, if we extended our sample language with an explicit error-handling construct, we could still define its meaning in our existing, single-continuation semantics, if we allowed ourselves to invoke c_{run}-like operations at arbitrary points, not only at the top level. Since the answer type of a continuation monad must be fixed, but may need to represent intermediate results of arbitrary type, we must first embed all such intermediate-result types into a single, universal cpo U. In the secondary semantics, we can then take $T_{\shortparallel}A = (A \to T_{\shortmid}U) \to T_{\shortmid}U$. (Such a choice of answer type was already proposed by Wadler [36, 3.3–3.4], but without a formal correctness argument.) However, for the purpose of constructing the relational action of the effect, the secondary interpretation is treated essentially as $T_{\shortparallel}A = \forall X.(A \to T_{\shortmid}X) \to T_{\shortmid}X$, where the universal quantification is interpreted as an intersection over all relations of interest, much like in general relational models of parametric polymorphism [31,35]. We omit the details here for reasons of space; the interested reader is referred to the author's PhD dissertation [7] (which does not explicitly cover recursive types, however). □

4. Conclusions and perspectives

With relatively minor adaptations, Reynolds' original schema for showing agreement between two denotational semantics is still highly useful today. We have demonstrated that the technique scales naturally to a general pair of semantics, where computational effects may be realized by different monads, of which one is typically optimized for reasoning, and the other for implementational efficiency. The hope is that the proposed metalanguage-based framework is indeed general enough to account directly for a wide variety of concrete semantics used in practice.

More specifically, we have shown that, under very natural assumptions on the interpretations of effects, the logical-relations construction used for relating them extends smoothly to FPC-style recursive types. That is, as long as the relational framework deals properly with recursive functions at the term level, and recursive types are interpreted in a sensible manner, relations between them do not pose any major further problems. In particular, the naive, circular characterization of the logical relation at μ-types is actually uniquely satisfiable.

There are, of course, a number of ways in which the proposed framework could be developed further. It would be very useful to also extend the account to other kinds of relations, notably those between syntax and semantics, as used to show agreement between a denotational semantics and an operational one [28,25] — or, for that matter, between two operational semantics. The minimal-invariant characterization of recursive types extends fairly smoothly to operational settings [2]; rather, as mentioned in Remark 5, the main challenge would presumably be to give a generic operational account of computational effects that uniformly captured the wide variety of operational-semantics styles (for handling state, exceptions, continuations, non-determinism, etc.) in a way comparable to the unifying role played by monads in denotational settings. A line of work by Plotkin and Power (e.g., [27]) may eventually lead to such an account, but so far it seems better developed for modeling effect-introducing operations (such as *raise* for exceptions) than for effect-delimiting ones (such as *handle*). Another such possibility is Mosses's Modular SOS [20], but it too does not quite seem to encompass the full range of effects expressible by monads, such as first-class continuations.

Two other directions for foundational future work are also worth mentioning. First, it should be possible to extend the framework with a general account of parametric polymorphism [31]; indeed, relational models over domain-theoretic semantics already play a crucial role in such accounts. Also, as apparently first noted by Plotkin, parametric polymorphic types in domain-theoretic settings (with suitable attention to linearity/strictness) effectively subsume

recursive ones [3], although the constructions are rather more involved. A comprehensive account would presumably also need to account for (bounded) effect-polymorphism, i.e., quantifying over effect names, in addition to types.

Second, it would be useful to have a general account of *Kripke logical relations* [17, 8.6.2] in an effect-based setting. We want to model that, in general, the interpretations of effects may evolve as the program is running, such as accounting for dynamically allocated storage cells or exception names in Standard ML [16], or stack-allocated local storage in Algol-like languages [33,21]. For both of these generalizations, it may be advantageous, or even necessary, to abstract from the concrete setting of cpos, into a more explicitly categorical formulation.

Finally, there is still much work to be done in *object* language design and implementation, exploiting the metalanguage constructions and results. Practical programs in languages such as Haskell already rely heavily on monads, and even hierarchies thereof, as a structuring tool [22,13], but so far make little explicit use of the possibility of interpreting the same conceptual effect in different ways. In ML-like settings, where all effects can a priori be considered subeffects of the full space of imperative behaviors expressible in the language, such a distinction is somewhat more natural: the specification semantics given by a programmer as one or more formal monads can be turned into an efficient, direct-style implementation of the monad's effectful operations in terms of the language's native continuation-and-state effects [6,8].

An immediate application of the theory presented here would thus be to formally extend (the correctness proofs for) the latter constructions to object languages with general recursive types, both in main programs and in effect definitions. In the longer term, it should also be possible to provide dedicated object-language support for working with effects, such as concise yet flexible effect-type systems, and seamless integration between code fragments expressed in imperative versus purely functional styles. Needless to say, a proper semantic foundation will be crucial for both programming in, reasoning about, and implementing such languages.

Acknowledgments

I would like to thank the anonymous referees for their insightful observations and suggestions, as well as Olivier Danvy and Kristian Støvring, for helpful ideas on improving the presentation.

References

[1] Martín Abadi, Anindya Banerjee, Nevin Heintze, Jon G. Riecke, A core calculus of dependency, in: Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, ACM Press, San Antonio, TX, January 1999, pp. 147–160.
[2] Lars Birkedal, Robert Harper, Relational interpretations of recursive types in an operational setting, Information and Computation 155 (1–2) (1999) 3–63.
[3] Lars Birkedal, Rasmus E. Møgelberg, Rasmus L. Petersen, Parametric domain-theoretic models of polymorphic intuitionistic/linear lambda calculus, in: Proceedings of the 21st International Conference on Mathematical Foundations of Programming Semantics, in: Electronic Notes in Theoretical Computer Science, vol. 155, 2005, pp. 191–217.
[4] Olivier Danvy, Andrzej Filinski, Abstracting control, in: Proceedings of the 1990 ACM Conference on Lisp and Functional Programming, Nice, France, June 1990, pp. 151–160.
[5] Matthias Felleisen, The theory and practice of first-class prompts, in: Proceedings of the Fifteenth Annual ACM Symposium on Principles of Programming Languages, San Diego, California, January 1988, pp. 180–190.
[6] Andrzej Filinski, Representing monads, in: Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Portland, Oregon, January 1994, pp. 446–457.
[7] Andrzej Filinski, Controlling effects, Ph.D. Thesis, School of Computer Science, Carnegie Mellon University, May 1996. Technical Report CMU–CS–96–119.
[8] Andrzej Filinski, Representing layered monads, in: Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, ACM Press, San Antonio, Texas, January 1999, pp. 175–188.
[9] Marcelo Fiore, Gordon D. Plotkin, An axiomatisation of computationally adequate domain theoretic models of FPC, in: Proceedings of the Ninth Symposium on Logic in Computer Science, Paris, France, 1994, pp. 92–102.
[10] Carl A. Gunter, Semantics of Programming Languages: Structures and Techniques, The MIT Press, 1992.
[11] Jakov Kučan, Retraction approach to CPS transform, Higher-Order and Symbolic Computation 11 (2) (1998) 145–175.
[12] Paul Blain Levy, Call-by-push-value: Decomposing call-by-value and call-by-name, Higher-Order and Symbolic Computation 19 (4) (2006) 377–414.
[13] Sheng Liang, Paul Hudak, Mark Jones, Monad transformers and modular interpreters, in: Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Francisco, California, January 1995, pp. 333–343.
[14] Saunders Mac Lane, Categories for the Working Mathematician, in: Graduate Texts in Mathematics, vol. 5, Springer-Verlag, 1971.
[15] Albert R. Meyer, Mitchell Wand, Continuation semantics in typed lambda-calculi (summary), in: Rohit Parikh (Ed.), Logics of Programs — Proceedings, in: Lecture Notes in Computer Science, vol. 193, Brooklyn, June 1985, pp. 219–224.

[16] Robin Milner, Mads Tofte, Robert Harper, David MacQueen, The Definition of Standard ML, revised edition, The MIT Press, 1997.
[17] John C. Mitchell, Foundations for Programming Languages, The MIT Press, 1996.
[18] Eugenio Moggi, Computational lambda-calculus and monads, in: Proceedings of the Fourth Annual Symposium on Logic in Computer Science, IEEE, Pacific Grove, CA, June 1989, pp. 14–23.
[19] Eugenio Moggi, Notions of computation and monads, Information and Computation 93 (1) (1991) 55–92.
[20] Peter D. Mosses, Modular structural operational semantics, Journal of Logic and Algebraic Programming 60–61 (2004) 195–228.
[21] Frank J. Oles, Type algebras, functor categories, and block structure, in: Maurice Nivat, John C. Reynolds (Eds.), Algebraic Methods in Semantics, Cambridge University Press, 1985.
[22] Simon L. Peyton Jones, Philip Wadler, Imperative functional programming, in: Proceedings of the Twentieth Annual ACM Symposium on Principles of Programming Languages, Charleston, South Carolina, January 1993, pp. 71–84.
[23] Frank Pfenning, Rowan Davies, A judgmental reconstruction of modal logic, Mathematical Structures in Computer Science 11 (4) (2001) 511–540.
[24] Andrew M. Pitts, Evaluation logic, Technical Report 198, University of Cambridge Computer Laboratory, August 1990.
[25] Andrew M. Pitts, Computational adequacy via 'mixed' inductive definitions, in: Mathematical Foundations of Programming Semantics, in: Lecture Notes in Computer Science, vol. 802, Springer-Verlag, April 1993, pp. 72–82.
[26] Andrew M. Pitts, Relational properties of domains, Information and Computation 127 (2) (1996) 66–90.
[27] Gordon Plotkin, John Power, Adequacy for algebraic effects, in: Foundations of Software Science and Computation Structures, 4th International Conference, in: Lecture Notes in Computer Science, vol. 2030, Springer-Verlag, Genova, Italy, April 2001, pp. 1–24.
[28] Gordon D. Plotkin, LCF considered as a programming language, Theoretical Computer Science 5 (3) (1977) 223–255.
[29] John C. Reynolds, Definitional interpreters for higher-order programming languages, in: Proceedings of 25th ACM National Conference, Boston, August 1972, pp. 717–740. Reprinted, with a foreword, in Higher-Order and Symbolic Computation, 11(4) 363–397.
[30] John C. Reynolds, On the relation between direct and continuation semantics, in: Jacques Loeckx (Ed.), 2nd Colloquium on Automata, Languages and Programming, Saarbrücken, West Germany, in: Lecture Notes in Computer Science, vol. 14, July 1974, pp. 141–156.
[31] John C. Reynolds, Types, abstraction and parametric polymorphism, in: R.E.A. Mason (Ed.), Information Processing 83, IFIP, 1983, pp. 513–523.
[32] Dana S. Scott, Domains for denotational semantics, in: Proceedings of the 9th Colloquium on Automata, Languages and Programming, vol. 140, Aarhus, Denmark, July 1982, pp. 577–613.
[33] Ian Stark, Names and higher-order functions, Ph.D. Thesis, University of Cambridge, 1994.
[34] Stephen Tse, Steve Zdancewic, Translating dependency into parametricity, in: Kathleen Fisher (Ed.), Proceedings of the Ninth ACM SIGPLAN International Conference on Functional Programming, ACM Press, Snowbird, Utah, September 2004.
[35] Philip Wadler, Theorems for free!, in: 4th International Conference on Functional Programming and Computer Architecture, London, UK, September 1989, pp. 347–359.
[36] Philip Wadler, The essence of functional programming (invited talk), in: Proceedings of the Nineteenth Annual ACM Symposium on Principles of Programming Languages, Albuquerque, New Mexico, January 1992, pp. 1–14.
[37] Glynn Winskel, The Formal Semantics of Programming Languages: An Introduction, MIT Press, 1993.